

Thought Tree Framework

Contents

Thought Tree Framework.....	1
Contents.....	2
1. Introduction.....	3
2. Strategic Vision: Cognitive Programming.....	5
From Prompting to Cognitive Programming.....	5
From Agents to Structured Cognitive Execution.....	6
Cognitive Programs as Executable Transformation Graphs.....	7
The Cognitive Engine as Compiler and Runtime.....	8
Concepts, Relationships and Transformations.....	9
Hybrid Execution: LLMs, Functions, Tools and Humans.....	10
Separation of Program and Model.....	11
Inspection, Review and Auditability.....	12
Semantic Contracts and Quality Control.....	12
Recursive Composition and Self-Improvement.....	13
Toward Reusable Cognitive Infrastructure.....	14
Strategic Objective.....	15
3. Core Model.....	15
1. Concepts.....	16
2. Data Units.....	17
3. Operations.....	18
4. Relationships.....	21
5. Modules.....	22
6. Collections.....	24
7. Iterators.....	25
8. Contracts.....	26
9. Provenance and Trace.....	27
10. The Cognitive Engine.....	28
11. The Transformation Graph.....	29
12. Summary of the Core Model.....	31
4. System Architecture.....	31
Architectural Layers.....	32
Internal Representation: Cognitive Transformation Graph.....	42
Registries.....	42
Separation of Concerns.....	44
Execution Flow.....	45
Example Architecture in Practice.....	45
Summary.....	46
5. Thought Tree Program Model.....	47
6. Cognitive Engine.....	62
7. Execution Semantics.....	80
8. Semantic Contracts and Validation.....	101

9. TTML Standard.....	115
10. Authoring Thought Tree Modules.....	141
11. Examples.....	161
12. Improvement Process.....	187
13. Commercial Applications.....	200
14. Roadmap.....	212
15. Glossary.....	223

1. Introduction

The Thought Tree Framework is a cognitive programming framework for designing, compiling, executing and improving complex LLM-assisted processes.

It provides a way to describe cognitive work as a structured program made from concepts, relationships, transformations, contracts and outputs. Instead of relying on a single prompt, an unstructured agent conversation or an opaque chain of model calls, a Thought Tree defines the process by which work should be performed: what information is required, what transformations should occur, what intermediate artefacts should be produced, how those artefacts should be reviewed, and what final outputs should be returned.

At the centre of the framework is a simple but powerful abstraction:

Inputs → Transformation → Outputs

A Thought Tree applies this abstraction recursively. Inputs are represented as Data Units: discrete artefacts such as documents, prompts, requirements, plans, reviews, images, structured files or other model-readable content. Transformations are represented as Operations: executable steps that may call an LLM, run a deterministic function, invoke a tool, execute another Module, request human review, or generate a new submodule. Outputs are new Data Units that can be inspected, validated, revised, reused or passed into later Operations.

A group of connected Operations forms a Thought Tree Module. A Module is a reusable cognitive program: it receives input Data Units, performs a defined process, and produces output Data Units. Modules may call other Modules, allowing complex processes to be built from smaller, understandable parts. This recursive structure allows the same framework to describe both simple tasks, such as summarising an article, and large production workflows, such as generating a technical design document, producing game content, preparing an audit pack, or developing a novel from a creative brief.

The framework is designed around a separation between the cognitive program and the systems used to execute it. A Thought Tree Module describes what should happen and how the work is structured. A Cognitive Engine is responsible for compiling and executing that description. The engine loads the Module, resolves inputs and dependencies, expands iterators, validates the execution plan, invokes LLMs or deterministic functions, manages intermediate artefacts, handles errors, applies semantic contracts, records an execution trace and returns the final outputs.

In this model, the Cognitive Engine acts as both compiler and runtime. It turns a human-readable process definition into an executable transformation graph. LLMs are not treated as the whole system; they are one class of execution backend used for ambiguous semantic transformations such as analysis, synthesis, drafting, planning, review and revision. Deterministic functions are used for mechanical operations such as file conversion, concatenation, validation, copying, archiving or API calls. Human reviewers may be included where judgment, approval or correction is required.

This makes the Thought Tree Framework different from ordinary prompt engineering. A prompt asks a model for an answer. A Thought Tree describes a repeatable cognitive process for producing,

checking and improving an answer. The framework, therefore, shifts LLM-assisted work from isolated completions toward structured cognitive software.

The current standard serialisation format for Thought Tree Modules is Thought Tree Markup Language, or TTML. TTML is an XML-based format for representing Modules, inputs, variables, iterators, collections, operations and final outputs. TTML allows Thought Tree programs to be written, shared, validated, executed and stored in a common form. However, TTML is best understood as one representation of the underlying Thought Tree Program Model, not the entirety of the framework. Other authoring formats, visual editors or higher-level languages could be developed in the future while compiling to the same conceptual model.

The purpose of the framework is not to make LLM outputs automatically correct, nor to remove human judgment from important decisions. Its purpose is to make LLM-assisted work more structured, inspectable, modular, reusable and improvable. Every significant step can produce an intermediate artefact. Every artefact can have provenance. Every output can be reviewed against a contract. Every execution can produce a trace. This allows users to understand not only what was produced, but how it was produced.

The framework is especially suited to complex cognitive production tasks where quality, traceability and iteration matter.

Examples include:

- transforming scattered notes into formal documentation;
- generating and reviewing creative media assets;
- producing technical design documents;
- expanding structured game content;
- preparing compliance, policy or audit materials;
- running cyclic research and reporting workflows;
- generating, validating and improving other Thought Tree Modules.

A Thought Tree workflow can be understood before it is run, inspected while it is running, and reviewed after it has completed. Intermediate outputs can be replaced without discarding the whole process. Modules can be reused across projects. Different Cognitive Engines can execute the same program using different LLMs, tools, functions or implementation strategies. Over time, successful Modules can become part of shared libraries of reusable cognitive programs.

The long-term aim of the Thought Tree Framework is to provide a foundation for a new kind of programming: programming with concepts, relationships and transformations. In conventional software, programs operate primarily on formal data structures. In Thought Tree programs, the primary objects may be ideas, documents, plans, requirements, reviews, policies, stories, designs or other semantic artefacts. The Cognitive Engine bridges the gap between these conceptual objects and executable systems.

The Thought Tree Framework is therefore best understood as a modular programming layer for cognitive work: a way to define not just the output desired from an LLM-enabled system, but the structured, inspectable and improvable process by which that output should be produced.

2.Strategic Vision: Cognitive Programming

Large Language Models have made it possible for software systems to perform useful work over ambiguous human concepts: documents, ideas, requirements, plans, stories, arguments, policies, designs and decisions. However, most current uses of LLMs remain centred on individual prompts, chat sessions, prompt chains or loosely controlled agents.

These approaches are powerful, but they often lack the qualities normally expected from production systems:

- explicit structure;
- inspectable intermediate results;
- repeatable execution;
- modular reuse;
- dependency management;
- validation;
- versioning;
- error handling;
- audit trails;
- and clear separation between process definition and execution environment.

The Thought Tree Framework addresses this gap by treating LLM-assisted work as a form of cognitive programming.

A cognitive program does not merely ask an LLM for an answer. It defines a structured process for transforming inputs into outputs through explicit intermediate artefacts, semantic operations, validation steps and execution traces.

In this model, complex cognitive work can be represented as a connected system of:

- concepts — the meaningful entities being worked with;
- relationships — the dependencies and associations between those entities;
- transformations — the operations that analyse, generate, revise, validate or combine them;
- contracts — the expectations that outputs must satisfy;
- artefacts — the concrete Data Units produced during execution;
- traces — the recorded history of how each artefact was created.

The strategic aim of the Thought Tree Framework is to make this kind of cognitive work programmable, inspectable, reusable and improvable.

From Prompting to Cognitive Programming

A prompt is an instruction given to a model. A cognitive program is a structured process for producing, validating and refining a result.

A single prompt may be sufficient for a simple task, such as summarising a short article or generating a paragraph of text. But larger tasks usually require more than one model response. They require planning, decomposition, intermediate outputs, review, revision, aggregation and final validation.

For example, producing a technical design document from scattered notes may require many distinct cognitive steps:

- normalise the source material;
- extract requirements;
- identify assumptions and unresolved questions;
- decompose the system;
- draft individual document sections;
- review the draft for gaps and unsupported claims;
- revise the document;
- validate the final output against expected criteria.

This is not just a prompt. It is a process.

The Thought Tree Framework allows that process to be written down, executed, inspected and improved.

The shift is therefore:

Prompting:

User → Prompt → LLM → Output

Cognitive Programming:

Inputs → Transformations → Intermediate Artefacts → Validation → Final Outputs → Execution Trace

The framework does not remove the value of prompts. Instead, it places prompts inside a broader executable structure.

Prompts become one implementation detail within a cognitive program, rather than the whole program itself.

From Agents to Structured Cognitive Execution

Autonomous agents attempt to solve complex tasks by allowing an LLM to plan, act, observe and continue iterating. This can be useful, but unconstrained agent behaviour is often difficult to predict, debug or trust.

Agents may:

- lose track of earlier decisions;
- repeat work unnecessarily;
- misuse tools;
- fail to preserve intermediate reasoning artefacts;
- make hidden assumptions;
- drift away from the original objective;
- or produce outputs that are difficult to audit.

The Thought Tree Framework takes a different approach.

Instead of relying on an LLM to remember and manage an entire process implicitly, the process is made explicit. The Cognitive Engine manages execution state, dependencies, artefacts, validation, errors and traceability. The LLM is used where it is strongest: performing semantic transformations over ambiguous or concept-rich material.

This creates a hybrid model:

- LLMs handle ambiguity.
- Code handles structure.
- The Cognitive Engine coordinates both.

The goal is not to make LLMs act like traditional software. Nor is it to make traditional software solve ambiguous semantic problems by itself. The goal is to combine them in a structured execution model where each part does what it is best suited to do.

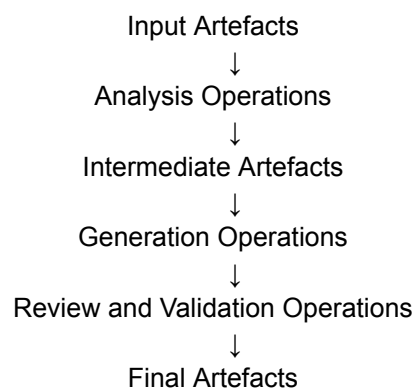
Cognitive Programs as Executable Transformation Graphs

A Thought Tree program describes how Data Units are transformed into other Data Units.

At the simplest level, this follows the pattern:

Inputs → Operation → Outputs

However, when operations are connected together, this becomes a larger transformation structure:



Although the framework uses the term “Thought Tree”, advanced workflows may be better understood as directed transformation graphs. Operations depend on earlier artefacts. Artefacts may feed multiple later operations. Modules may call other Modules. Iterators may expand one operation into many concrete executions. Collections may aggregate many outputs into one logical input. Review operations may create feedback that drives revision.

The “tree” metaphor remains useful because complex work is often decomposed into smaller branches of work. But the execution model is more general: a Thought Tree program is a structured graph of cognitive transformations.

This graph-like structure makes the process:

- easier to inspect before execution;

- easier to execute reliably;
- easier to parallelise where safe;
- easier to debug when failures occur;
- easier to revise when requirements change;
- and easier to reuse across projects.

The Cognitive Engine as Compiler and Runtime

In traditional programming, source code is parsed, validated, compiled or interpreted, and executed by a runtime environment.

The Thought Tree Framework applies a similar pattern to cognitive work.

A Thought Tree definition describes what should happen. The Cognitive Engine determines how to execute it.

The Cognitive Engine is responsible for:

- loading a Thought Tree definition;
- validating its structure;
- resolving inputs, variables, iterators and dependencies;
- expanding abstract operations into concrete operation instances;
- detecting output collisions;
- selecting appropriate execution strategies;
- invoking LLMs, deterministic functions, tools or submodules;
- managing a workspace of artefacts;
- validating outputs against contracts;
- handling errors and retries;
- preserving intermediate outputs;
- and recording an execution trace.

In this sense, the Cognitive Engine acts as both compiler and runtime.

It compiles a human-readable cognitive program into an executable plan, then runs that plan using the available execution backends.

Those backends may include:

- LLMs;
- deterministic functions;
- external APIs;
- search tools;
- databases;
- file systems;
- generated submodules;
- pre-existing modules;
- and human review steps.

The LLM is therefore not the whole system. It is one execution capability used by the Cognitive Engine.

This distinction is important. The Cognitive Engine provides the structure, control and traceability. The LLM provides semantic flexibility.

Concepts, Relationships and Transformations

Traditional software usually operates on formal data structures: numbers, strings, records, objects, files, tables and messages.

Cognitive programming operates on meaningful artefacts: briefs, plans, summaries, requirements, chapters, reviews, policies, outlines, analyses, character profiles, design documents and other concept-bearing units of work.

The Thought Tree Framework does not require these concepts to be reduced entirely to rigid schemas before work can begin. Instead, it allows ambiguous human-readable artefacts to be processed in a structured way.

A concept is a meaningful entity within a workflow.

Examples include:

- a story premise;
- a character profile;
- a plot outline;
- a technical requirement;
- a risk register;
- a policy obligation;
- a game mechanic;
- a design note;
- a review report.

A relationship describes how concepts connect.

Examples include:

- this output is derived from that input;
- this review evaluates that draft;
- this revision incorporates that feedback;
- this chapter depends on that plot outline;
- this requirement is supported by that source note;
- this final document aggregates these sections.

A transformation is an operation that changes, creates, evaluates or combines concepts.

Examples include:

- extract requirements from notes;
- draft a plot outline from a premise;
- review a chapter for continuity;
- revise a document using feedback;

- validate XML against a schema;
- compile multiple sections into a complete manuscript;
- generate a submodule to complete a complex task.

This gives the framework its central strategic identity:

Thought Tree programs are programs made from concepts, relationships and transformations.

Hybrid Execution: LLMs, Functions, Tools and Humans

Not every operation in a cognitive workflow should be performed by an LLM.

Some tasks are semantic and ambiguous. These are well suited to LLMs.

Examples:

- summarise this document;
- infer requirements from these notes;
- draft a chapter;
- identify contradictions;
- propose a plan;
- review this output against a rubric.

Other tasks are mechanical and deterministic. These should be performed by ordinary software functions.

Examples:

- concatenate files;
- validate XML;
- convert Markdown to HTML;
- copy an asset;
- calculate a checksum;
- query a database;
- resize an image;
- archive outputs.

Other tasks may require tools or external systems.

Examples:

- search a document store;
- retrieve similar examples from a vector database;
- call an API;
- run a build process;
- generate an image;
- render a report.

Some tasks may require human judgement.

Examples:

- approve a generated plan;
- resolve conflicting requirements;
- assess creative quality;
- approve regulated content;
- decide whether a workflow revision is acceptable.

The Thought Tree Framework is designed to coordinate all of these forms of execution.

This hybrid approach is strategically important because it avoids two common mistakes:

- using LLMs for tasks that deterministic software can already do more reliably;
- forcing deterministic software to handle ambiguity that LLMs are better suited to interpret.

The Cognitive Engine decides, or is instructed, which kind of execution is appropriate for each operation.

Separation of Program and Model

One of the central aims of the Thought Tree Framework is to separate the cognitive workflow from the specific model used to execute it.

A Thought Tree Module should describe:

- what inputs are required;
- what transformations should be performed;
- what outputs should be produced;
- what contracts should be satisfied;
- and what dependencies exist between steps.

It should not depend unnecessarily on a specific LLM provider.

This allows the same cognitive program to be executed in different environments:

- with different LLM providers;
- with different model sizes;
- with local or remote models;
- with different cost and quality settings;
- with or without human review;
- with different deterministic function libraries;
- or under different organisational policies.

This separation is similar to the way traditional source code can be compiled or interpreted in different environments. The source program defines intent and structure. The execution environment determines the concrete implementation strategy.

This makes Thought Tree Modules more reusable, portable and future-proof.

As LLMs improve, the same cognitive programs may produce better results without needing to be completely rewritten.

Inspection, Review and Auditability

LLM outputs are not automatically correct. The framework should not pretend otherwise.

The purpose of cognitive programming is not to remove human judgement or guarantee perfect outputs. The purpose is to make the process of producing those outputs more structured, inspectable and improvable.

A Thought Tree execution preserves intermediate artefacts so that users can see how a final output was produced.

For example, a final technical design document may be traced back through:

- source notes;
- source digest;
- requirements register;
- system decomposition;
- drafted sections;
- review report;
- correction plan;
- final revision.

This allows errors to be found and corrected at the level where they entered the process.

If a final output is weak, the user does not need to rerun an opaque prompt and hope for improvement. They can inspect the workflow, identify the weak operation, revise the instruction, replace an intermediate artefact, add a review step, strengthen a contract or change the execution strategy.

This is a major advantage over unstructured chat-based production.

The framework aims to make cognitive work auditable in the same way that software builds, data pipelines and document workflows can be audited.

Semantic Contracts and Quality Control

Producing an output file is not enough. The output must be fit for its intended purpose.

A cognitive program therefore needs more than execution correctness. It needs semantic correctness.

The Thought Tree Framework distinguishes between three levels of correctness:

- **Schema correctness**
The module definition is structurally valid.
- **Execution correctness**

The Cognitive Engine can resolve dependencies, execute operations and produce the declared outputs.

- **Semantic correctness**

The produced outputs satisfy the content, structure, quality and purpose required by the workflow.

Semantic contracts allow Data Units to define what they are expected to contain.

A contract may specify:

- expected semantic type;
- required sections;
- required format;
- completeness criteria;
- quality criteria;
- downstream usage expectations;
- deterministic validation functions;
- LLM-assisted review rubrics;
- and optional human approval requirements.

For example, a PlotOutline may be required to include a premise, major acts, turning points, climax and resolution. A TechnicalRequirementsRegister may be required to distinguish explicit requirements, inferred requirements and unresolved questions. A ComplianceGapAnalysis may require traceability to source obligations.

Contracts make quality control explicit.

They allow the Cognitive Engine to check not only whether an output exists, but whether it is suitable for use by later operations.

Recursive Composition and Self-Improvement

A key strategic feature of the framework is recursive modularity.

A Module is itself a structured artefact. It can be used, reviewed, generated or improved by other Modules.

This enables several powerful patterns:

- an operation can call a pre-existing Module;
- an operation can generate a new Module;
- a Module can review the output of another Module;
- a Module can validate another Module;
- a Module can improve another Module;
- a library of Modules can evolve over time.

This creates the possibility of self-improving cognitive workflows.

For example, a Thought Tree Module can be created to:

- inspect an existing TTML Module;
- validate it against the schema;
- validate it against execution rules;
- review its design;
- generate test cases;
- compare expected behaviour against actual behaviour;
- produce an improved version;
- summarise the changes.

This does not mean the system becomes magically self-correcting. Human review may still be required, especially for high-impact workflows. But it does mean the framework can apply its own principles — decomposition, review, validation, revision and traceability — to its own modules.

This is strategically significant because it allows module libraries to improve over time.

Toward Reusable Cognitive Infrastructure

The long-term value of the Thought Tree Framework is not only in executing individual workflows. It is in creating reusable cognitive infrastructure.

Organisations repeatedly perform cognitive processes such as:

- reviewing documents;
- extracting requirements;
- producing reports;
- generating plans;
- analysing risks;
- preparing compliance evidence;
- summarising research;
- standardising legacy material;
- drafting communications;
- reviewing creative content;
- producing design documents.

Many of these processes are currently performed through ad hoc human effort, informal prompting or bespoke automation.

Thought Tree Modules allow these processes to be captured as reusable cognitive programs.

Over time, organisations could build libraries of modules that encode their preferred ways of working:

- their document standards;
- their review methods;
- their risk analysis processes;
- their creative development pipelines;
- their compliance procedures;
- their technical documentation patterns;
- their quality criteria.

These modules could be versioned, tested, reviewed, improved and shared.

A public ecosystem of modules could also emerge, allowing reusable cognitive processes to be exchanged in the same way software packages, workflow templates or automation recipes are exchanged today.

Strategic Objective

The strategic objective of the Thought Tree Framework is to move LLM-assisted work from isolated prompting and opaque automation toward structured cognitive software.

The framework aims to make complex cognitive production:

- **programmable** — because processes can be explicitly defined;
- **modular** — because workflows can be decomposed and reused;
- **model-independent** — because process definitions are separated from model providers;
- **hybrid** — because LLMs, functions, tools and humans can work together;
- **inspectable** — because intermediate artefacts are preserved;
- **validatable** — because outputs can be checked against contracts;
- **traceable** — because execution history is recorded;
- **improvable** — because modules and outputs can be reviewed and revised;
- **shareable** — because modules can be serialised and exchanged;
- **scalable** — because large cognitive projects can be built from smaller executable parts.

In this sense, the Thought Tree Framework is not simply a workflow format for LLMs.

It is an early model for cognitive programming: a way to define and execute programs whose primary objects are ideas, documents, requirements, plans, reviews, concepts and transformations between them.

3. Core Model

The Thought Tree Framework is based on a simple but powerful model:

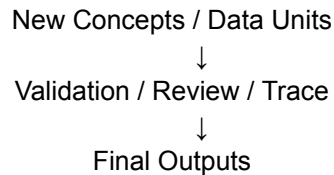
- Concepts are represented as Data Units.
- Operations transform Data Units into new Data Units.
- Relationships connect Data Units, Operations and Modules.
- The Cognitive Engine compiles and executes these transformations.

At its smallest scale, the framework follows the pattern:

Inputs → Operation → Outputs

At larger scales, this pattern becomes a connected cognitive program:

Concepts / Data Units
↓
Transformations / Operations
↓



A Thought Tree program therefore does not merely ask an LLM for an answer. It describes a structured process by which meaningful artefacts are created, transformed, reviewed, validated and assembled.

The core model consists of six primary elements:

- Concepts
- Data Units
- Operations
- Relationships
- Modules
- The Cognitive Engine

Together, these elements allow complex cognitive work to be represented as an executable structure.

1. Concepts

A Concept is a meaningful object or idea within a cognitive workflow.

Examples of concepts include:

- a character profile;
- a plot outline;
- a technical requirement;
- a design constraint;
- a risk;
- a compliance obligation;
- a chapter draft;
- a review report;
- a policy summary;
- a game mechanic;
- a user persona;
- a test case;
- a generated Thought Tree Module.

Concepts are the things the workflow reasons about, creates, transforms or evaluates.

In ordinary programming, a program may primarily manipulate numbers, strings, objects, files or database records. In the Thought Tree Framework, a cognitive program manipulates meaningful conceptual artefacts.

For example:

Story Requirements → Character Profiles → Chapter Drafts → Revised Manuscript

or:

Design Notes → Requirements Register → System Decomposition → Technical Design Document

or:

Policy Documents → Obligation Register → Evidence Map → Audit Preparation Pack

The framework does not require a Concept to be represented in only one way. A concept may be stored as plain text, Markdown, XML, JSON, an image, a database record, a vector search result, or another implementation-specific asset.

What matters is that the workflow can identify the concept, pass it between operations, transform it and trace its use.

2. Data Units

A Data Unit is the concrete representation of a Concept within a Thought Tree program.

A Data Unit is a discrete artefact that can be supplied to, produced by, referenced by, reviewed by or transformed by an Operation.

In the current TTML format, Data Units are commonly represented as files:

```
<File id="plot_outline" extension="txt"/>
```

or referenced using:

```
<FileRef id="plot_outline"/>
```

However, the theoretical model is broader than files. A Data Unit may represent:

- a text document;
- a Markdown section;
- a JSON object;
- an XML document;
- a prompt;
- a list of requirements;
- a database query result;
- an image;
- an audio file;
- a video file;
- a vector search result;
- a generated module;
- a validation report;
- a human review decision;
- an in-memory value.

A Data Unit should be understood as a logical artefact, not merely a physical file.

A Data Unit may have:

- an identifier;
- a content body;
- a physical representation;
- a semantic type;
- a contract;
- a version;
- a provenance record.

For example:

Identifier: plot_outline Physical representation: plot_outline.txt Semantic type: PlotOutline Contract: Must include premise, acts, turning points, climax and resolution Produced by: GeneratePlotOutline Inputs used: story_requirements, character_system_brief, setting_overview Version: 1

This distinction is important.

The file plot_outline.txt is the storage representation.

The semantic type PlotOutline describes what the artefact means in the cognitive program.

The contract describes what must be true for the artefact to be considered valid.

3. Operations

An Operation is a transformation applied to one or more input Data Units to produce one or more output Data Units.

The basic operation model is:

Input Data Units → Operation → Output Data Units

For example:

story_requirements → AnalyzeRequirements → requirements_analysis

or:

draft_chapter + beta_reader_feedback → ReviseChapter → revised_chapter

or:

tdd_sections → ConcatenateFiles → technical_design_document

Operations are the active units of a Thought Tree program. They describe what should happen to the available Data Units.

An Operation may be performed by:

- an LLM;
- a deterministic software function;
- a tool;
- a submodule;
- a human reviewer;
- a generated execution plan;
- or a combination of these.

The current TTML draft includes several Operation types:

- TextCompletion
- DynamicCompletion
- ProjectCompletion
- PreExisting
- ExecuteFunction

These can be understood as different kinds of transformation.

3.1 LLM Operations

An LLM Operation uses a language model to perform ambiguous or semantic work.

Examples include:

- analyse;
- summarise;
- classify;
- extract;
- draft;
- review;
- revise;
- compare;
- synthesize;
- plan;
- generate alternatives.

Example:

```
<Operation id="DraftPlotOutline" type="TextCompletion" desc="Draft a coherent plot outline from the story requirements.">
  <FileRef id="story_requirements"/>
  <Output>
    <File id="plot_outline" extension="txt"/>
  </Output>
</Operation>
```

The Operation description defines the intended transformation. The input Data Units provide context. The output declaration defines the artefact that should be produced.

3.2 Deterministic Operations

A deterministic Operation uses ordinary software code rather than an LLM.

Examples include:

- copy a file;
- concatenate files;
- validate XML;
- convert a document to plain text;
- generate a checksum;
- resize an image;
- query a database;
- archive outputs;
- transform JSON;
- run a test suite.

Example:

```
<Operation id="CompileNovel" type="ExecuteFunction" desc="Concatenate revised chapter files into one manuscript."
function="ConcatenateFiles">
  <CollectionRef id="all_revised_chapters"/>
  <Output>
    <File id="compiled_novel_manuscript" extension="txt"/>
  </Output>
</Operation>
```

Deterministic Operations are important because they prevent LLMs from being used for tasks that ordinary software can perform more reliably.

The framework is therefore hybrid by design:

- LLMs perform semantic transformations.
- Code performs deterministic transformations.

Humans perform judgement and approval where required.

3.3 Module Operations

An Operation may execute another Module.

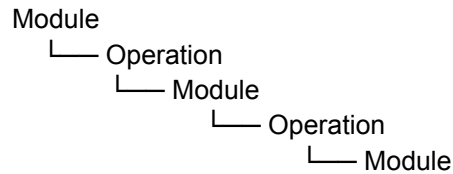
This allows Thought Tree programs to be recursively composed.

For example:

```
<Operation id="ConceptDevelopment" type="PreExisting" desc="Run the concept development subproject."
target="/subprojects/ConceptDevelopment.html">
  <FileRef id="story_requirements"/>
  <Output>
    <File id="plot_outline" extension="txt"/>
    <File id="setting_overview" extension="txt"/>
  </Output>
</Operation>
```

The parent Module treats the submodule as an Operation, while the submodule may itself contain many smaller Operations.

This creates a recursive structure:



This is one of the key strengths of the framework. It allows high-level cognitive work to be decomposed into reusable, inspectable sub-processes.

3.4 Dynamic Operations

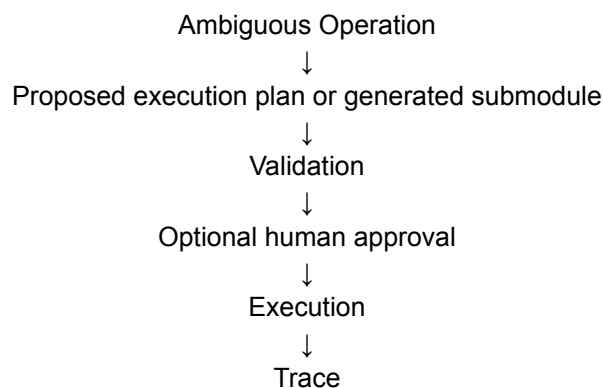
Some Operations may be too ambiguous or complex to define fully in advance.

For these cases, the framework supports dynamic execution patterns such as:

- deciding whether an Operation should be a direct LLM completion;
- generating a new submodule;
- selecting a pre-existing module;
- decomposing a task into smaller operations;
- creating a bespoke review or validation process.

This allows a Thought Tree program to handle ambiguity without becoming an uncontrolled agent.

The preferred execution pattern for dynamic work is:



Dynamic behaviour should remain inspectable. If the Cognitive Engine generates a submodule or chooses an execution strategy, that decision should be recorded in the execution trace.

4. Relationships

A Relationship connects concepts, Data Units, Operations and Modules.

Relationships are what turn isolated operations into a coherent cognitive program.

Common relationships include:

- uses
- produces
- depends on
- derived from
- reviews
- revises
- validates
- aggregates
- decomposes
- implements
- references
- replaces
- version of

For example:

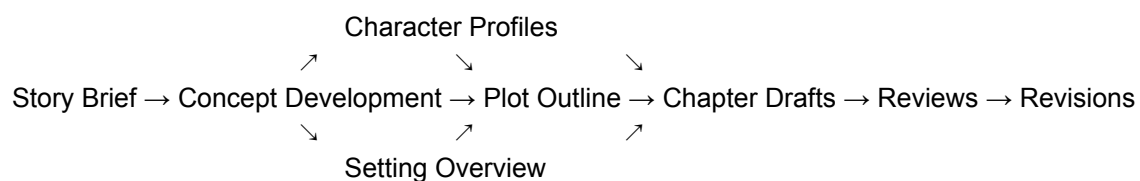
- DraftChapter uses PlotOutline.
- DraftChapter produces DraftChapter_1.
- ReviewChapter reviews DraftChapter_1.
- ReviseChapter revises DraftChapter_1 using ReviewChapter_1.
- FinalManuscript aggregates RevisedChapter_1 to RevisedChapter_12.

These relationships can be represented through FileRef, CollectionRef, operation outputs, module calls, execution traces and provenance metadata.

In a simple linear workflow, the relationships may look like this:

Input → Analyze → Draft → Review → Revise → Final Output

In a larger workflow, the structure becomes a graph:



Although the framework is called the Thought Tree Framework, advanced workflows may form directed graphs rather than strict trees. This is especially true when workflows use loops, conditional execution, shared dependencies, review cycles, generated submodules or reusable intermediate artefacts.

The tree metaphor remains useful for visualising decomposition, but the execution model is better understood as a connected transformation graph.

5. Modules

A Module is a reusable cognitive program.

A Module defines:

- required inputs;
- variables;
- iterators;
- collections;
- ordered operations;
- intermediate outputs;
- final outputs;
- optional contracts;
- optional validation or review stages.

In TTML, a Module is represented by a single TTML document.

At a conceptual level, a Module is similar to a function, procedure or workflow in ordinary programming.

It receives inputs:

- story_requirements

performs operations:

- AnalyzeRequirements
- DevelopCharacters
- DraftPlotOutline
- ReviewConcepts
- ReviseConcepts

and returns outputs:

- character_profiles
- setting_overview
- plot_outline

A Module can be small:

- Summarise one article.

or large:

- Generate a complete technical design document from design notes.

or recursive:

- Generate a new Thought Tree Module to complete a complex operation.

The same core structure applies at every scale:

Inputs → Operations → Outputs

This recursive simplicity is central to the framework.

6. Collections

A Collection is a named group of Data Units.

Collections allow multiple related artefacts to be treated as one logical input or output.

For example:

```
<Collection id="all_revised_chapters" orderBy="ChapterIterator">
  <FileRef id="revised_chapter_{{ChapterIterator}}"/>
</Collection>
```

This Collection may represent:

```
revised_chapter_1
revised_chapter_2
revised_chapter_3
...
revised_chapter_12
```

The Collection can then be passed into an Operation:

```
<CollectionRef id="all_revised_chapters"/>
```

Collections are useful for:

- grouping generated artefacts;
- passing many files into one Operation;
- compiling a document from sections;
- aggregating feedback;
- reviewing a set of outputs;
- avoiding unintended iterator expansion;
- making grouped dependencies explicit.

Collections are important because cognitive work often involves sets of related artefacts rather than isolated files.

For example:

- All character profiles
- All chapter drafts
- All beta reader feedback reports
- All requirements
- All risks
- All test case reviews
- All policy documents

A Collection gives the framework a way to reason about such groups directly.

7. Iterators

An Iterator defines repeated execution over a range of values.

For example:

```
<Iterator id="ChapterIterator" from="1" to="{{ChapterCount}}"/>
```

If ChapterCount is 12, the iterator represents:

```
1, 2, 3, ..., 12
```

An Operation can use this iterator to generate repeated outputs:

```
<File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
```

This produces:

```
draft_chapter_1  
draft_chapter_2  
...  
draft_chapter_12
```

Iterators allow a compact Module definition to represent many concrete operation instances.

For example:

```
DraftChapter[1]  
DraftChapter[2]  
DraftChapter[3]  
...  
DraftChapter[12]
```

Iterators are especially useful for:

- chapters;
- scenes;
- characters;
- test cases;
- design sections;
- documents;
- personas;
- requirements;
- risks;
- assets;
- variants.

When multiple iterators are used together, the Cognitive Engine may expand them into combinations of values. This allows the framework to express patterns such as:

- For each chapter, simulate feedback from each beta reader persona.

which becomes:

Chapter 1, Persona 1
Chapter 1, Persona 2
Chapter 1, Persona 3
Chapter 2, Persona 1
...

Because iterator expansion can create many operation instances, the Cognitive Engine must validate the expansion before execution. In particular, it should detect output collisions, unresolved dependencies and unintended multi-writer outputs.

8. Contracts

A Contract defines what a Data Unit or Operation output must satisfy in order to be considered valid.

Schema validation can confirm that a Thought Tree Module is structurally valid, but it cannot confirm that the generated content is useful, complete or correct.

For example, a workflow may successfully produce a file called:

plot_outline.txt

but that file may still be too vague, incomplete or inconsistent to function as a real plot outline.

A Contract addresses this problem by defining semantic expectations.

A Contract may specify:

- expected semantic type;
- required sections;
- required structure;
- formatting rules;
- completeness criteria;
- quality criteria;
- prohibited content;
- traceability requirements;
- validation functions;
- LLM review rubrics;
- human approval requirements.

For example, a PlotOutline contract might require:

- premise;

- main characters;
- setting summary;
- act structure;
- inciting incident;
- major turning points;
- climax;
- resolution;
- unresolved continuity notes.

A TechnicalRequirementsRegister contract might require:

- functional requirements;
- non-functional requirements;
- platform requirements;
- assumptions;
- dependencies;
- risks;
- open questions;
- source traceability.

Contracts allow the framework to distinguish between three levels of correctness:

- **Schema correctness**
The module definition is structurally valid.
- **Execution correctness**
The Cognitive Engine can execute the operations and produce the declared outputs.
- **Semantic correctness**
The produced outputs satisfy the meaning and quality requirements needed by the workflow.

A mature Cognitive Engine should be able to validate outputs against contracts and trigger repair, review or failure when contracts are not satisfied.

9. Provenance and Trace

A Trace records what happened during execution.

A Provenance record describes where a Data Unit came from and how it was produced.

This is essential because LLM-assisted work can otherwise become opaque. The Thought Tree Framework is designed to preserve not only final outputs, but also the process that produced them.

For each Operation, the execution trace may record:

- Module ID;
- operation ID;
- operation type;
- input Data Units;
- output Data Units;
- variable values;

- iterator values;
- model used;
- model settings;
- generated prompt;
- function called;
- tool called;
- start and end time;
- validation results;
- errors;
- retries;
- human review decisions;
- output hashes.

For each Data Unit, provenance may record:

- producing Operation;
- source inputs;
- previous versions;
- semantic type;
- contract;
- validation status;
- timestamp;
- model or function used;
- relationship to downstream outputs.

This makes Thought Tree workflows:

- inspectable;
- auditable;
- debuggable;
- repeatable;
- improvable.

The goal is not always perfect deterministic reproduction, because LLM outputs may vary between runs. The goal is to make the process explicit enough that it can be understood, reviewed, repeated and improved.

10. The Cognitive Engine

The Cognitive Engine is the compiler and runtime for Thought Tree programs.

It takes a source definition, such as a TTML file, and turns it into executable work.

The Cognitive Engine is responsible for:

- loading the source Module;
- validating its structure;
- resolving inputs, variables, iterators and references;
- compiling the Module into an executable transformation graph;
- detecting dependency issues and output collisions;
- planning execution;

- invoking LLMs, functions, tools, submodules or human review;
- validating outputs;
- storing intermediate artefacts;
- resolving final outputs;
- recording the execution trace.

The Cognitive Engine should not be understood as merely a prompt runner. It is the system that enforces the semantics of the framework.

The LLM is not the whole engine. The LLM is one execution backend used by the engine for ambiguous semantic transformations.

A Thought Tree execution may involve:

- LLM completions
- deterministic functions
- external tools
- submodule calls
- validation gates
- human review
- trace recording

The Cognitive Engine coordinates these components.

This separation is central to the framework:

- Thought Tree Program = what should happen
- Cognitive Engine = how it is compiled and executed
- LLM / tools / functions / humans = execution resources
- Workspace = where artefacts and traces are stored

Because the program is separate from the model, different Cognitive Engines may execute the same Thought Tree program using different LLM providers, model versions, tools, storage systems or runtime policies.

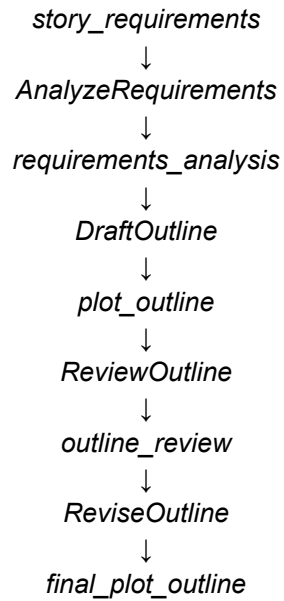
11. The Transformation Graph

Although Thought Tree Modules may be visualised as trees, the execution structure is more accurately described as a Cognitive Transformation Graph.

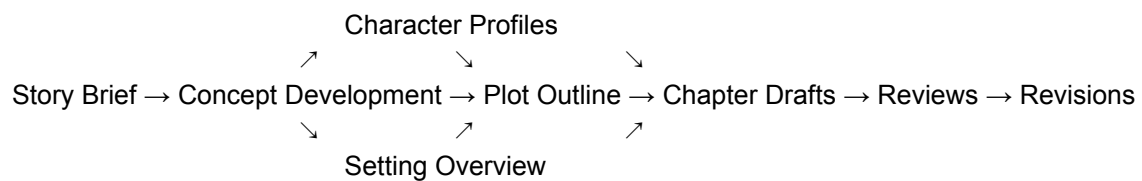
In this graph:

- Data Units are artefact nodes;
- Operations are transformation nodes;
- Collections are grouped artefact nodes;
- Modules are reusable subgraphs;
- relationships are edges;
- contracts are validation rules;
- traces record execution events.

A simple graph may look like this:



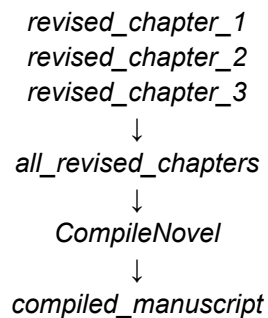
A larger graph may branch and merge:



It may also expand through iterators:

DraftChapter[1] → draft_chapter_1
 DraftChapter[2] → draft_chapter_2
 DraftChapter[3] → draft_chapter_3
 ...

And it may aggregate outputs through Collections:

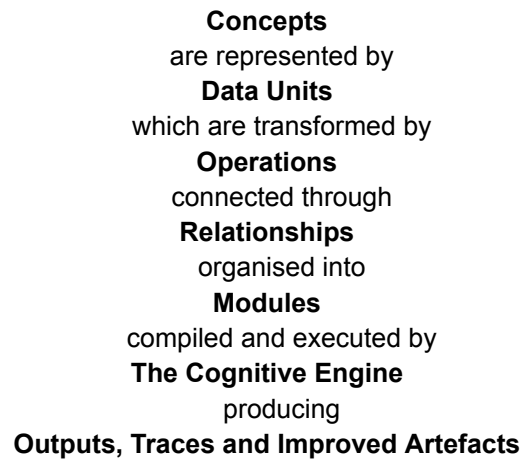


This graph-based view allows the Cognitive Engine to reason about dependencies, parallel execution, validation, provenance, caching and final output resolution.

12. Summary of the Core Model

The Thought Tree Framework represents cognitive work as executable transformations over meaningful artefacts.

The core model can be summarised as follows:



Or more simply:

Data Units → Operations → Data Units

At small scale, this is a single LLM-assisted task.

At large scale, it becomes a modular cognitive program.

The purpose of the framework is to make these programs:

- explicit;
- inspectable;
- reusable;
- model-independent;
- auditable;
- validatable;
- improvable.

This is the foundation of Thought Tree as a cognitive programming framework: a way to program not only with data and functions, but with concepts, relationships, transformations, contracts and traces.

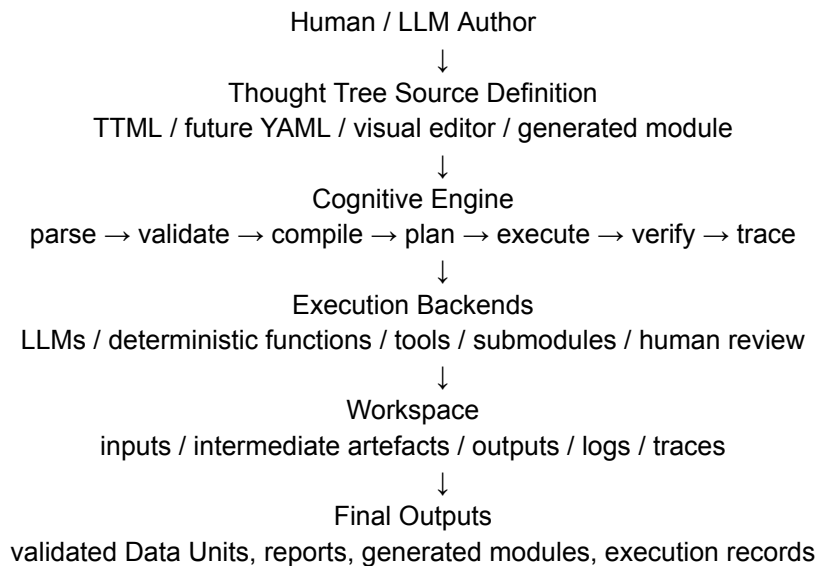
4. System Architecture

The Thought Tree Framework is organised as a layered architecture for defining, compiling, executing and improving cognitive programs.

A Thought Tree program describes a cognitive production process in terms of concepts, Data Units, relationships, transformations, contracts and outputs. The Cognitive Engine turns that program into an executable process by resolving dependencies, planning execution, invoking LLMs, deterministic

functions, tools, submodules and human review steps, then recording the resulting artefacts and execution trace.

At a high level, the architecture can be understood as:



The purpose of this architecture is to separate what cognitive work should be performed from how that work is executed. A Thought Tree program defines the structure and intent of the work. The Cognitive Engine decides how to realise that structure using available models, functions, tools and runtime capabilities.

Architectural Layers

The framework consists of five main architectural layers:

- Source Layer
- Program Model Layer
- Compilation and Planning Layer
- Execution Layer
- Workspace and Trace Layer

Together, these layers allow LLM-assisted work to become modular, inspectable, reusable and auditable.

1. Source Layer

The Source Layer contains the human-readable or machine-generated definition of a Thought Tree program.

In the current framework, the main source format is Thought Tree Markup Language, or TTML. TTML is an XML-based serialisation format for defining a Thought Tree Module.

However, TTML should be understood as one representation of the underlying Thought Tree Program Model, not as the entire framework itself. In future implementations, the same program model could be represented using:

- XML through TTML;
- YAML;
- JSON;
- a visual graph editor;
- a database-backed module definition;
- an LLM-generated module;
- or another domain-specific authoring syntax.

The Source Layer is designed to be both human-readable and LLM-authorable. This is important because Thought Tree programs may be written by humans, generated by LLMs, improved by other Thought Tree programs, or assembled from reusable module libraries.

Example source formats may include:

- TTML file
- Visual workflow graph
- Generated submodule
- Module library entry
- Authoring assistant output

The Source Layer defines the intended cognitive process, but it is not directly responsible for executing it.

2. Program Model Layer

The Program Model Layer is the abstract representation of a Thought Tree program.

This layer defines the core concepts that exist independently of any particular source syntax.

The main elements are:

Module

A Module is a reusable cognitive program. It defines a task or process in terms of inputs, operations and outputs.

A Module may represent a small task, such as summarising an article, or a large process, such as generating a complete technical design document.

Data Unit

A Data Unit is a discrete artefact used or produced by the program.

A Data Unit may represent:

- a text file;
- a document;
- a prompt;
- a requirements register;
- a character profile;
- a plot outline;
- an image;
- a JSON object;

- a validation report;
- a generated TTML module;
- or any other processable artefact.

A Data Unit should be understood as more than a file. It may have:

- a logical identifier;
- physical representation;
- semantic type;
- contract;
- version;
- provenance;
- validation status.

For example:

Logical ID: plot_outline
 Physical representation: plot_outline.txt
 Semantic type: PlotOutline
 Contract: Must include premise, structure, major turning points and resolution
 Produced by: GeneratePlotOutline operation

Operation

An Operation is a transformation that consumes Data Units and produces new Data Units.

Operations may be performed by:

-
- an LLM;
- a deterministic function;
- an external tool;
- another Module;
- a generated submodule;
- a human reviewer;
- or a combination of these.

The basic pattern is:

Input Data Units → Operation → Output Data Units

This is the smallest unit of cognitive computation in the framework.

Collection

A Collection is a named group of Data Units treated as a single logical input or output.

Collections are useful when an operation needs to consume or produce a family of related artefacts, such as:

- all character profiles;
- all chapter drafts;
- all test case reviews;
- all compliance evidence files;
- all generated TDD sections.

Relationship

A Relationship describes how Data Units, Operations and Modules are connected.

Examples include:

- depends on;
- derived from;
- reviews;
- revises;
- validates;
- aggregates;
- decomposes;
- implements;
- replaces;
- produces.

These relationships allow the Cognitive Engine to build a dependency graph and execution plan.

Contract

A Contract defines the expected form, purpose and quality requirements of a Data Unit or operation output.

Contracts may include:

- required structure;
- expected sections;
- format rules;
- completeness criteria;
- quality criteria;
- validation functions;
- LLM review rubrics;
- human approval requirements.

Contracts are what allow the framework to distinguish between “an output exists” and “the correct output has been produced.”

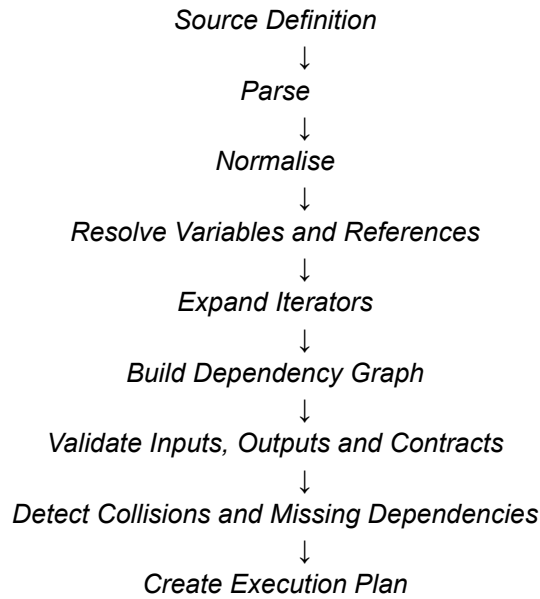
3. Compilation and Planning Layer

The Compilation and Planning Layer is handled by the Cognitive Engine.

The Cognitive Engine takes a source definition, such as TTML, and converts it into an executable internal representation.

This process is similar to compilation in traditional software systems. The source document is not merely read line by line. It is parsed, checked, expanded, resolved and converted into an execution plan.

A typical compilation process includes:



Parsing

The Cognitive Engine loads the source definition and converts it into an internal structure.

For TTML, this means parsing the XML document and validating its basic schema structure.

Normalisation

The engine converts the source-specific representation into the common Thought Tree Program Model.

This allows different source syntaxes to compile into the same internal representation.

Symbol and Dependency Resolution

The engine resolves references such as:

FileRef;
CollectionRef;
variables;
iterators;
function names;
module targets;
contract names;
tool references.

If a reference cannot be resolved, the program should fail validation unless the engine has an explicit strategy for obtaining or generating the missing dependency.

Iterator Expansion

The engine expands iterator-dependent operations into concrete operation instances.

For example:

DraftChapter[ChapterIterator=1] DraftChapter[ChapterIterator=2] DraftChapter[ChapterIterator=3] ...
--

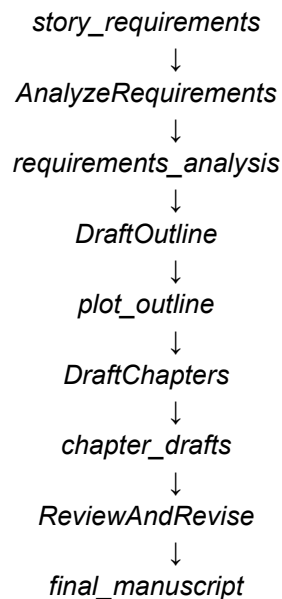
This makes repeated execution explicit and inspectable before runtime.

Dependency Graph Construction

The engine builds a graph of operations and artefacts.

Although the framework is called a Thought Tree, advanced execution is often better understood as a directed graph. Operations depend on Data Units, and Data Units are produced by operations.

A simplified dependency graph may look like:



This graph allows the engine to understand execution order, parallelisation opportunities, missing dependencies and output collisions.

Execution Planning

Once the graph is validated, the Cognitive Engine creates an execution plan.

The execution plan identifies:

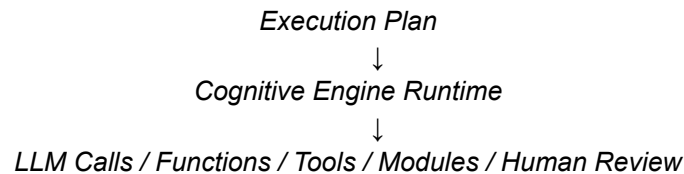
- which operations must run;
- in what order;
- which operations can run in parallel;
- which inputs each operation requires;
- which outputs each operation should produce;
- which contracts must be satisfied;
- which model, function, tool or module should perform each operation;
- what should happen if an operation fails.

The execution plan should be inspectable before execution, especially for large, expensive or high-impact workflows.

4. Execution Layer

The Execution Layer performs the actual work defined by the compiled program.

The Cognitive Engine does not perform all work itself. Instead, it coordinates execution across several possible backends.



The main execution backends are:

- LLM Providers
- Deterministic Functions
- External Tools
- Submodules
- Human Review Interfaces

LLM Providers

LLMs are used for ambiguous semantic transformations.

Examples include:

- analysis;
- summarisation;
- planning;
- drafting;
- rewriting;
- reviewing;
- classification;
- synthesis;
- ideation;
- module generation.

The Thought Tree program should not depend on a specific LLM provider. Instead, the Cognitive Engine should map operation requirements to an available model.

For example, an operation may require:

- high-quality long-context creative reasoning

The engine may then choose an appropriate model based on its own configuration, cost constraints, privacy requirements and available providers.

Deterministic Functions

Deterministic functions are used for mechanical, repeatable work.

Examples include:

- copying files;
- concatenating files;
- validating XML;
- converting documents to text;

- resizing images;
- querying a database;
- creating archives;
- calculating hashes;
- converting formats.

These functions should be registered with the Cognitive Engine and invoked explicitly by operations.

Using deterministic functions prevents LLMs from being used for tasks that ordinary software can perform more reliably.

External Tools

External tools extend the capabilities of the engine.

Examples include:

- search systems;
- vector databases;
- web APIs;
- document stores;
- rendering engines;
- code execution environments;
- build systems;
- content management systems.

Tool use should be governed by the engine's permissions, security model and execution trace requirements.

Submodules

A Module may call another Module.

This allows complex programs to be built from smaller reusable components.

For example:

```
NovelProduction
├── ConceptDevelopment
├── ChapterDrafting
├── ContinuityReview
├── Revision
└── FinalCompilation
```

Submodules may be:

- pre-existing modules from a library;
- local files;
- remote modules;
- generated modules;
- previous outputs from another Thought Tree execution.

The parent Module maps inputs into the submodule and maps the submodule's outputs back into the parent workflow.

Human Review

Human review is an optional but important execution backend.

Some workflows require human judgement, approval or correction.

Human review may be used to:

- approve generated plans;
- check high-impact outputs;
- resolve ambiguous decisions;
- validate generated submodules;
- approve compliance-sensitive outputs;
- correct failed or low-quality artefacts;
- make creative or editorial decisions.

Human interventions should be recorded in the execution trace.

5. Workspace and Trace Layer

The Workspace and Trace Layer stores and records the results of execution.

A Thought Tree execution produces more than final outputs. It also produces intermediate artefacts, generated prompts, validation reports, errors, retries, review decisions, generated submodules and provenance records.

The workspace is the execution context where these assets are stored or referenced.

A workspace may be implemented as:

- a local directory;
- a cloud storage bucket;
- a database;
- an in-memory execution context;
- a version-controlled repository;
- a project asset store;
- or a hybrid storage system.

The workspace maps logical identifiers to concrete assets.

For example:

Logical ID:
plot_outline

Workspace asset:
/workspaces/run_001/outputs/plot_outline.txt
or:

asset_id: plot_outline
run_id: 2026_06_25_001
version: 1

mime_type: text/plain semantic_type: PlotOutline produced_by: GeneratePlotOutline

Artefact Storage

The workspace should preserve:

- root input files;
- intermediate outputs;
- final outputs;
- generated prompts;
- generated submodules;
- validation reports;
- error reports;
- function outputs;
- human review decisions;
- execution logs;
- execution traces.

Preserving intermediate artefacts is one of the core advantages of the framework. It allows users to inspect, revise, replace and rerun parts of a cognitive process without treating the entire workflow as a single opaque prompt.

Execution Trace

Every Thought Tree run should produce an execution trace.

The execution trace records what happened during execution.

A trace may include:

- source module version;
- engine version;
- input identifiers;
- input hashes;
- resolved variables;
- iterator expansions;
- concrete operation instances;
- operation start and end times;
- operation status;
- LLM provider;
- model name;
- model settings;
- generated prompts;
- function calls;
- tool calls;
- submodule paths;
- output identifiers;
- output hashes;
- validation results;
- errors;
- retries;
- human review decisions;

- final outputs.

The trace makes the workflow inspectable, auditable, repeatable and improvable.

Exact reproduction may not always be possible when LLMs are involved, because model behaviour can vary. However, the trace should make the process as reproducible as practical.

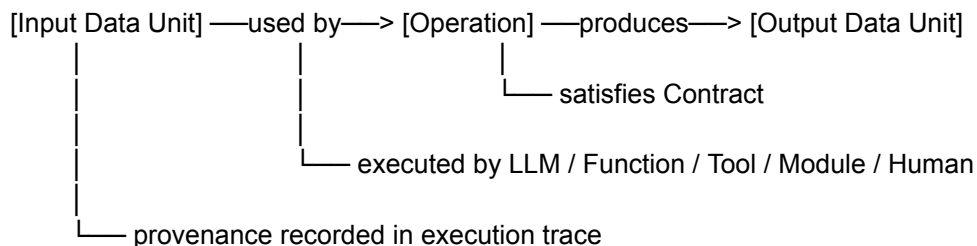
Internal Representation: Cognitive Transformation Graph

Internally, the Cognitive Engine should represent a Thought Tree program as a Cognitive Transformation Graph.

This graph contains:

- Data Unit nodes;
- Operation nodes;
- Collection nodes;
- Module nodes;
- contract nodes or annotations;
- dependency edges;
- provenance edges;
- validation edges.

A simplified graph representation is:



This internal graph gives the engine a clear structure for validation, planning, execution, parallelisation, caching, error handling and traceability.

Although Thought Trees may be visualised as trees, the underlying execution model is more general. Iterators, shared dependencies, collections, submodules, loops and conditional branches may form a directed graph.

Registries

A complete Cognitive Engine may use several registries to support execution.

Module Registry

Stores reusable Thought Tree Modules.

A Module Registry may include:

- module ID;
- version;

- description;
- author;
- required inputs;
- declared outputs;
- required functions;
- required contracts;
- compatible schema version;
- dependencies;
- evaluation history.

Function Registry

Stores deterministic functions available to the engine.

A Function Registry may define:

- function name;
- purpose;
- input requirements;
- output requirements;
- supported formats;
- deterministic behaviour;
- side effects;
- error conditions.

Contract Registry

Stores reusable semantic contracts.

A Contract Registry may include:

- contract name;
- semantic type;
- required structure;
- validation rules;
- review rubrics;
- human approval requirements;
- compatible validators.

Model Registry

Stores information about available LLMs.

A Model Registry may include:

- provider;
- model name;
- context length;
- supported modalities;
- cost profile;
- latency profile;
- quality profile;
- privacy constraints;
- tool support;
- structured output support.

Tool Registry

Stores available external tools and their permissions.

A Tool Registry may include:

- tool name;
- purpose;
- input schema;
- output schema;
- access permissions;
- rate limits;
- safety restrictions.

Registries allow Thought Tree programs to remain abstract while still being executable in a specific environment.

Separation of Concerns

The architecture depends on a clear separation of concerns.

Thought Tree Program

Defines:

-
- what work should be done;
- what inputs are required;
- what transformations should occur;
- what outputs should be produced;
- what contracts should be satisfied.

Cognitive Engine

Determines:

- how to parse and compile the program;
- how to resolve dependencies;
- how to plan execution;
- how to select models, functions and tools;
- how to manage the workspace;
- how to validate outputs;
- how to record traces.

LLMs

Perform:

- semantic reasoning;
- language generation;
- ambiguous transformation;
- review;
- synthesis;
- planning assistance;
- module generation where permitted.

Deterministic Functions

Perform:

- reliable mechanical transformations;
- validation;
- conversion;
- assembly;
- copying;
- archiving;
- other code-driven tasks.

Humans

Provide:

- judgement;
- approval;
- correction;
- governance;
- creative direction;
- high-impact review.

This separation prevents the framework from treating LLMs as universal executors. LLMs are used where they are valuable, while deterministic software performs tasks that require reliability and precision.

Execution Flow

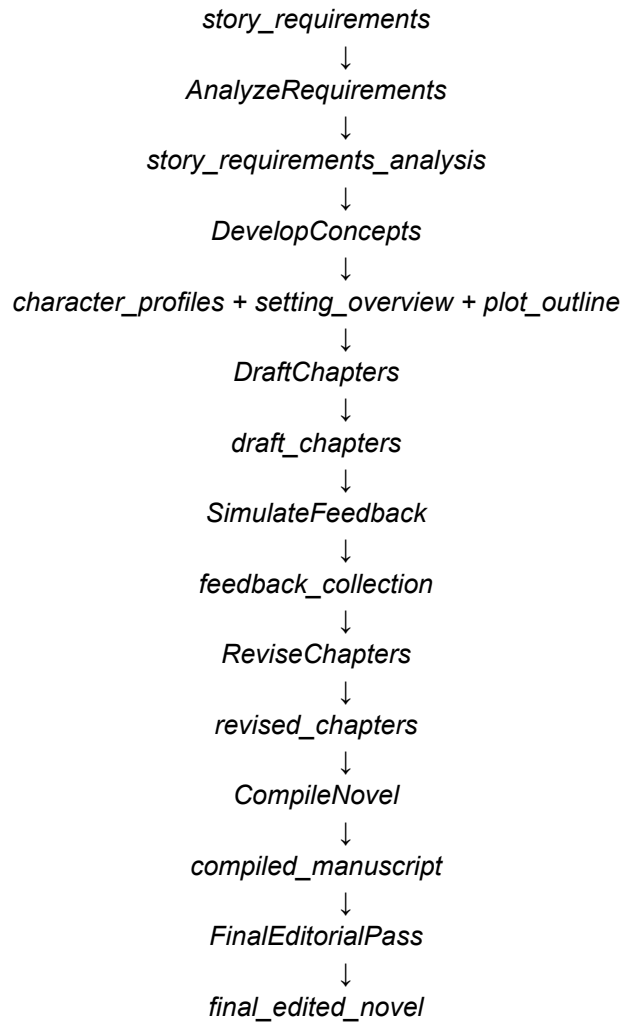
A typical Thought Tree execution follows this flow:

1. Load source definition
2. Validate source syntax
3. Convert source into internal program model
4. Resolve variables, references, collections and iterators
5. Build dependency graph
6. Detect missing inputs and output collisions
7. Create execution plan
8. Initialise workspace
9. Execute operations
10. Validate outputs against contracts
11. Handle errors, retries or review gates
12. Resolve final outputs
13. Record execution trace
14. Return final artefacts

This flow turns a Thought Tree from a static description into an executed cognitive production process.

Example Architecture in Practice

A simplified creative writing workflow might be represented as:



The Cognitive Engine compiles this into a graph, identifies repeated chapter operations, resolves collections, invokes LLMs for creative transformations, invokes deterministic functions for compilation, validates outputs, stores intermediate artefacts and records the full trace.

The final output is not just the novel. The execution also produces the intermediate drafts, reviews, revisions, prompts, validation reports and provenance data that explain how the novel was produced.

Summary

The Thought Tree Framework architecture separates cognitive program definition from execution.

A Thought Tree program defines concepts, Data Units, transformations, relationships, contracts and outputs. The Cognitive Engine compiles this definition into an executable transformation graph, plans the work, invokes the appropriate execution backends, validates the results, stores artefacts and records a trace.

This architecture allows LLM-assisted work to become:

- modular;
- inspectable;

- reusable;
- model-independent;
- auditable;
- improvable;
- and suitable for large-scale cognitive production workflows.

5. Thought Tree Program Model

Thought Tree Program Model

The Thought Tree Program Model is the abstract model underlying the Thought Tree Framework. It defines how cognitive work is represented before it is serialised into TTML, compiled by a Cognitive Engine, or executed using LLMs, deterministic functions, tools and human review.

A Thought Tree program describes a process for transforming meaningful inputs into meaningful outputs. These inputs and outputs may be documents, plans, summaries, creative artefacts, requirements, reviews, datasets, media assets, structured records or other forms of information. The model treats these artefacts not merely as files, but as concept-bearing Data Units connected by relationships and transformed through explicit operations.

At its simplest, the model follows the pattern:

Inputs → Operation → Outputs

In the wider program model, this becomes:

Concepts / Data Units



Transformations / Operations



New Concepts / Data Units



Validation / Review / Trace

A Thought Tree program is therefore a structured, inspectable description of cognitive production. It defines what artefacts are required, what transformations should be performed, what outputs should be produced, how those outputs should be validated, and how the process should be recorded.

The core elements of the Thought Tree Program Model are:

Concepts

Data Units

Semantic Types

Relationships

Operations

Modules

Collections

Variables and Iterators

Contracts

Workspace

Execution Trace

Together, these elements allow complex cognitive work to be represented as an executable program.

1. Concepts

A Concept is a meaningful entity within a cognitive workflow.

Concepts are the things the workflow is about. They may be abstract ideas, document types, creative entities, analytical categories, project artefacts or production targets.

Examples of Concepts include:

Character
Plot Outline
Setting Overview
Chapter Draft
Technical Requirement
Risk Register
Compliance Obligation
Evidence Map
Game Mechanic
Design Constraint
User Persona
Market Summary
Policy Draft
Test Case
Review Report
Improved Module

Concepts are not necessarily stored directly. Instead, they are usually represented by Data Units.

For example, a file called `plot_outline.txt` may represent the Concept `PlotOutline`. A JSON file called `weapon_balance_data.json` may represent the Concept `GameBalanceDataset`. A Markdown file called `technical_design_document.md` may represent the Concept `TechnicalDesignDocument`.

The distinction is important:

Concept: `PlotOutline`
Data Unit: `plot_outline.txt`
Content: the actual text of the plot outline
Semantic Type: `PlotOutline`
Contract: must include premise, act structure, conflict, climax and resolution
The Thought Tree Program Model is concerned with both the concrete artefact and the meaning it carries.

2. Data Units

A Data Unit is a discrete artefact that can be supplied to, produced by, referenced by, validated by or transformed by a Thought Tree program.

In TTML, Data Units are commonly represented as files. However, in the abstract program model, a Data Unit may be any addressable unit of information that a Cognitive Engine can resolve.

A Data Unit may be:

a text file;
a Markdown document;
a JSON object;

an XML document;
a PDF;
an image;
a database record;
a vector search result;
an API response;
an audio file;
a video file;
an in-memory object;
a generated prompt;
a generated TTML module;
a validation report;
a human review decision.

A Data Unit has at least:

Identifier

A logical name used by the Thought Tree program.

Content

The actual information contained in the Data Unit.

Representation

The physical or technical format of the content.

In a fuller implementation, a Data Unit may also have:

Semantic Type

The kind of concept the Data Unit represents.

Contract

The requirements the Data Unit must satisfy.

Provenance

A record of how the Data Unit was produced.

Version

The specific version of the Data Unit within a workflow run.

Validation Status

Whether the Data Unit has passed required checks.

Example:

Data Unit Identifier: character_1_profile

Representation: txt

Semantic Type: CharacterProfile

Produced By: GenerateCharacterProfileDrafts

Inputs Used: story_requirements_analysis, character_system_brief

Contract: CharacterProfileContract

Version: 1

Validation Status: passed

Data Units are the main artefacts that move through a Thought Tree program.

3. Semantic Types

A Semantic Type defines what kind of meaningful artefact a Data Unit is intended to represent.

The representation format tells the Cognitive Engine how the content is stored. The semantic type tells the Cognitive Engine, author or reviewer what the content means.

For example:

plot_outline.txt
has a physical representation:

text/plain
but it may have the semantic type:

PlotOutline
Similarly:

risk_register.csv
may have the semantic type:

RiskRegister
and:

chapter_3_review.md
may have the semantic type:

ChapterReview

Semantic Types are important because they allow Thought Tree programs to reason about artefact suitability, not just artefact existence.

For example, a chapter drafting operation may require:

PlotOutline
CharacterProfile collection
SettingOverview
ChapterPlan

It should not merely require arbitrary text files. It requires inputs that are meaningful in specific ways.

Semantic Types support:

clearer operation definitions;
stronger validation;
better module reuse;
improved execution planning;
safer dynamic module generation;
better human review;
more meaningful error messages.

A Cognitive Engine may use Semantic Types to select prompts, validators, functions, tools or review rubrics.

4. Relationships

A Relationship describes how Concepts, Data Units, Operations or Modules are connected.

Relationships make a Thought Tree program more than a list of steps. They define the dependency and meaning structure of the workflow.

Common relationships include:

Relationship	Meaning
depends on	One artefact is required before another operation can run.
derived from	One artefact was produced using another artefact.
reviews	One artefact evaluates another.
revises	One artefact is an improved version of another.
aggregates	A collection combines multiple artefacts.
decomposes	A module or operation breaks a task into smaller tasks.
validates	A report or function checks an artefact against a contract.
implements	An output satisfies a requirement or plan.
references	An artefact uses another artefact as contextual information.
replaces	A new version supersedes an older one.

For example:

story_requirements_analysis is derived from story_requirements
plot_outline depends on story_requirements_analysis
chapter_1_draft depends on plot_outline and character profiles
chapter_1_review reviews chapter_1_draft
revised_chapter_1 revises chapter_1_draft
final_novel aggregates revised chapters
These relationships can be represented explicitly or inferred from operation inputs and outputs.

The Cognitive Engine uses relationships to:

resolve dependencies;
build an execution graph;
identify parallelisable operations;
detect missing inputs;
prevent output collisions;
trace provenance;
support review and debugging.

5. Operations

An Operation is a transformation that consumes zero or more inputs and produces one or more outputs.

Operations are the active elements of a Thought Tree program. They describe what should happen to Data Units.

The smallest executable pattern is:

Input Data Units → Operation → Output Data Units

Examples of Operations include:

analyse source requirements;
extract risks from a policy document;
generate a plot outline;

draft a chapter;
review a chapter for continuity;
revise a chapter using feedback;
validate XML against a schema;
concatenate files;
generate a TTML submodule;
run a pre-existing module;
request human approval.
An Operation has at least:

Identifier

A unique name within the Module.

Operation Type

The kind of transformation to perform.

Description

A human-readable explanation of the intended transformation.

Inputs

Data Units or Collections used by the operation.

Outputs

Data Units or Collections produced by the operation.

A more advanced Operation may also define:

Preconditions

Conditions that must be true before execution.

Postconditions

Conditions that must be true after execution.

Contracts

Requirements attached to its outputs.

Model or Capability Requirements

Requirements for the LLM, tool or function used.

Retry Strategy

How failure should be handled.

Review Requirements

Whether human or automated review is required.

Cost or Safety Constraints

Limits on execution.

Example:

Operation: ReviseChapter

Type: TextCompletion

Inputs: draft_chapter_1, feedback_for_chapter_1
Output: revised_chapter_1
Description: Revise the chapter using all beta reader feedback.
Contract: RevisedChapterContract
Relationship: revised_chapter_1 revises draft_chapter_1
Operations may be executed by LLMs, deterministic code, external tools, submodules or humans.

6. Operation Types

The Thought Tree Program Model supports different types of Operations. These allow a Cognitive Engine to combine ambiguous semantic work with deterministic computation and external system interaction.

Core operation types include:

6.1 Text Completion Operation

A Text Completion Operation invokes an LLM to produce one or more outputs from a set of inputs and instructions.

It is suitable for:

summarisation;
analysis;
drafting;
review;
rewriting;
planning;
classification;
ideation;
synthesis;
extraction.

Example:

Input: story_requirements

Task: Draft a plot outline

Output: plot_outline

6.2 Execute Function Operation

An Execute Function Operation invokes deterministic code.

It is suitable for:

copying files;
concatenating files;
validating XML;
converting formats;
resizing images;
querying databases;
generating checksums;
archiving outputs.

Example:

Input: revised_chapter_1 ... revised_chapter_12

Task: Concatenate files

Output: compiled_novel_manuscript

6.3 Pre-Existing Module Operation

A Pre-Existing Module Operation executes another Thought Tree Module.

It is suitable when a reusable sub-process already exists.

Example:

Input: story_requirements

Task: Run ConceptDevelopment module

Output: character_profiles, setting_overview, plot_outline

6.4 Project Completion Operation

A Project Completion Operation allows the Cognitive Engine to generate a new submodule to complete a complex task.

It is suitable when the task is too large or ambiguous for a single operation.

Example:

Input: high_level_story_brief

Task: Develop all world-building material

Output: generated TTML submodule and resulting world-building documents

6.5 Dynamic Completion Operation

A Dynamic Completion Operation allows the Cognitive Engine to choose an execution strategy.

It may resolve into:

a Text Completion;

an Execute Function;

a Pre-Existing Module;

a generated submodule;

a combination of operations.

Because this introduces ambiguity, the selected execution strategy should be recorded in the execution trace.

6.6 Human Review Operation

A Human Review Operation pauses or routes execution for human judgement.

It is suitable for:

approving generated plans;

checking high-impact outputs;

resolving ambiguous decisions;

approving dynamically generated submodules;

reviewing failed validation outputs.

Human review may be implemented as a formal operation type, an engine-level pause, or an external approval workflow.

7. Modules

A Module is a reusable cognitive program composed of inputs, variables, iterators, collections, operations and outputs.

A Module defines a bounded task.

Examples of Modules include:

Generate Novel Outline
Develop Character Profiles
Produce Technical Design Document
Extract Compliance Obligations
Review Policy Draft
Generate Test Cases
Improve TTML Module
Compile Final Report

A Module has:

Metadata

Project or module identity, description, author, version and other descriptive information.

Inputs

Data Units required before execution.

Variables

Configurable values used during execution.

Iterators

Repetition structures used to expand operations.

Collections

Groups of related Data Units.

Operations

Ordered or dependency-linked transformations.

Outputs

Final Data Units exported by the Module.

A Module may be used as:

a complete standalone program;
a submodule inside another Module;
a reusable process in a module library;
a generated execution plan;
an artefact to be reviewed and improved.

Modules are recursively composable. An Operation can execute a Module, and a Module can contain Operations that generate or execute other Modules.

This recursive structure allows complex cognitive work to be decomposed into smaller, inspectable, reusable parts.

8. Collections

A Collection is a named group of Data Units treated as a single logical input or output.

Collections are useful when an operation needs to consume or produce a family of related artefacts.

Examples:

all_character_profiles
all_revised_chapters
feedback_for_chapter_1
test_case_reviews
policy_evidence_documents
A Collection may contain:

individual Data Units;
references to Data Units;
other Collections;
generated members from iterator expansion.
Collections are important because they distinguish between two different patterns:

Repeated execution
Run an operation once for each item.

Grouped context
Run an operation once using many items together.

For example, when generating feedback from three beta reader personas over twelve chapters, the workflow may produce:

feedback_chapter_1_persona_1
feedback_chapter_1_persona_2
feedback_chapter_1_persona_3

...

feedback_chapter_12_persona_3

When revising chapter 1, the operation should not run three times and overwrite the same revised chapter. Instead, it should receive a Collection:

feedback_for_chapter_1
and produce:

revised_chapter_1
This makes the intended aggregation explicit.

Collections help prevent:

accidental output collisions;
ambiguous multi-iterator execution;
unclear grouped dependencies;
excessive prompt construction ambiguity.

9. Variables and Iterators

Variables are named values used to configure a Module.

Examples:

ChapterCount = 12
CharacterCount = 4

PersonaCount = 3
TestCaseCount = 5

Variables allow a Module to be reused with different parameters.

Iterators define repeated execution ranges or sets.

Example:

ChapterIterator = 1 to ChapterCount

An Iterator can be used to generate repeated artefacts:

draft_chapter_{{ChapterIterator}}

which may expand to:

draft_chapter_1
draft_chapter_2
draft_chapter_3

...

draft_chapter_12

Iterators allow Thought Tree programs to scale across repeated structures such as:

chapters;
characters;
scenes;
personas;
files;
requirements;
test cases;
assets;
policy sections;
products;
time periods.

The Cognitive Engine is responsible for expanding iterators into concrete operation instances before execution.

If multiple iterators are used in the same operation, the engine may create a Cartesian product of their values unless the workflow uses Collections or another explicit aggregation strategy.

Iterator expansion must be validated before execution to detect unresolved references, missing inputs and output collisions.

10. Contracts

A Contract defines what a Data Unit or Operation output must satisfy to be considered valid.

Contracts are central to cognitive programming because LLMs can produce fluent but incomplete, malformed or unsuitable outputs. A Thought Tree program should not treat an output as successful merely because a file was produced. It should be able to check whether the output is fit for its intended role.

A Contract may specify:

expected semantic type;

required format;
required sections;
required fields;
length constraints;
completeness criteria;
quality criteria;
prohibited content;
source traceability requirements;
downstream usage expectations;
deterministic validation functions;
LLM-assisted review rubrics;
human approval requirements.
Example contract for a PlotOutline:

Semantic Type: PlotOutline

Required Sections:

- Premise
- Main Characters
- Setting
- Act I
- Act II
- Act III
- Climax
- Resolution

Quality Criteria:

- central conflict is clear
- protagonist motivation is explicit
- ending resolves the main conflict
- outline is consistent with story requirements

Validation:

- LLM review against rubric
- optional human approval

Contracts support three levels of correctness:

Schema Correctness

The program definition is structurally valid.

Execution Correctness

The program can run and produce declared outputs.

Semantic Correctness

The outputs satisfy the meaning, structure and quality required by the workflow.

Contracts allow a Cognitive Engine to enforce validation gates. If an output fails its contract, the engine may:

- retry the operation;
- request a repaired output;
- invoke a review module;
- escalate to human review;

mark the workflow as failed;
produce a diagnostic report.

11. Workspace

A Workspace is the execution environment where Data Units, intermediate artefacts, generated prompts, submodules, validation reports, logs and final outputs are stored or referenced.

A Workspace may be implemented as:

- a local directory;
- a cloud storage bucket;
- a database;
- an in-memory context;
- a version-controlled project;
- a hybrid artefact store.

The Workspace maps logical identifiers from the Thought Tree program to concrete assets.

For example:

Logical Identifier: plot_outline

Concrete Asset: /workspaces/run_001/outputs/plot_outline.txt

or:

Logical Identifier: plot_outline

Database Record: asset_id=plot_outline, run_id=001, version=1

The exact implementation is engine-specific, but the mapping must remain traceable.

The Workspace may contain:

- original inputs;
- resolved variables;
- expanded operation plans;
- intermediate outputs;
- final outputs;
- generated prompts;
- generated submodules;
- function outputs;
- validation reports;
- error reports;
- human review decisions;
- execution traces.

The Workspace allows the Thought Tree program to be inspected before, during and after execution.

12. Execution Trace

An Execution Trace records what happened during a Thought Tree run.

The trace is essential for auditability, debugging, reproducibility and improvement.

A trace may include:

- source module version;
- schema version;
- engine version;

module metadata;
input identifiers;
input content hashes;
resolved variables;
iterator expansions;
operation execution order;
operation start and end times;
active iterator values;
resolved inputs;
generated prompts;
LLM provider and model;
model settings;
deterministic function calls;
tool calls;
generated submodules;
output identifiers;
output hashes;
validation results;
retries;
errors;
human review decisions;
final outputs.

The trace provides the provenance of the workflow.

It allows a user to answer questions such as:

Which inputs produced this output?
Which model was used?
What prompt was sent?
Which operation generated this artefact?
Did the output pass validation?
Was a human reviewer involved?
Which version of a module was executed?
What failed and how was it repaired?

Because LLM outputs may not be perfectly deterministic, the goal of the Execution Trace is not always exact reproduction. The goal is to make the process explicit, inspectable, repeatable and improvable.

13. Thought Tree Programs as Graphs

Although the framework uses the term “Thought Tree,” the underlying program model is often closer to a directed graph.

A simple Module may look like a linear sequence:

Input → Analyse → Draft → Review → Final Output

A more complex Module may branch, merge, repeat, call submodules or generate new modules:

→ Character Profiles →
Story Requirements → → Setting Overview → Draft Chapters → Review → Revise → Compile
→ Plot Outline →

When iterators, Collections, submodules, validation gates and conditional execution are introduced, the structure becomes a graph of dependencies and transformations.

In this graph:

Data Units are artefact nodes;

Operations are transformation nodes;

relationships are edges;

Collections are grouped artefact nodes;

Modules are reusable subgraphs;

Contracts are validation rules attached to nodes or edges;

the Execution Trace records how the graph was executed.

The Cognitive Engine compiles a Thought Tree source definition into an executable transformation graph. This graph determines what must run, in what order, with what dependencies, and what outputs must be produced.

14. Minimal Abstract Example

A simple Thought Tree program might be described as:

Input:

- article

Operations:

1. SummariseArticle

Input: article

Output: article_summary

2. ReviewSummary

Inputs: article, article_summary

Output: summary_review

3. ReviseSummary

Inputs: article_summary, summary_review

Output: final_summary

Final Output:

- final_summary

In concept terms:

Article

→ SummariseArticle

ArticleSummary

→ ReviewSummary

SummaryReview

→ ReviseSummary

FinalSummary

This program does not merely ask an LLM to “summarise the article.” It defines a small cognitive production process with an intermediate summary, a review stage and a final revised output.

The process can be inspected, rerun, modified, validated and reused.

15. Summary

The Thought Tree Program Model provides a way to describe cognitive work as an executable structure of concepts, artefacts, relationships and transformations.

Its key idea is that complex LLM-assisted work should not be treated as a single opaque prompt. Instead, it should be represented as a modular cognitive program.

A Thought Tree program defines:

- what inputs are required;
- what concepts and artefacts are involved;
- what transformations should occur;
- what outputs should be produced;
- how outputs should be grouped, reviewed and validated;
- how the process should be traced;
- how the workflow can be reused or improved.

This model allows the Cognitive Engine to act as a compiler and runtime for cognitive programs, translating high-level process definitions into concrete execution using LLMs, deterministic functions, tools, submodules and human review.

6. Cognitive Engine

Cognitive Engine

A **Cognitive Engine** is the compiler, runtime and execution environment for Thought Tree programs.

It is responsible for taking a human-readable cognitive program — such as a TTML Module — and turning it into an executable process. It parses the program, validates its structure, resolves its references, expands its iterators, builds an execution plan, invokes LLMs and deterministic functions, manages intermediate artefacts, validates outputs, handles errors and records an execution trace.

The Cognitive Engine is the bridge between a declarative description of cognitive work and the practical systems that perform that work.

A Thought Tree program describes **what cognitive process should happen**.

The Cognitive Engine determines **how that process is compiled, executed, monitored and recorded**.

In this sense, the Cognitive Engine plays a role similar to a compiler and runtime in conventional software systems. However, instead of compiling source code into machine instructions, it compiles cognitive workflow definitions into a graph of semantic transformations executed through LLMs, tools, functions, submodules and human review.

The LLM is not the Cognitive Engine itself. The LLM is one execution backend used by the Cognitive Engine for ambiguous semantic work such as drafting, analysis, planning, review and synthesis. The Cognitive Engine remains responsible for structure, validation, execution control, traceability and correctness.

Role of the Cognitive Engine

The Cognitive Engine performs four broad roles:

1. ****Compiler****
 - Converts a Thought Tree definition into an internal executable representation.
2. ****Runtime****
 - Executes operations using LLMs, deterministic functions, tools, submodules and human review.
3. ****Workspace Manager****
 - Stores, resolves, versions and tracks inputs, intermediate artefacts and final outputs.
4. ****Trace and Validation System****
 - Records what happened and validates that produced outputs satisfy structural, execution and semantic requirements.

A Cognitive Engine therefore does more than send prompts to an LLM. It manages the full lifecycle of a cognitive program.

Core Responsibilities

A conformant Cognitive Engine should be able to:

- load a Thought Tree program;
- parse its source representation;
- validate its structure;
- resolve variables, inputs, Data Units, Collections and module references;
- expand iterators into concrete operation instances;
- detect unresolved dependencies and output collisions;
- build an execution plan;
- execute operations in the correct order;
- invoke LLMs for semantic transformations;
- invoke deterministic functions for mechanical transformations;
- invoke tools and external systems where supported;
- execute submodules;
- support human review where required;
- validate outputs against declared contracts;
- preserve intermediate artefacts;
- handle errors and retries;
- resolve final outputs;
- record an execution trace;
- return or store the completed outputs.

The exact implementation may vary between engines, but the intended meaning of a Thought Tree program should remain stable across conformant engines.

The Cognitive Engine as Compiler

Before a Thought Tree program can be executed, it must be interpreted and transformed into an executable form. This is the compilation phase.

The Cognitive Engine should not treat TTML or any future source format as merely a sequence of prompts. It should compile the source into an internal representation that makes dependencies, transformations, artefacts and validation requirements explicit.

A typical compilation process includes:

```
```text
Source Module
 ↓
Parse
 ↓
Normalise
 ↓
Resolve Symbols
 ↓
Expand Iterators
 ↓
Build Dependency Graph
 ↓
Validate Execution Requirements
 ↓
Create Execution Plan
```
```

Parsing

The engine loads the source document, such as a TTML file, and parses it into structured objects.

For TTML, this means reading elements such as:

- ``<Project>``
- ``<Inputs>``
- ``<Vars>``
- ``<Iterators>``
- ``<Collections>``
- ``<Operations>``
- ``<Output>``

The result is an initial in-memory representation of the Module.

Normalisation

The engine converts the parsed source into the engine's internal program model.

This allows different source formats to compile to the same representation. For example, future versions of the framework may support TTML, JSON, YAML or visual graph authoring, while all compile into the same internal Thought Tree program model.

Normalisation separates the abstract meaning of the program from the syntax used to write it.

Symbol Resolution

The engine resolves all named references.

This includes:

- `FileRef` references;
- `CollectionRef` references;
- variable references such as `{{ChapterCount}}`;
- iterator references such as `{{ChapterIterator}}`;
- function names;
- target module paths;
- contract references;
- semantic type references, where supported.

If a required reference cannot be resolved, the engine should report a validation error before execution begins, unless the program explicitly allows dynamic resolution.

Iterator Expansion

The engine expands iterator-dependent operations into concrete operation instances.

For example:

```
```xml
<File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
```
```

with:

```
```xml
<Iterator id="ChapterIterator" from="1" to="12"/>
```
```

expands into:

```
```text
draft_chapter_1
draft_chapter_2
...
draft_chapter_12
```
```

If an operation references multiple active iterators, the engine determines whether to use the Cartesian product, an explicit aggregation strategy, or another declared iteration behaviour.

Iterator expansion should happen before execution so that the engine can detect dependency problems, output collisions and excessive execution cost.

Dependency Graph Construction

The engine builds a graph of dependencies between operation instances and Data Units.

In this graph:

- Data Units are artefact nodes;
- Operations are transformation nodes;
- edges represent dependency relationships;
- Collections are grouped artefact nodes;
- Modules are reusable subgraphs.

Although the framework is called a Thought Tree, the executable structure is often better understood as a directed transformation graph. Simple workflows may look like linear chains or trees. More advanced workflows may include branching, aggregation, submodules, loops or conditional execution.

Execution Validation

Before execution, the engine should confirm that the program can meaningfully run.

This includes checking that:

- required inputs are available;
- all dependencies can be resolved;
- operation outputs have unique identifiers or declared versioning behaviour;
- deterministic functions exist;
- target submodules are available;
- Collections can be resolved;
- iterator expansion does not create unintended collisions;
- required tools, models and permissions are available;
- final outputs can be produced by the planned workflow.

Execution validation is distinct from schema validation. A TTML file can be structurally valid XML but still impossible to execute.

Execution Planning

Once validated, the engine creates an execution plan.

The plan identifies:

- the concrete operation instances to execute;
- the order in which they should run;
- dependencies between operations;
- opportunities for parallel execution;
- required models, functions or tools;
- expected outputs;
- validation gates;
- review points;
- failure strategies.

A Cognitive Engine may display this plan to the user before execution, especially for large, expensive or dynamically generated workflows.

The Cognitive Engine as Runtime

After compilation and planning, the Cognitive Engine executes the program.

Runtime execution involves coordinating multiple kinds of transformation.

A Thought Tree operation may be executed by:

- an LLM;
- a deterministic function;
- a tool call;
- a pre-existing Module;
- a dynamically generated Module;
- a human reviewer;
- or a combination of these.

The runtime is responsible for ensuring that each operation receives the correct inputs, produces the expected outputs, and records what happened.

LLM Execution

LLMs are used for operations that require semantic interpretation, generation, synthesis or judgement.

Examples include:

- analysing requirements;
- drafting text;
- summarising source material;
- generating ideas;
- reviewing consistency;
- extracting structured information;
- comparing alternatives;
- designing a plan;
- revising outputs;
- generating a submodule.

For a `TextCompletion` operation, the Cognitive Engine constructs an LLM request from:

- the operation description;
- resolved input Data Units;
- Collection contents;
- relevant variables;
- active iterator values;
- output requirements;
- semantic contracts;
- system instructions;
- engine-specific prompt policies;
- model selection rules.

The engine may make additional internal LLM calls to improve reliability. For example, it may use one call to create a prompt, another to generate the output, and another to validate or repair the result.

These internal calls are implementation details, but where practical they should be recorded in the execution trace.

Deterministic Function Execution

Deterministic functions are used for mechanical operations that do not require LLM reasoning.

Examples include:

- copying files;
- concatenating files;
- converting formats;
- validating XML;
- querying a database;
- calculating checksums;
- resizing images;
- creating archives;
- parsing structured data;
- applying templates.

For an `ExecuteFunction` operation, the engine resolves the operation's inputs, calls the registered function, then stores the declared outputs.

For example:

```
``xml
<Operation id="CompileNovel"
  type="ExecuteFunction"
  desc="Concatenate the revised chapter files into one manuscript."
  function="ConcatenateFiles">
  <CollectionRef id="all_revised_chapters"/>
  <Output>
    <File id="compiled_novel_manuscript" extension="txt"/>
  </Output>
</Operation>
``
```

The engine resolves `all\_revised\_chapters`, passes its members to `ConcatenateFiles` in the declared order, and stores the resulting manuscript as `compiled\_novel\_manuscript`.

Deterministic functions are important because they prevent LLMs from being used for tasks that ordinary software can perform more reliably.

Tool and External System Execution

A Cognitive Engine may also support tool-based execution.

Tools may include:

- search engines;
- vector databases;
- web APIs;
- document stores;
- code execution environments;
- rendering tools;
- translation systems;
- image generation systems;
- game engines;
- data analysis systems;
- enterprise databases.

Tool execution should be governed by explicit permissions and recorded in the execution trace.

If a tool has side effects, such as sending an email, writing to a database or publishing content, the engine should require explicit authorisation or a declared policy allowing that action.

Submodule Execution

A Thought Tree program may call another Module.

This allows complex cognitive processes to be built recursively from smaller reusable processes.

For a `PreExisting` operation, the engine loads a target Module, maps parent inputs into the submodule, executes the submodule, then maps the submodule outputs back into the parent workflow.

For example:

```
```xml
<Operation id="ConceptDevelopment"
 type="PreExisting"
 desc="Run the concept development subproject."
 target="/subprojects/ConceptDevelopment.html">
 <FileRef id="story_requirements"/>
 <Output>
 <File id="plot_outline" extension="txt"/>
 </Output>
</Operation>
```
```

The Cognitive Engine should record:

- the target module path or identifier;
- the target module version;

- input mappings;
- output mappings;
- execution trace of the submodule;
- any errors or validation results.

Submodule execution is central to the recursive nature of the framework.

Dynamic Planning and Generated Modules

Some operations may be too broad or ambiguous to execute directly.

A `ProjectCompletion` or `DynamicCompletion` operation allows the Cognitive Engine to generate or select an execution strategy.

For example, an operation such as:

```
```text
Develop the character, setting and plot concepts for this novel.
```
```

may be expanded into a generated submodule containing separate operations for:

- analysing requirements;
- developing a character system;
- generating character profiles;
- developing setting overview;
- drafting plot outline;
- reviewing consistency;
- revising final concept documents.

Dynamic planning is powerful, but it must be controlled.

A recommended process is:

```
```text
Ambiguous Operation
↓
Generate Proposed Plan or Submodule
↓
Validate Proposed Plan
↓
Optional Human Approval
↓
Execute Approved Plan
↓
Record Generated Plan in Trace
```
```

Generated Modules should be preserved as execution artefacts. They form part of the audit trail and may later be reviewed, reused or improved.

Human Review

Human review may be part of the execution process.

The Cognitive Engine may pause execution and request human input when:

- a generated plan requires approval;
- an output has high impact;
- a semantic contract fails;
- a collision requires resolution;
- a dynamic submodule is generated;
- quality is uncertain;
- a policy requires human authorisation;
- an unrecoverable error occurs.

Human review can be represented as:

- a dedicated operation type;
- a pause in the runtime;
- an external approval workflow;
- a deterministic function;
- or a review Module.

If a human modifies an input, output, operation or Module, that change should be recorded in the execution trace.

Workspace Management

Every Thought Tree execution takes place inside a workspace.

The workspace is the execution context where the Cognitive Engine stores or references:

- initial inputs;
- intermediate Data Units;
- Collections;
- generated prompts;
- generated submodules;
- function outputs;
- tool results;
- validation reports;
- human review decisions;
- final outputs;
- execution traces;
- error reports.

The workspace maps logical identifiers from the Thought Tree program to concrete assets.

For example:

```
```xml
<File id="plot_outline" extension="txt"/>
```
```

may correspond to a local file:

```
```text
/workspaces/run_001/outputs/plot_outline.txt
```
```

or to a database record:

```
```text
asset_id: plot_outline
run_id: run_001
version: 1
mime_type: text/plain
semantic_type: PlotOutline
```
```

The exact storage mechanism is implementation-specific, but logical identifiers should remain traceable.

Data Units and Artefact Resolution

A Cognitive Engine must resolve Data Units before they can be used by operations.

A Data Unit may originate from:

- root-level inputs;
- outputs of previous operations;
- outputs of submodules;
- workspace assets;
- external references;
- tool calls;
- human uploads;
- dynamically generated content.

When an operation references a Data Unit, the engine should determine:

- whether it exists;
- where it is stored;
- what version should be used;
- what its content type is;
- what semantic type it represents;
- whether it satisfies its contract;
- whether the current operation has permission to use it.

A `FileRef` should not merely be treated as a filename. It is a logical reference to an artefact in the execution workspace.

Collections

Collections allow multiple Data Units to be treated as one logical input or output.

The Cognitive Engine is responsible for resolving Collections into ordered sets of Data Units.

For example:

```
```xml
<Collection id="all_revised_chapters" orderBy="ChapterIterator">
 <FileRef id="revised_chapter_{{ChapterIterator}}"/>
</Collection>
```
```

The engine resolves this into:

```
```text
revised_chapter_1
revised_chapter_2
...
revised_chapter_12
```
```

A Collection may be passed to an LLM as grouped context, or to a deterministic function as an ordered input list.

Collections are important because they allow aggregation without unintentionally expanding an operation over every member.

Semantic Contracts and Validation

A Cognitive Engine should support validation beyond simple file existence.

A Thought Tree program may declare or imply contracts for outputs. A contract defines what must be true for an artefact to be considered valid.

Contracts may specify:

- expected semantic type;
- expected physical format;
- required sections;
- required fields;
- completeness criteria;
- quality criteria;
- style requirements;

- source traceability requirements;
- validation functions;
- LLM review rubrics;
- human approval requirements.

The Cognitive Engine should distinguish between three levels of correctness:

1. **Schema correctness**
 - The source program is structurally valid.
2. **Execution correctness**
 - The engine can execute the program and produce the declared outputs.
3. **Semantic correctness**
 - The produced outputs satisfy their intended meaning and quality requirements.

Without semantic validation, a workflow may appear successful while passing weak, incomplete or malformed outputs into downstream operations.

Where contracts are declared, an operation should not be considered complete until its outputs satisfy the required contracts or an explicit failure/review path is triggered.

Error Handling and Recovery

Cognitive Engine execution may fail for many reasons.

Common failure types include:

- invalid source syntax;
- schema validation failure;
- missing inputs;
- unresolved references;
- invalid iterator expansion;
- output collisions;
- missing functions;
- unavailable target modules;
- unavailable model provider;
- LLM API failure;
- malformed LLM output;
- failed semantic validation;
- failed tool call;
- permission errors;
- storage errors;
- human rejection;
- final output resolution failure.

The engine may recover by:

- retrying the operation;
- using a different model;

- repairing malformed output;
- invoking a validation or repair Module;
- requesting human review;
- rolling back partial outputs;
- creating a diagnostic report;
- skipping optional operations, where explicitly allowed;
- failing the run safely.

Silent failure should not be allowed. Silent overwriting should also be invalid unless explicitly declared.

All errors and recovery attempts should be recorded in the execution trace.

Execution Trace

Every Thought Tree execution should produce an execution trace.

The execution trace records what happened during execution and allows the process to be inspected, debugged, reviewed and improved.

A trace may include:

- source program identifier;
- source program version;
- schema version;
- engine version;
- input identifiers;
- input content hashes;
- resolved variables;
- iterator expansions;
- dependency graph;
- execution plan;
- operation start and end times;
- operation statuses;
- model providers;
- model names;
- model settings;
- prompts;
- tool calls;
- function calls;
- submodule executions;
- output identifiers;
- output versions;
- output hashes;
- validation reports;
- contract results;
- retries;
- errors;
- human review decisions;
- final outputs.

The goal of the trace is not always perfect deterministic reproduction, because LLM behaviour may vary. The goal is to make the cognitive process explicit, inspectable, repeatable and improvable.

Model Independence

A key purpose of the Cognitive Engine is to separate Thought Tree programs from specific LLM providers.

A Module should describe the intended cognitive process, not hard-code a particular model unless necessary.

Instead of specifying:

```
```text
Use Model X from Provider Y.
```
```

a Module may specify capability requirements such as:

```
```text
Requires long-context reasoning.
Requires high-quality creative synthesis.
Requires structured JSON output.
Requires low-cost summarisation.
Requires local execution due to data sensitivity.
```
```

The Cognitive Engine can then map those requirements to an available model.

This allows the same Thought Tree program to run on different execution backends as models improve, costs change or organisational policies evolve.

Engine Registries

A Cognitive Engine may maintain registries of available execution resources.

Useful registries include:

Module Registry

A catalogue of reusable Thought Tree Modules.

Stores:

- module identifier;
- version;
- description;
- required inputs;

- declared outputs;
- supported schema version;
- dependencies;
- author;
- validation status.

Function Registry

A catalogue of deterministic functions.

Stores:

- function name;
- input requirements;
- output behaviour;
- deterministic status;
- side effects;
- supported file types;
- permissions required.

Model Registry

A catalogue of available LLMs and model capabilities.

Stores:

- provider;
- model name;
- context window;
- supported modalities;
- cost profile;
- quality profile;
- latency profile;
- structured output support;
- privacy constraints.

Tool Registry

A catalogue of external tools available to the engine.

Stores:

- tool name;
- purpose;
- permissions;
- input schema;
- output schema;
- side effects;
- audit requirements.

Contract Registry

A catalogue of reusable semantic contracts.

Stores:

- contract identifier;
- semantic type;
- validation criteria;
- required structure;
- machine validators;
- LLM review rubrics;
- human approval requirements.

Registries make Thought Tree programs more portable, reusable and robust.

Parallel and Optimised Execution

The default execution model is sequential document-order execution.

However, once the Cognitive Engine has built a dependency graph, it may safely optimise execution.

Possible optimisations include:

- running independent operations in parallel;
- caching previously generated outputs;
- skipping unchanged deterministic steps;
- reusing validated artefacts;
- selecting cheaper models for low-risk steps;
- selecting stronger models for high-risk steps;
- batching similar LLM calls;
- rerunning only affected downstream operations after an input change.

Optimisation must not change the intended semantics of the program.

Correctness, traceability and safety should take priority over speed.

Security and Governance

Because a Cognitive Engine may execute functions, call tools, read files, write outputs, generate modules and use external models, it must include safeguards.

Important governance concerns include:

- permissions for file access;
- permissions for tool use;
- sandboxing deterministic functions;
- controlling side effects;
- managing API credentials;
- preventing prompt injection;

- validating untrusted Modules;
- limiting dynamic execution;
- enforcing human approval policies;
- controlling cost;
- protecting confidential data;
- recording audit logs.

A Cognitive Engine should be able to distinguish between trusted and untrusted Modules, safe and unsafe tools, local and external model execution, and read-only versus side-effecting operations.

Dynamic or generated Modules should be validated before execution.

Conformance

A basic Cognitive Engine should support:

- loading a Thought Tree source document;
- validating its structure;
- resolving inputs, variables and iterators;
- resolving Data Units and Collections;
- executing operations in document order;
- supporting LLM-based TextCompletion operations;
- supporting registered deterministic functions;
- producing declared outputs;
- recording an execution trace.

A more advanced Cognitive Engine may additionally support:

- semantic types;
- contracts;
- validation gates;
- human review;
- model selection;
- tool use;
- generated submodules;
- DynamicCompletion;
- ProjectCompletion;
- module search;
- parallel execution;
- caching;
- automated repair;
- test harnesses;
- module improvement workflows.

Conformance levels may be defined separately to distinguish simple executors from full cognitive programming runtimes.

Summary

The Cognitive Engine is the operational centre of the Thought Tree Framework.

It turns declarative cognitive programs into executable, inspectable and improvable processes.

It acts as:

- a compiler, by transforming source definitions into executable graphs;
- a runtime, by coordinating LLMs, functions, tools, submodules and humans;
- a workspace manager, by preserving Data Units and intermediate artefacts;
- a validator, by checking structure, execution and semantic correctness;
- a trace system, by recording what happened and why.

This separation is essential to the framework's strategic purpose.

Thought Tree programs define cognitive work independently of any particular model provider. The Cognitive Engine executes that work using the best available combination of models, software functions, tools and review processes.

In this way, the Cognitive Engine makes cognitive programming practical: it transforms concepts, relationships and transformations into real artefacts, validated outputs and traceable production processes.

7. Execution Semantics

Execution Semantics

Execution semantics define how a Cognitive Engine interprets, compiles and executes a Thought Tree Program.

A Thought Tree Program describes a cognitive process in terms of inputs, Data Units, Collections, Operations, Modules, semantic contracts and final outputs. The Cognitive Engine is responsible for turning this declarative program into an executable process. It does this by parsing the source definition, resolving references, expanding iterators, validating dependencies, planning execution, invoking LLMs or deterministic functions, validating outputs and recording an execution trace.

The purpose of execution semantics is to ensure that Thought Tree Programs are not merely human-readable descriptions, but executable cognitive programs whose behaviour can be inspected, validated, repeated and improved.

A Thought Tree Program should be understandable before execution, observable during execution and reviewable after execution.

1. Execution Overview

A Cognitive Engine executes a Thought Tree Program through the following general lifecycle:

1. ****Load the source definition****

2. ****Parse the source into the Thought Tree Program Model****
3. ****Validate the program structure****
4. ****Initialise the workspace****
5. ****Resolve inputs, variables, Data Units and Collections****
6. ****Expand iterators****
7. ****Compile the program into an executable transformation graph****
8. ****Validate dependencies, contracts and output behaviour****
9. ****Plan operation execution****
10. ****Execute operations****
11. ****Validate produced outputs****
12. ****Handle errors, retries and review gates****
13. ****Resolve final outputs****
14. ****Record the execution trace****
15. ****Return or store the final outputs****

Different Cognitive Engines may implement these stages differently, but they should preserve the intended meaning of the Thought Tree Program.

A conformant Cognitive Engine should not treat a Thought Tree Program as a sequence of prompts. It should treat it as a structured cognitive program whose operations produce, consume, validate and transform identifiable artefacts.

2. Source Definition and Program Model

A Thought Tree Program may be authored in TTML or in another supported source format. The source format is not itself the execution model. Before execution, the Cognitive Engine should convert the source definition into an internal representation of the Thought Tree Program Model.

This internal representation should include:

- Module metadata;
- declared inputs;
- variables;
- iterators;
- Data Units;
- Collections;
- Operations;
- operation inputs and outputs;
- semantic types, where supported;
- contracts, where supported;
- final output declarations;
- dependencies between operations;
- references to external Modules, functions, tools or models.

For TTML, the Cognitive Engine maps source elements into the program model. For example:

| TTML Element | Program Model Meaning |
|--------------|-----------------------|
| <TTML> | A Thought Tree Module |
| <Project> | Module metadata |

- | ``<Inputs>`` | Required initial Data Units |
- | ``<Vars>`` | Reusable values |
- | ``<Iterators>`` | Repeated execution dimensions |
- | ``<Collections>`` | Grouped Data Units |
- | ``<Operations>`` | Transformations |
- | ``<Output>`` | Final exported Data Units or Collections |
- | ``<File>`` | Declared Data Unit |
- | ``<FileRef>`` | Reference to a Data Unit |
- | ``<Collection>`` | Declared group of Data Units |
- | ``<CollectionRef>`` | Reference to a Collection |

The Cognitive Engine may retain the original source definition as part of the execution trace.

3. Workspace Initialisation

Each Thought Tree execution takes place inside a workspace.

The workspace is the execution context where the Cognitive Engine stores or references:

- supplied inputs;
- intermediate Data Units;
- Collections;
- generated prompts;
- generated submodules;
- deterministic function outputs;
- validation reports;
- review reports;
- error diagnostics;
- final outputs;
- execution trace data.

A workspace may be implemented as:

- a local directory;
- cloud object storage;
- a database;
- an in-memory environment;
- a project asset store;
- a version-controlled repository;
- or a hybrid storage system.

The workspace maps logical identifiers from the Thought Tree Program to concrete assets.

For example, a logical Data Unit:

```
```xml
<File id="plot_outline" extension="txt"/>
```
```

may be stored as:

```
```text
/workspaces/run_001/outputs/plot_outline.txt
```
```

or as a database record:

```
```text
asset_id: plot_outline
run_id: run_001
version: 1
mime_type: text/plain
semantic_type: PlotOutline
```
```

The storage mechanism is implementation-specific, but the logical identifier must remain traceable.

4. Data Unit Resolution

A Data Unit is a discrete unit of information that can be consumed, produced, reviewed or validated by a Thought Tree Program.

Before executing an Operation, the Cognitive Engine must resolve every required Data Unit reference.

A Data Unit reference may resolve to:

1. a root-level input;
2. an output produced by an earlier Operation;
3. an asset already present in the workspace;
4. an output produced by a submodule;
5. an external asset available through an engine-specific resolver;
6. a generated or recovered asset produced by an explicit recovery strategy.

If a required Data Unit cannot be resolved, the operation must not execute unless the Cognitive Engine has an explicit mechanism for obtaining or generating the missing input.

Silent creation of missing inputs should be avoided. If the engine generates, requests or substitutes a missing Data Unit, this must be recorded in the execution trace.

5. Variables

Variables are named values available during execution.

Variables may be used for:

- iterator ranges;
- file identifiers;

- Collection identifiers;
- operation descriptions;
- output naming;
- model or tool configuration;
- contract parameters;
- module configuration.

Example:

```
```xml
<Vars>
 <Var name="ChapterCount" type="integer" value="12"/>
</Vars>
```
```

A variable may be referenced using the syntax:

```
```text
{{ChapterCount}}
```
```

The Cognitive Engine should resolve variable references before operation execution.

If a variable reference cannot be resolved, validation should fail.

Variables are local to the current Module unless the Cognitive Engine or Module execution semantics explicitly define inheritance, override or parameter passing behaviour.

6. Iterator Expansion

Iterators define repeated execution dimensions.

Example:

```
```xml
<Iterator id="ChapterIterator" from="1" to="{{ChapterCount}}"/>
```
```

If `ChapterCount` is `12`, then `ChapterIterator` expands to:

```
```text
1, 2, 3, ..., 12
```
```

Iterator values may be inserted into identifiers:

```
```xml
<File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
```
```

This represents the concrete Data Units:

```
```text
draft_chapter_1
draft_chapter_2
...
draft_chapter_12
```
```

An operation expands over its **active iterators**.

An iterator becomes active for an operation when it appears in:

- a direct Data Unit reference used by the operation;
- a direct output Data Unit produced by the operation;
- a CollectionRef identifier used by the operation;
- the operation description or attributes, if the engine supports substitution there.

An iterator used only inside the internal member definition of a Collection does not automatically become active for the consuming operation.

This distinction is important because Collections may aggregate a repeated set of Data Units without causing the consuming operation to repeat once per Collection member.

7. Multiple Iterators

If an operation references multiple active iterators, the default behaviour is Cartesian expansion.

For example:

```
```xml
<FileRef id="draft_chapter_{{ChapterIterator}}"/>
<FileRef id="beta_reader_persona_{{PersonalIterator}}"/>
<Output>
 <File id="feedback_chapter_{{ChapterIterator}}_persona_{{PersonalIterator}}" extension="txt"/>
</Output>
```
```

If there are 12 chapters and 3 personas, the operation expands into 36 concrete operation instances:

```
```text
Chapter 1, Persona 1
Chapter 1, Persona 2
Chapter 1, Persona 3
Chapter 2, Persona 1
...
Chapter 12, Persona 3
```
```

The Cognitive Engine should make this expansion inspectable before execution, especially where execution may be costly.

8. Collections

A Collection is a named group of Data Units.

Collections allow a Thought Tree Program to pass multiple related artefacts as one logical input or output.

Example:

```
```xml
<Collection id="all_revised_chapters" orderBy="ChapterIterator">
 <FileRef id="revised_chapter_{{ChapterIterator}}"/>
</Collection>
```
```

If `ChapterIterator` has values 1 to 12, the Collection represents:

```
```text
revised_chapter_1
revised_chapter_2
...
revised_chapter_12
```
```

An operation may consume the Collection as a grouped input:

```
```xml
<CollectionRef id="all_revised_chapters"/>
```
```

A CollectionRef does not automatically cause the consuming operation to execute once per Collection member. Instead, the Cognitive Engine resolves the Collection into an ordered set of Data Units and supplies that set to the operation as grouped context.

Collections are useful for:

- aggregating outputs from iterator-expanded operations;
- passing many files into one LLM completion;
- compiling review sets;
- supplying all feedback for a single artefact;
- preventing unintended output collisions;
- making grouped dependencies explicit.

If an order is declared, the Cognitive Engine should preserve that order when supplying the Collection to an Operation or deterministic function.

9. Dependency Graph

Although a Thought Tree may be visually represented as a tree, execution is usually better understood as a directed transformation graph.

The graph contains:

- Data Unit nodes;
- Collection nodes;
- Operation nodes;
- Module nodes, where submodules are invoked;
- dependency edges;
- output edges;
- validation gates;
- review gates.

An Operation depends on all Data Units and Collections it references as inputs.

An Operation produces one or more Data Units or Collections as outputs.

The Cognitive Engine should compile the program into a dependency graph before execution.

This graph is used to determine:

- which operations can run;
- which operations must wait;
- which outputs are required by later operations;
- whether dependencies are missing;
- whether output collisions exist;
- whether parallel execution is safe;
- whether final outputs can be produced.

By default, operations execute in document order. However, an engine may execute independent operations in parallel if doing so does not change the logical result of the program.

10. Operation Planning

Before execution, the Cognitive Engine should produce an execution plan.

The execution plan should identify:

- concrete operation instances;
- active iterator values for each instance;
- required input Data Units;
- required input Collections;
- expected outputs;
- operation type;
- target Module or function, if applicable;
- semantic contracts to apply;

- whether human review is required;
- whether the operation can be parallelised;
- expected cost or resource usage, where available.

For small workflows, this plan may be implicit. For large, expensive or high-impact workflows, the plan should be inspectable before execution.

A Cognitive Engine may provide a “dry run” mode that validates and displays the execution plan without invoking LLMs or producing outputs.

11. Operation Execution

Each Operation is executed according to its operation type.

At minimum, a Cognitive Engine may support the following operation types:

1. **TextCompletion**
2. **ExecuteFunction**
3. **PreExisting**
4. **ProjectCompletion**
5. **DynamicCompletion**

Additional operation types may be supported by specific engines or future schema versions.

11.1 TextCompletion Operations

A `TextCompletion` Operation invokes an LLM to produce one or more output Data Units.

Example:

```

<<<xml
<Operation id="DraftOutline" type="TextCompletion" desc="Draft a plot outline.">
  <FileRef id="story_requirements"/>
  <Output>
    <File id="plot_outline" extension="txt"/>
  </Output>
</Operation>
>>>

```

The Cognitive Engine should construct an LLM request using:

- the operation description;
- resolved input Data Units;
- resolved Collections;
- relevant variables;
- active iterator values;
- Module metadata;
- output requirements;

- semantic contracts, where supported;
- implementation-specific system instructions.

The declared output tells the engine what artefact the LLM is expected to produce.

A Cognitive Engine may use internal LLM calls to improve prompt construction, choose model settings, validate output format or repair malformed responses. These internal calls are implementation details, but should be recorded in the execution trace where practical.

When a TextCompletion Operation receives a CollectionRef, the engine should supply the Collection as grouped context, preserving member identifiers and order where possible.

11.2 ExecuteFunction Operations

An `ExecuteFunction` Operation invokes deterministic code rather than an LLM.

Example:

```

```xml
<Operation id="CompileNovel"
 type="ExecuteFunction"
 desc="Concatenate the revised chapter files into one manuscript."
 function="ConcatenateFiles">
 <CollectionRef id="all_revised_chapters"/>
 <Output>
 <File id="compiled_novel_manuscript" extension="txt"/>
 </Output>
</Operation>
```

```

The `function` attribute identifies the deterministic function to execute.

Functions are appropriate for tasks such as:

- copying files;
- concatenating files;
- converting formats;
- validating XML;
- querying databases;
- resizing images;
- generating checksums;
- archiving outputs;
- calling controlled APIs.

The Cognitive Engine should validate that the requested function exists before execution.

If a function receives a CollectionRef, the Collection should be expanded into the function's input list according to its declared order.

Function calls should be recorded in the execution trace, including function name, input identifiers, output identifiers, version information where available and any errors.

11.3 PreExisting Operations

A `PreExisting` Operation executes an existing Thought Tree Module.

Example:

```
```xml
<Operation id="ConceptDevelopment"
 type="PreExisting"
 desc="Run the concept development subproject."
 target="/subprojects/ConceptDevelopment.ttml">
 <FileRef id="story_requirements"/>
 <Output>
 <File id="plot_outline" extension="txt"/>
 </Output>
</Operation>
```
```

The `target` attribute identifies the Module to execute.

When executing a pre-existing submodule, the Cognitive Engine should:

1. load the target Module;
2. validate the target Module;
3. map parent operation inputs to submodule inputs;
4. execute the submodule;
5. map submodule final outputs back to the parent operation's declared outputs;
6. record the mapping in the execution trace;
7. continue executing the parent Module.

Mapping may be explicit, inferred by matching identifiers, or determined by engine-specific rules. However, ambiguous mappings should require validation or review.

11.4 ProjectCompletion Operations

A `ProjectCompletion` Operation expands a complex operation into a newly generated submodule.

This is useful when a task is too large or ambiguous to execute as a single completion.

Example:

```
```xml
<Operation id="DevelopConcept"
 type="ProjectCompletion"
 desc="Develop the character, setting and plot concepts.">
```

```

 <FileRef id="story_requirements"/>
 <Output>
 <File id="character_profiles" extension="txt"/>
 <File id="setting_overview" extension="txt"/>
 <File id="plot_outline" extension="txt"/>
 </Output>
 </Operation>
 ...

```

A Cognitive Engine may use an LLM to generate a new Thought Tree Module that decomposes the operation into smaller steps.

The generated submodule should:

- accept mapped inputs from the parent operation;
- produce outputs satisfying the parent operation's declared outputs;
- be validated before execution;
- be stored in the workspace;
- be included in the execution trace.

Where appropriate, generated submodules should be subject to human or automated review before execution.

A recommended execution pattern is:

```

```text
ProjectCompletion Operation
    ↓
Generate proposed submodule
    ↓
Validate submodule
    ↓
Review or approve submodule, if required
    ↓
Execute submodule
    ↓
Map outputs back to parent operation
```

```

This preserves the flexibility of dynamic generation while maintaining traceability and control.

---

### ### 11.5 DynamicCompletion Operations

A `DynamicCompletion` Operation allows the Cognitive Engine to choose the execution strategy.

A DynamicCompletion may resolve into:

- a direct TextCompletion;
- an ExecuteFunction call;
- a PreExisting Module call;

- a generated ProjectCompletion submodule;
- a sequence of smaller operations;
- a combination of the above.

DynamicCompletion is useful when the author wants to describe the intended cognitive transformation without specifying exactly how it should be executed.

Because DynamicCompletion introduces ambiguity, the engine should record:

- the chosen execution strategy;
- why that strategy was selected, where possible;
- any generated operations or submodules;
- validation results;
- human approval, if required.

DynamicCompletion should not be used to bypass validation, traceability or security controls.

---

## ## 12. Output Resolution

Each Operation declares the Data Units or Collections it is expected to produce.

An operation is considered complete only when:

1. its execution has finished;
2. all declared outputs exist;
3. outputs are stored or referenced in the workspace;
4. required validation gates have passed;
5. no unrecovered operation errors remain.

The root-level final output declaration defines the final outputs of the Module.

Example:

```
```xml
<Output>
  <FileRef id="final_edited_novel"/>
</Output>
```
```

After all required operations have executed, the Cognitive Engine resolves each final output reference.

A Thought Tree execution is complete when:

- all required operations have completed;
- all final outputs can be resolved;
- final outputs are available in the workspace;
- required contracts have passed, where supported;
- no unrecovered execution errors remain;
- the execution trace has been recorded.

If a final output cannot be resolved, execution should be considered incomplete or failed.

---

### ## 13. Output Collisions

An output collision occurs when two or more operation instances attempt to write to the same logical output identifier.

For example:

```
``xml
<FileRef id="draft_chapter_{{ChapterIterator}}"/>
<FileRef id="feedback_chapter_{{ChapterIterator}}_persona_{{PersonalIterator}}"/>
<Output>
 <File id="revised_chapter_{{ChapterIterator}}" extension="txt"/>
</Output>
``
```

If `PersonalIterator` has multiple values, several concrete operation instances may attempt to produce the same `revised\_chapter\_{{ChapterIterator}}` output.

By default, this is invalid.

A Cognitive Engine should detect output collisions during pre-execution validation.

Valid collision resolutions include:

1. including all active iterators in the output identifier;
2. aggregating the extra iterator dimension into a Collection;
3. inserting a separate aggregation operation;
4. creating versioned outputs;
5. explicitly declaring overwrite behaviour;
6. requiring human or automated review.

Silent overwriting should be invalid by default.

---

### ## 14. Versioning and Replacement

Outputs should be treated as versioned artefacts unless explicitly overwritten.

If an operation produces an output identifier that already exists, the Cognitive Engine should use one of the following strategies:

1. reject the operation as an output collision;
2. create a new version;
3. overwrite only if explicitly permitted;
4. preserve both outputs and require resolution;
5. invoke an error-handling or review process.

A recommended logical versioning pattern is:

```
```text
draft_chapter_1
  version: 1
  version: 2
  version: 3
```
```

or a file-based pattern:

```
```text
draft_chapter_1.v1.txt
draft_chapter_1.v2.txt
draft_chapter_1.v3.txt
```
```

The chosen versioning mechanism is implementation-specific, but provenance should be preserved.

---

## ## 15. Semantic Contracts and Validation Gates

Schema validation confirms that a source definition is structurally valid. Execution validation confirms that dependencies can be resolved and operations can run. Semantic validation confirms that the produced outputs are fit for their intended purpose.

A Thought Tree Program may define semantic contracts for Data Units.

A contract may specify:

- expected semantic type;
- required structure;
- required sections;
- formatting rules;
- completeness criteria;
- quality criteria;
- validation functions;
- LLM review rubrics;
- human approval requirements;
- downstream usage expectations.

An operation output should be considered valid only when it satisfies its associated contract.

This creates three levels of correctness:

### 1. **Schema correctness**

The source definition is structurally valid.

### 2. **Execution correctness**

The Cognitive Engine can resolve dependencies, execute operations and produce declared outputs.

### 3. **\*\*Semantic correctness\*\***

The produced outputs satisfy the content requirements needed by the workflow.

If a semantic contract fails, the Cognitive Engine may:

- retry the operation;
- repair the output;
- invoke a review operation;
- request human intervention;
- produce a diagnostic report;
- fail the workflow.

Contract validation results should be included in the execution trace.

---

## ## 16. Error Handling

Execution may fail for several reasons, including:

- invalid source syntax;
- schema validation failure;
- unresolved Data Unit references;
- unresolved Collection references;
- invalid variables;
- invalid iterators;
- output collisions;
- missing input files;
- missing deterministic functions;
- unavailable target Modules;
- LLM API failure;
- malformed LLM output;
- failed semantic contract validation;
- permission errors;
- storage errors;
- non-terminating loops;
- human review rejection.

A Cognitive Engine should report errors with enough context for diagnosis, including:

- Module ID;
- operation ID;
- operation type;
- active iterator values;
- input references;
- expected outputs;
- error message;
- timestamp;
- retry attempts;
- validation results.



A Cognitive Engine may attempt recovery through:

1. retrying the operation;
2. regenerating malformed outputs;
3. invoking a repair operation;
4. invoking an error-handling Module;
5. requesting human review;
6. skipping optional operations, if supported;
7. rolling back partial outputs;
8. producing a diagnostic report.

Errors and recovery attempts must be recorded in the execution trace.

Silent failure should be avoided.

---

## ## 17. Human Review

A Cognitive Engine may support human review steps.

Human review may be implemented as:

- a specialised operation type;
- a pause in execution;
- a deterministic function;
- an external approval workflow;
- a review Module;
- an engine-level validation gate.

Human review is useful when:

- generated plans require approval;
- outputs are high-impact;
- semantic contract validation fails;
- output collisions require resolution;
- generated submodules need inspection;
- quality is uncertain;
- the workflow affects compliance, safety, finance or publication.

If human review changes a Data Unit, Operation, Module, contract or final output, the change should be recorded in the execution trace.

Human intervention should be treated as part of the execution process, not as an invisible external correction.

---

## ## 18. Parallel and Asynchronous Execution

The default execution model is sequential document-order execution.

However, a Cognitive Engine may execute independent operations asynchronously or in parallel where doing so does not change the logical result.

Parallel execution is safe when:

- operations do not depend on each other's outputs;
- operations do not write to the same output identifiers;
- operation ordering does not affect later results;
- shared resources are handled safely;
- execution trace ordering remains clear.

For example, generating feedback from three different beta reader personas may be parallelisable. Revising a chapter using all feedback is not parallelisable until the feedback Collection has been produced.

The Cognitive Engine should prioritise correctness and traceability over speed.

---

### ## 19. Loops and Conditional Execution

Simple Thought Tree Programs execute in a finite sequence. More advanced programs may include loops, conditions, scheduling or decision points.

Where loops or conditional execution are supported, they should be represented explicitly through operation types, function calls, generated submodules or future schema extensions.

A loop should define:

- loop condition;
- maximum iteration count;
- stopping criteria;
- state carried between iterations;
- delay or scheduling behaviour, if applicable;
- error handling behaviour;
- review or approval requirements, if applicable.

Indefinite loops should require explicit author intent and engine-level safeguards.

Conditional execution should define:

- the condition being evaluated;
- the operation or function that evaluates it;
- possible branches;
- default behaviour;
- traceable decision output.

Because loops and conditionals can turn a tree into a directed graph, the Cognitive Engine should make the resulting execution structure inspectable.

---

## ## 20. Execution Trace

Every Thought Tree execution should produce an execution trace.

The execution trace is a record of what happened during execution. It allows the process to be inspected, debugged, reviewed, reproduced and improved.

An execution trace may include:

- source definition version;
- Module ID;
- Module metadata;
- input identifiers;
- input hashes;
- resolved variables;
- iterator expansions;
- dependency graph;
- execution plan;
- operation start and end times;
- operation statuses;
- active iterator values;
- model provider;
- model name;
- model version, where available;
- model settings;
- generated prompts, where appropriate;
- function calls;
- tool calls;
- submodule paths;
- generated submodules;
- output identifiers;
- output hashes;
- semantic contract results;
- validation reports;
- errors;
- retries;
- human review decisions;
- final outputs.

The goal is not always exact deterministic reproduction, because LLM behaviour may vary. The goal is to make the process explicit, inspectable, repeatable and improvable.

---

## ## 21. Determinism and Reproducibility

Thought Tree Programs make the structure of cognitive work reproducible. They define what operations should occur, what inputs they should use and what outputs they should produce.

However, LLM-based operations may not produce identical outputs on every run unless the Cognitive Engine controls and records relevant execution settings.

For stronger reproducibility, the Cognitive Engine should record:

- model provider;
- model name;
- model version;
- temperature;
- seed, where available;
- system prompt;
- generated prompt;
- input content hashes;
- tool versions;
- function versions;
- contract versions;
- execution timestamp;
- retry count;
- validation results.

A deterministic function should produce the same output for the same inputs, assuming the same function version and environment.

An LLM operation should be treated as probabilistic unless the engine and model provider support deterministic execution.

---

## ## 22. Execution Completion

A Thought Tree execution is complete when:

1. all required operations have completed;
2. all required validation gates have passed;
3. all declared final outputs can be resolved;
4. final outputs are available in the workspace;
5. no unrecovered errors remain;
6. the execution trace has been recorded.

If any final output cannot be resolved, the execution should be considered incomplete or failed.

If a semantic contract fails and no recovery strategy succeeds, the execution should be considered semantically failed even if files were produced.

---

## ## 23. Conformance Requirements

A Cognitive Engine conforms to the basic Thought Tree execution model if it can:

- load a Thought Tree source definition;
- parse it into the program model;
- validate its structure;
- initialise a workspace;
- resolve inputs, variables and Data Unit references;

- execute operations in document order;
- support TextCompletion operations;
- support ExecuteFunction operations where functions are defined;
- produce declared outputs;
- resolve final outputs;
- detect unresolved dependencies;
- detect output collisions;
- record an execution trace.

A more advanced Cognitive Engine may additionally support:

- Collections;
- iterator expansion;
- semantic types;
- semantic contracts;
- validation gates;
- PreExisting Module execution;
- ProjectCompletion;
- DynamicCompletion;
- generated submodules;
- parallel execution;
- caching;
- human review;
- module libraries;
- model selection;
- tool use;
- loop and conditional execution;
- automated testing and improvement.

A Thought Tree Program should not execute if pre-execution validation detects unresolved required inputs, invalid iterator expansion, missing functions, missing target modules or unintended output collisions.

---

## ## 24. Summary

Execution Semantics turn a Thought Tree Program from a human-readable cognitive process definition into an executable, inspectable and traceable production process.

A Cognitive Engine should not simply send prompts to an LLM. It should:

- parse and compile the program;
- resolve dependencies;
- build an execution graph;
- execute transformations;
- preserve intermediate artefacts;
- validate outputs;
- handle errors;
- support review;
- resolve final outputs;
- record what happened.

This allows LLM-assisted work to move beyond isolated prompts and unstructured agents toward a form of cognitive programming: modular, auditable, reusable and improvable execution of transformations over concepts and artefacts.

## 8. Semantic Contracts and Validation

### Semantic Contracts and Validation

A Thought Tree program should not be considered successful merely because it produced files with the expected identifiers. In cognitive workflows, the more important question is whether the produced artefacts are meaningful, complete, usable and fit for their intended downstream purpose.

For example, an operation may successfully create a file called `plot_outline.txt`, but that file may still be too vague to support chapter drafting. It may omit the climax, contradict the character profiles, ignore the original story requirements, or fail to define a coherent narrative arc. At the file-system level, the output exists. At the cognitive level, the output has failed.

Semantic Contracts address this problem.

A Semantic Contract defines the expected meaning, structure, quality and usage requirements of a Data Unit. It allows a Cognitive Engine to distinguish between:

- an output that exists;

- an output that has the correct format;

- an output that satisfies the cognitive requirements of the workflow.

Semantic Contracts are therefore a core part of cognitive programming. They allow Thought Tree programs to validate not only execution, but also the usefulness of the concepts and artefacts being produced.

### Purpose of Semantic Contracts

In conventional software, a compiler or runtime can often validate whether data has the expected type. For example, a function may require an integer, a string, a JSON object or a file path.

In LLM-assisted cognitive workflows, this is not enough.

A Data Unit may be physically represented as plain text, but its semantic role may be much more specific. For example:

| File                                      | Physical Format | Semantic Type             |
|-------------------------------------------|-----------------|---------------------------|
| <code>plot_outline.txt</code>             | Text            | Plot Outline              |
| <code>character_1_profile.txt</code>      | Text            | Character Profile         |
| <code>risk_register.md</code>             | Markdown        | Risk Register             |
| <code>requirements_analysis.txt</code>    | Text            | Requirements Analysis     |
| <code>policy_gap_assessment.json</code>   | JSON            | Compliance Gap Assessment |
| <code>technical_design_document.md</code> | Markdown        | Technical Design Document |

The physical format tells the Cognitive Engine how the artefact is stored.

The semantic type tells the Cognitive Engine what the artefact is supposed to mean.

A Semantic Contract defines what must be true for that artefact to be acceptable.

For example, a CharacterProfile contract may require:

- character name;
- narrative role;
- motivation;
- internal conflict;
- external conflict;
- relationships to other characters;
- character arc;
- voice and personality;
- continuity notes;
- relevance to the plot.

A TechnicalRequirementsRegister contract may require:

- functional requirements;
- non-functional requirements;
- assumptions;
- dependencies;
- risks;
- unresolved questions;
- source traceability;
- explicit/inferred/unresolved classification.

Semantic Contracts make these expectations explicit.

### Three Levels of Correctness

Thought Tree validation should distinguish between three levels of correctness.

#### 1. Schema Correctness

Schema correctness means that the Thought Tree source document is structurally valid.

For TTML, this means the XML conforms to the TTML schema. The document contains valid elements, required attributes, valid operation types and structurally valid sections.

Schema correctness answers:

Is this a valid Thought Tree program document?

Example checks:

- Is the XML well-formed?
- Does the document contain a valid root <TTML> element?
- Are required sections present?
- Are operation types valid?
- Are required file extensions declared?
- Are variables and iterators structurally valid?

Schema correctness is necessary, but not sufficient.

A TTML file may be schema-valid while still containing unresolved dependencies, missing functions or poorly designed operations.

#### 2. Execution Correctness

Execution correctness means that the Cognitive Engine can run the Thought Tree program.

It confirms that the declared workflow can be resolved, planned and executed.

Execution correctness answers:

Can this Thought Tree program actually run?

Example checks:

Are required input Data Units available?

Can every FileRef be resolved?

Can every CollectionRef be resolved?

Do all referenced variables exist?

Are iterator ranges valid?

Does iterator expansion create output collisions?

Are required deterministic functions available?

Are referenced submodules available?

Can final outputs be produced?

Does the engine have access to required models, tools and permissions?

Execution correctness confirms that the workflow can produce the declared artefacts.

However, it still does not guarantee that those artefacts are good enough.

### 3. Semantic Correctness

Semantic correctness means that produced Data Units satisfy their intended cognitive purpose.

Semantic correctness answers:

Is this output meaningful, complete and fit for use in the workflow?

Example checks:

Does the output satisfy its semantic contract?

Does it contain required sections?

Is it consistent with its inputs?

Does it avoid unsupported claims?

Is it detailed enough for downstream operations?

Does it preserve necessary constraints?

Does it follow the required format?

Does it meet quality criteria?

Has it passed required review or approval gates?

Semantic correctness is the level most directly associated with cognitive programming. It is what prevents a workflow from silently passing weak, incomplete or misleading artefacts into later stages.

### Semantic Types

A Semantic Type describes what kind of cognitive artefact a Data Unit represents.

Examples include:

SourceDigest

PlotOutline



CharacterProfile  
ChapterDraft  
ContinuityReview  
TechnicalRequirementsRegister  
RiskRegister  
ComplianceGapAssessment  
EvidenceMap  
GameMechanicSpecification  
PolicySummary  
TestCaseSpecification  
TTMLModule

A Semantic Type is different from a file extension.

For example:

character\_1\_profile.txt  
has:

Physical format: txt

Semantic type: CharacterProfile

A Thought Tree program may use Semantic Types to make operation expectations clearer.

For example:

<File id="plot\_outline" extension="txt" semanticType="PlotOutline"/>  
or in a future higher-level syntax:

outputs:

plot\_outline:

format: txt

semantic\_type: PlotOutline

contract: ThreeActPlotOutlineContract

Semantic Types allow the Cognitive Engine, authoring tools and review modules to reason about what the Data Unit is supposed to be, rather than treating it as an anonymous text file.

Data Contracts

A Data Contract defines the expected properties of a Data Unit.

A contract may specify:

semantic type;

physical format;

required sections;

required fields;

required relationships;

formatting requirements;

completeness criteria;

consistency criteria;

source traceability requirements;

quality criteria;

validation functions;

LLM-assisted review rubrics;

human approval requirements;  
downstream usage expectations.

A Data Contract may be strict, loose or advisory depending on the needs of the workflow.

For example, a contract for a structured JSON output may be strict and machine-validatable. A contract for a creative plot outline may include looser quality criteria that require LLM-assisted or human review.

Example: Plot Outline Contract

A simple contract for a plot outline might state:

contract: PlotOutlineContract

semantic\_type: PlotOutline

format: markdown

required\_sections:

- premise
- genre
- target\_audience
- main\_characters
- central\_conflict
- act\_1
- act\_2
- act\_3
- climax
- resolution
- unresolved\_questions

quality\_criteria:

- The outline must be consistent with the story requirements.
- The protagonist's motivation must be clear.
- The central conflict must escalate across the story.
- The climax must resolve the primary conflict.
- The outline must provide enough detail to support chapter drafting.

validation:

machine:

- ValidateMarkdownSections

llm\_review:

- ReviewNarrativeCompleteness
- ReviewContinuityAgainstRequirements

human\_review:

required: false

This contract tells the Cognitive Engine that producing a text file is not enough. The output must contain the expected structure and satisfy narrative requirements.

Example: Technical Requirements Register Contract

A contract for a technical requirements register might be more analytical:

contract: TechnicalRequirementsRegisterContract

semantic\_type: TechnicalRequirementsRegister

format: markdown

required\_sections:

- functional\_requirements
- non\_functional\_requirements

- platform\_requirements
- data\_requirements
- integration\_requirements
- assumptions
- dependencies
- risks
- open\_questions

required\_fields\_per\_requirement:

- id
- description
- source\_basis
- status
- priority
- verification\_method

allowed\_status\_values:

- explicit
- inferred
- unresolved

quality\_criteria:

- Requirements must not invent unsupported technical claims.
- Inferred requirements must be labelled as inferred.
- Unresolved points must be captured as open questions.
- Requirements should be specific enough to support implementation planning.

validation:

machine:

- ValidateRequirementTableStructure

llm\_review:

- ReviewForUnsupportedClaims
- ReviewRequirementsCompleteness

human\_review:

required: true

reason: "High-impact technical planning artefact."

This contract supports both machine validation and judgement-based review.

## Operation Contracts

Semantic Contracts may apply not only to Data Units, but also to Operations.

An Operation Contract defines what must be true before and after an operation executes.

This may include:

preconditions;  
 expected inputs;  
 expected outputs;  
 required semantic types;  
 postconditions;  
 validation gates;  
 failure handling;  
 retry behaviour;  
 review requirements.

For example:

operation: ReviseChapter

preconditions:

- draft\_chapter exists
- feedback\_for\_chapter is non\_empty
- plot\_outline satisfies PlotOutlineContract

inputs:

draft\_chapter:

semantic\_type: ChapterDraft

feedback\_for\_chapter:

semantic\_type: FeedbackCollection

outputs:

revised\_chapter:

semantic\_type: ChapterDraft

contract: RevisedChapterContract

postconditions:

- revised\_chapter incorporates relevant feedback
- revised\_chapter remains consistent with plot\_outline
- revised\_chapter does not introduce unresolved continuity errors

on\_contract\_failure:

- retry\_with\_repair\_prompt
- generate\_validation\_report
- request\_human\_review

This makes the operation more like a typed function in a programming language, while allowing for the ambiguity inherent in cognitive work.

### Validation Gates

A Validation Gate is a point in the workflow where an output must be checked before downstream execution continues.

An operation should be considered complete only when:

the declared output exists;

the output is accessible in the workspace;

the output satisfies required format checks;

the output satisfies its Semantic Contract, where one is declared;

required review or approval steps have passed.

If validation fails, the Cognitive Engine may:

retry the operation;

ask an LLM to repair the output;

run a specialised review module;

generate a diagnostic report;

request human intervention;

branch to an error-handling module;

mark the workflow as failed;

preserve the failed output for audit purposes.

Validation Gates are especially important before outputs are used by later operations.

For example, a weak plot\_outline should not be passed into chapter drafting simply because the file exists. It should first satisfy the PlotOutlineContract.

### Validation Methods

Semantic validation can be performed in several ways.

### 1. Structural Validation

Structural validation checks whether an output has the required form.

Examples:

Does the Markdown document contain required headings?

Does the JSON parse correctly?

Does the XML conform to a schema?

Does a table contain required columns?

Does a file use the expected extension?

Are required sections present?

Structural validation is often deterministic.

### 2. Content Validation

Content validation checks whether the output contains required information.

Examples:

Does a character profile include motivation, conflict and arc?

Does a requirements register include assumptions and open questions?

Does a review report identify specific issues?

Does a policy summary include obligations, risks and evidence gaps?

Content validation may require a mixture of deterministic checks and LLM-assisted review.

### 3. Consistency Validation

Consistency validation checks whether an output agrees with other Data Units.

Examples:

Does a chapter contradict the plot outline?

Does a character profile contradict the character system brief?

Does a technical design document introduce unsupported features?

Does a compliance report accurately reflect the source policy?

Does a generated TTML module preserve the intent of the original operation?

Consistency validation is often semantic and may require LLM review or human judgement.

### 4. Source Grounding Validation

Source grounding validation checks whether claims in an output are supported by the input material.

This is especially important for analytical, compliance, technical and factual workflows.

Examples:

Requirements should be traceable to design notes.

Compliance obligations should be traceable to source policy text.

Risk statements should be derived from supplied evidence.

Technical assumptions should be labelled as assumptions, not facts.

A Thought Tree program may require outputs to distinguish between:

explicit source-supported claims;

inferred claims;  
assumptions;  
unresolved questions.

## 5. Quality Validation

Quality validation checks whether the output is useful, coherent and fit for purpose.

Examples:

Is the plot outline detailed enough to support drafting?  
Is the technical design document useful to engineers?  
Is the executive summary concise and decision-ready?  
Is the review report actionable?  
Is the generated TTML modular and inspectable?

Quality validation may be performed through:

LLM review rubrics;  
human review;  
comparison against examples;  
scoring functions;  
test cases;  
downstream performance.

## 6. Downstream Readiness Validation

Downstream readiness validation checks whether an output is suitable for the operations that depend on it.

For example:

A source\_digest must be detailed enough to support requirements extraction.  
A character\_system\_brief must be coherent enough to support individual character profiles.  
A technical\_requirements\_register must be specific enough to support architecture drafting.  
A draft\_generated\_ttml must be valid enough to pass executor validation.  
This form of validation is practical rather than abstract. It asks:

Can the next step safely use this output?

### Contract Outcomes

A contract check should produce a clear outcome.

Recommended outcomes include:

#### Pass

The output satisfies the contract and execution may continue.

#### Pass with Warnings

The output is usable, but issues were detected.

Warnings should be recorded in the execution trace and may be passed to downstream operations.

#### Fail and Retry

The output does not satisfy the contract, but the engine may retry or repair the operation automatically.

### Fail and Request Review

The output requires human or specialised review before execution continues.

### Fail Terminally

The output cannot be accepted and the workflow should stop unless an explicit recovery path exists.

### Contract Failure Handling

When a Semantic Contract fails, the Cognitive Engine should not silently continue unless the contract is advisory.

Possible failure handling strategies include:

Retry the operation

Run the same operation again, possibly with adjusted model settings or clearer instructions.

Repair the output

Ask an LLM to revise the failed output using the validation report.

Run a review module

Execute a specialised Thought Tree Module designed to diagnose and fix the output.

Fallback to deterministic correction

Use a function to fix formatting, structure or schema issues.

Request human intervention

Pause execution for review, correction or approval.

Create a diagnostic artefact

Generate a report explaining the failure and recommended next steps.

Abort execution

Stop the workflow if continuing would compromise downstream outputs.

All contract failures and recovery attempts should be recorded in the execution trace.

### Contracts and Execution Trace

Semantic validation should be traceable.

For each contract check, the execution trace should record:

Data Unit identifier;

semantic type;

contract used;

validation method;

validation result;

validation timestamp;

validation function or review module used;

LLM model used for review, if applicable;

reviewer identity, if human review occurred;

warnings;

errors;

repair attempts;

final validation status.

This allows users to understand not only what was produced, but also why the output was accepted.

### Contracts and Versioning

When an output fails a contract and is repaired, the failed version should not necessarily be overwritten.

A recommended approach is to preserve versions:

plot\_outline.v1.txt  
plot\_outline.v1.validation\_report.txt  
plot\_outline.v2.txt  
plot\_outline.v2.validation\_report.txt  
or through internal workspace metadata:

Data Unit: plot\_outline

Version 1:

status: failed\_contract

issues: missing climax, weak character motivation

Version 2:

status: passed

repaired\_from: version 1

This supports auditability, comparison and improvement.

### Contracts in TTML

The initial TTML schema may treat contracts as optional or implementation-specific. However, future versions of TTML should support explicit contract references.

A simple form could be:

```
<File
 id="plot_outline"
 extension="txt"
 semanticType="PlotOutline"
 contract="PlotOutlineContract"/>
```

An operation could also declare validation behaviour:

```
<Operation
 id="GeneratePlotOutline"
 type="TextCompletion"
 desc="Generate a coherent plot outline from the story requirements."
 validation="required">
 <FileRef id="story_requirements"/>
 <Output>
 <File
 id="plot_outline"
 extension="txt"
 semanticType="PlotOutline"
 contract="PlotOutlineContract"/>
 </Output>
</Operation>
```



Alternatively, contracts could be declared in a separate section:

```
<Contracts>
 <Contract id="PlotOutlineContract" semanticType="PlotOutline" target="plot_outline">
 <RequiredSection name="Premise"/>
 <RequiredSection name="Central Conflict"/>
 <RequiredSection name="Act I"/>
 <RequiredSection name="Act II"/>
 <RequiredSection name="Act III"/>
 <RequiredSection name="Climax"/>
 <RequiredSection name="Resolution"/>
 <Validator function="ValidateMarkdownSections"/>
 <Review rubric="ReviewNarrativeCompleteness"/>
 </Contract>
</Contracts>
```

The exact syntax may evolve, but the conceptual requirement is clear: important Data Units should be able to declare what makes them valid.

#### Contract Reuse

Contracts should be reusable across Modules.

For example, an organisation may define standard contracts for:

ExecutiveSummary  
RequirementsRegister  
RiskRegister  
ComplianceGapAssessment  
TechnicalDesignDocument  
CreativeBrief  
CharacterProfile  
PlotOutline  
ReviewReport  
TTMLModule

A shared contract library would allow different Thought Tree programs to produce compatible outputs.

This supports:

standardisation;  
quality control;  
module interoperability;  
reusable validation;  
organisational governance;  
benchmarking across engines and models.

#### Contract Strictness

Not all outputs require the same level of validation.

A Thought Tree program may use different contract strictness levels.

#### Advisory

The contract provides guidance, but failure does not stop execution.

Useful for early ideation or low-impact creative outputs.

### Warning

Failure records warnings but allows execution to continue.

Useful when downstream operations can compensate for minor defects.

### Required

The contract must pass before downstream execution continues.

Useful for important intermediate artefacts.

### Approval Required

The contract must pass and receive human approval.

Useful for high-impact, regulated, commercial or safety-sensitive outputs.

### Example:

contract\_policy:

source\_digest: required

plot\_outline: required

beta\_reader\_feedback: advisory

final\_edited\_novel: approval\_required

compliance\_gap\_assessment: approval\_required

Human Review as Contract Validation

Some contracts require judgement that should not be delegated entirely to an LLM.

Human review may be appropriate when:

the output has legal, financial, safety or reputational significance;

the workflow makes strategic business decisions;

creative quality is subjective and important;

source material is ambiguous;

an LLM review produces low confidence;

a generated module changes execution behaviour;

the output will be published externally.

Human review should be treated as a validation method, not as an informal external step.

If a human approves, rejects or edits an output, that decision should be recorded in the execution trace.

### Contracts and Dynamic Modules

Semantic Contracts are especially important for DynamicCompletion and ProjectCompletion operations.

When the Cognitive Engine generates a submodule, the generated module should be validated before execution.

Validation should check:

schema correctness;

execution correctness;

contract coverage;  
input/output compatibility;  
unresolved dependencies;  
unsafe tool or function use;  
output collision risks;  
preservation of the parent operation's intent.

A dynamically generated module should satisfy the parent operation's declared outputs and contracts.

In other words:

Dynamic planning may change how an operation is executed, but it should not change what the operation is required to produce.

#### Contracts and Module Improvement

Semantic Contracts also support module improvement.

A Thought Tree Module can be reviewed by comparing:

its intended task;  
its declared inputs and outputs;  
its operation decomposition;  
its contracts;  
its validation results;  
its execution trace;  
the quality of outputs it produced.

An improvement process may then revise the Module to add:

clearer intermediate artefacts;  
stronger contracts;  
better review operations;  
deterministic validation functions;  
improved output naming;  
safer dynamic planning;  
more useful final outputs.

This allows Thought Tree programs to improve not only their outputs, but also their own production processes.

#### Summary

Semantic Contracts are the mechanism by which a Thought Tree program defines what its outputs are supposed to mean.

They allow the framework to move beyond:

Did the operation produce a file?  
towards:

Did the operation produce the right kind of artefact, with the right structure, quality, meaning and downstream usefulness?

This is essential for cognitive programming.

Without Semantic Contracts, a Thought Tree workflow may appear to execute successfully while passing weak, incomplete or misleading outputs into later operations.

With Semantic Contracts, the Cognitive Engine can validate outputs, trigger repair, request review, preserve provenance and improve reliability over time.

Semantic Contracts, therefore, connect the central goals of the Thought Tree Framework:

- modularity;
- inspectability;
- auditability;
- reusability;
- model independence;
- hybrid execution;
- quality control;
- self-improvement.

They are one of the key features that distinguish a cognitive program from a simple chain of prompts.

## 9. TTML Standard

# TTML Standard

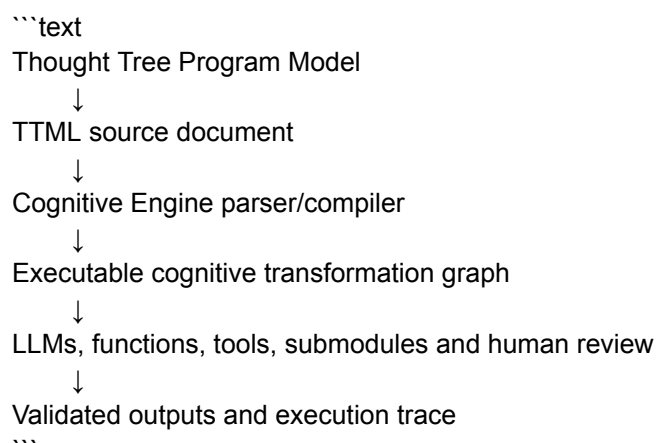
## Thought Tree Markup Language

**Thought Tree Markup Language**, or **TTML**, is the standard XML-based serialisation format for Thought Tree Modules.

A TTML document describes a **cognitive program**: a structured, executable definition of how input Data Units should be transformed into output Data Units through a sequence of Operations. These Operations may invoke LLMs, deterministic functions, external tools, pre-existing Modules, generated submodules, or human review steps, depending on the capabilities of the Cognitive Engine executing the program.

TTML is not itself the whole Thought Tree Framework. Instead, it is one representation of the framework's underlying program model.

The relationship can be understood as:



The purpose of TTML is to provide a shared, inspectable, machine-readable and LLM-authorable format for defining reusable cognitive workflows.

A TTML file should be understandable by:

- a human author;
- an LLM assisting with authoring or review;
- a Cognitive Engine validating and executing the workflow;
- another system importing or converting the workflow;
- a reviewer auditing what the workflow was intended to do.

TTML therefore acts as both a **source language** and an **interchange format** for Thought Tree Modules.

---

## ## Role of TTML in the Framework

The Thought Tree Framework separates the **definition of a cognitive process** from the **execution of that process**.

TTML is used to define the process.

The Cognitive Engine is responsible for execution.

This separation allows the same TTML Module to be executed by different engines, different LLM providers, different deterministic functions, different storage systems, and different review policies.

A TTML Module should describe:

- what inputs are required;
- what intermediate artefacts should be produced;
- what transformations should be performed;
- what outputs are expected;
- how repeated operations should expand;
- how grouped Data Units should be passed;
- what submodules or functions may be required;
- what final outputs should be returned;
- enough structure for the process to be inspected, validated and traced.

A TTML Module should not depend on a specific LLM provider unless that dependency is deliberately introduced through an implementation-specific extension.

The intended abstraction is:

```
```text
```

TTML defines the cognitive process.

The Cognitive Engine decides how to execute it.

The execution trace records what actually happened.

```
```
```

---

## ## Design Goals of TTML

TTML is designed around the following goals.

### ### 1. Human Readability

A TTML document should be readable enough that a human can understand the workflow before it runs.

The reader should be able to answer:

- What is this Module trying to do?
- What inputs does it need?
- What intermediate outputs does it create?
- What operations happen, and in what order?
- What are the final outputs?
- Where are the review, validation or assembly steps?

### ### 2. Machine Executability

A TTML document should be structured enough for a Cognitive Engine to parse, validate, compile and execute.

The engine should be able to:

- load the Module;
- validate the XML structure;
- resolve Data Unit references;
- expand variables and iterators;
- construct an execution plan;
- execute operations;
- detect dependency errors;
- detect output collisions;
- produce declared final outputs;
- record an execution trace.

### ### 3. LLM Authorability

TTML should be explicit and regular enough that LLMs can assist in creating, reviewing, revising and improving Modules.

This is important because the framework is intended to support workflows that can generate or improve other workflows.

A good TTML structure allows LLMs to reason about:

- decomposition;
- missing dependencies;
- unclear operation descriptions;
- invalid output references;

- excessive operation size;
- weak validation;
- reuse opportunities;
- schema conformance.

#### ### 4. Modularity

TTML should support reusable Modules.

A Module should be able to:

- run as a standalone cognitive program;
- be called by another Module;
- call submodules;
- generate submodules;
- be reviewed by another Module;
- be improved by another Module;
- be stored in a module library.

#### ### 5. Model Independence

TTML should describe the intended cognitive work without binding that work to a particular LLM.

The Module defines the transformation.

The Cognitive Engine selects or configures the model, tool or function used to perform the transformation.

#### ### 6. Traceability

TTML should support execution traces by making every major transformation explicit.

Each Operation should have:

- an identifier;
- a type;
- a description;
- declared inputs;
- declared outputs.

This allows the Cognitive Engine to record what happened during execution and relate outputs back to the operations that produced them.

#### ### 7. Extensibility

TTML should be simple enough for early implementations but extensible enough to support future capabilities, such as:

- semantic types;
- contracts;
- validation gates;
- human review operations;

- conditional execution;
- scheduled execution;
- module registries;
- function registries;
- model capability requirements;
- security policies;
- cost constraints;
- test harnesses.

---

## ## TTML as Source Format and Interchange Format

TTML has two related uses.

### ### 1. Source Format

TTML can be written directly by humans or generated by LLMs.

In this role, TTML acts like source code for a cognitive program.

A human or LLM author defines the Module, then the Cognitive Engine compiles and executes it.

### ### 2. Interchange Format

TTML can also be used as a standard exchange format between systems.

For example:

- a visual editor may export TTML;
- an LLM may generate TTML;
- a module library may store TTML;
- a Cognitive Engine may execute TTML;
- an improvement workflow may take TTML as input and produce revised TTML;
- one organisation may share a TTML Module with another.

Future versions of the framework may support other authoring syntaxes, such as YAML, JSON or a visual graph interface. These formats may compile into the same internal Thought Tree Program Model.

TTML remains valuable because XML provides mature tooling for schema validation, parsing, transformation and exchange.

---

## ## TTML and the Thought Tree Program Model

TTML represents the main concepts of the Thought Tree Program Model using XML elements.

| Program Model Concept | TTML Representation | Purpose                                    |
|-----------------------|---------------------|--------------------------------------------|
| Module                | <TTML> document     | Defines one executable Thought Tree Module |



- | Metadata | ``<Project>`` | Describes the Module |
- | Input Data Units | ``<Inputs>`` and ``<File>`` | Declares required starting artefacts |
- | Variables | ``<Vars>`` and ``<Var>`` | Declares reusable values |
- | Iterators | ``<Iterators>`` and ``<Iterator>`` | Defines repeated execution ranges |
- | Collections | ``<Collections>`` and ``<Collection>`` | Groups multiple Data Units |
- | Transformation | ``<Operation>`` | Defines an executable step |
- | Input reference | ``<FileRef>`` / ``<CollectionRef>`` | Refers to existing Data Units or Collections |
- | Operation output | ``<Output>`` inside ``<Operation>`` | Declares produced Data Units |
- | Final Module output | root-level ``<Output>`` | Declares final exported outputs |

This mapping allows TTML to express cognitive workflows as connected transformations over named Data Units.

---

## ## High-Level TTML Structure

A TTML document has the following high-level structure:

```

<?xml
<TTML version="0.12.0">

 <Project ... />

 <Inputs>
 ...
 </Inputs>

 <Vars>
 ...
 </Vars>

 <Iterators>
 ...
 </Iterators>

 <Collections>
 ...
 </Collections>

 <Operations>
 ...
 </Operations>

 <Output>
 ...
 </Output>

</TTML>

```

The major sections are:

1. **\*\*Project\*\***
  - Metadata about the Module.
2. **\*\*Inputs\*\***
  - Data Units required before execution begins.
3. **\*\*Vars\*\***
  - Reusable named values.
4. **\*\*Iterators\*\***
  - Repetition ranges for repeated operation execution.
5. **\*\*Collections\*\***
  - Named groups of Data Units.
6. **\*\*Operations\*\***
  - Ordered transformations that produce intermediate and final artefacts.
7. **\*\*Output\*\***
  - Final outputs exported by the Module.

---

## ## Project Element

The `<Project>` element contains metadata describing the TTML Module.

Example:

```
``xml
<Project
 id="VideoGameTDDGeneration"
 genre="Technical"
 author="Robert Bateman"
 date="2026-06-23"
 desc="Generate a complete video game Technical Design Document from design notes."/>
``
```

The Project element should help a human, LLM or Cognitive Engine understand the purpose of the Module.

Useful metadata may include:

- Module ID;
- description;
- author;
- creation date;
- domain or genre;
- schema version;
- intended use;
- required engine capabilities;

- required functions;
- required submodules;
- expected outputs.

The current schema supports a small set of metadata attributes. Future versions may expand this to support richer module registry and dependency information.

---

## ## Inputs

The ``<Inputs>`` section declares Data Units that must be available before execution begins.

Example:

```
``xml
<Inputs>
 <File id="story_requirements" folder="/requirements" extension="txt"/>
</Inputs>
``
```

A root-level input file represents a Data Unit supplied by the user, workspace, external system or parent Module.

The ``id`` attribute is the logical identifier used inside the TTML document.

The ``folder`` and ``extension`` attributes help the Cognitive Engine locate or construct the corresponding concrete asset.

A Cognitive Engine is responsible for mapping logical identifiers to actual workspace assets.

For example:

```
``xml
<File id="story_requirements" folder="/requirements" extension="txt"/>
``
```

may resolve to:

```
``text
/workspaces/run_001/requirements/story_requirements.txt
``
```

or to an internal asset record in a database-backed workspace.

---

## ## Data Units and File Elements

In the current TTML schema, Data Units are represented primarily as files.

A ``<File>`` declares a Data Unit.

A `<FileRef>` refers to a Data Unit that has already been declared or produced.

Example:

```
```xml
<File id="plot_outline" extension="txt"/>
<FileRef id="plot_outline"/>
```
```

The `id` is not merely a filename. It is a logical identifier for a Data Unit in the workflow.

In future versions, Data Units may support richer metadata, such as:

- semantic type;
- content contract;
- MIME type;
- validation status;
- provenance;
- version;
- sensitivity level;
- retention policy.

For the current TTML standard, the primary Data Unit attributes are:

- `id`;
- `folder`, where applicable;
- `extension`.

---

### ## Variables

The `<Vars>` section declares reusable values.

Example:

```
```xml
<Vars>
  <Var name="ChapterCount" type="integer" value="12"/>
</Vars>
```
```

Variables may be referenced elsewhere using double curly braces:

```
```xml
{{ChapterCount}}
```
```

Example:

```
```xml
```

```
<Iterator id="ChapterIterator" from="1" to="{{ChapterCount}}"/>
...
```

Variables are useful for:

- iterator ranges;
- file identifiers;
- operation descriptions;
- module configuration;
- reusable constants;
- execution parameters.

A Cognitive Engine should resolve variable references before operation execution.

Unless otherwise specified, variables are local to the current Module.

Iterators

The ``<Iterators>`` section declares repeated execution ranges.

Example:

```
```xml
<Iterators>
 <Iterator id="ChapterIterator" from="1" to="{{ChapterCount}}"/>
</Iterators>
```
```

If ``ChapterCount`` resolves to ``12``, then ``ChapterIterator`` represents the values:

```
```text
1, 2, 3, ..., 12
```
```

Iterator values can be inserted into identifiers:

```
```xml
<File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
```
```

This represents a family of output Data Units:

```
```text
draft_chapter_1
draft_chapter_2
...
draft_chapter_12
```
```

An operation expands over its active iterators.

An iterator becomes active when it appears directly in:

- a scalar ``<FileRef>`` inside an Operation;
- a scalar ``<File>`` inside an Operation output;
- a ``<CollectionRef>`` identifier;
- the operation description or attributes, if supported by the Cognitive Engine.

A Cognitive Engine should expand iterator-dependent operations before execution and detect output collisions.

Collections

Collections are named groups of Data Units.

They allow multiple files or generated artefacts to be passed as one logical input or output.

Example:

```
```xml
<Collection id="all_revised_chapters" orderBy="ChapterIterator">
 <FileRef id="revised_chapter_{{ChapterIterator}}"/>
</Collection>
```
```

If ``ChapterIterator`` ranges from 1 to 12, this Collection represents:

```
```text
revised_chapter_1
revised_chapter_2
...
revised_chapter_12
```
```

A Collection can then be passed into an Operation:

```
```xml
<CollectionRef id="all_revised_chapters"/>
```
```

Collections are useful for:

- aggregating generated files;
- passing many Data Units into one LLM completion;
- compiling groups of outputs;
- avoiding accidental iterator collisions;
- making grouped dependencies explicit;
- passing review sets into review operations.

A ``<CollectionRef>`` should not automatically cause an Operation to execute once per member of the Collection.

Instead, the Cognitive Engine resolves the Collection into an ordered group of Data Units and supplies that group to the Operation as grouped context.

Operations

The ``<Operations>`` section defines the executable transformations in the Module.

Operations are normally executed in document order, subject to dependency resolution, iterator expansion, error handling and any safe parallelisation performed by the Cognitive Engine.

Example:

```
``xml
<Operation
  id="DraftOutline"
  type="TextCompletion"
  desc="Draft a plot outline from the story requirements.">

  <FileRef id="story_requirements"/>

  <Output>
    <File id="plot_outline" extension="txt"/>
  </Output>

</Operation>
``
```

An Operation has:

- an ``id``;
- a ``type``;
- a ``desc``;
- zero or more input references;
- one or more declared outputs;
- optional type-specific attributes, such as ``target`` or ``function``.

The ``desc`` attribute describes the intended transformation.

The input references identify the Data Units or Collections used as context.

The Operation output declares the Data Units or Collections expected to be produced.

Operation Types

The current TTML standard defines several operation types.

TextCompletion

A `TextCompletion` Operation invokes an LLM to produce one or more output Data Units.

Example:

```
```xml
<Operation
 id="AnalyzeRequirements"
 type="TextCompletion"
 desc="Analyze the project requirements and identify goals, constraints, risks and open questions.">

 <FileRef id="project_requirements"/>

 <Output>
 <File id="requirements_analysis" extension="txt"/>
 </Output>

</Operation>
```
```

The Cognitive Engine constructs an LLM request using:

- the Operation description;
- resolved input Data Units;
- relevant variables;
- active iterator values;
- Module metadata;
- engine-level system instructions;
- declared output requirements.

ExecuteFunction

An `ExecuteFunction` Operation invokes deterministic code rather than an LLM.

Example:

```
```xml
<Operation
 id="CompileDocument"
 type="ExecuteFunction"
 desc="Concatenate all drafted sections into a single document."
 function="ConcatenateFiles">

 <CollectionRef id="all_document_sections"/>

 <Output>
 <File id="compiled_document" extension="md"/>
 </Output>

</Operation>
```
```


...

This is suitable for tasks such as:

- file copying;
- concatenation;
- format conversion;
- XML validation;
- archiving;
- database queries;
- checksum generation;
- image resizing;
- deterministic data transformation.

The `function` attribute identifies the function to execute.

A Cognitive Engine should validate that the requested function exists before execution.

PreExisting

A `PreExisting` Operation executes another TTML Module.

Example:

```
```xml
<Operation
 id="ConceptDevelopment"
 type="PreExisting"
 desc="Run the concept development submodule."
 target="/subprojects/ConceptDevelopment.ttml">

 <FileRef id="story_requirements"/>

 <Output>
 <File id="plot_outline" extension="txt"/>
 <File id="setting_overview" extension="txt"/>
 </Output>

</Operation>
```
```

The `target` attribute identifies the TTML Module to execute.

The Cognitive Engine should:

1. load the target Module;
2. validate it;
3. map parent inputs to submodule inputs;
4. execute the submodule;
5. map submodule outputs back to parent outputs;
6. record the mapping in the execution trace.

ProjectCompletion

A `ProjectCompletion` Operation allows the Cognitive Engine to generate a new TTML submodule to complete a complex or ambiguous task.

Example:

```
``xml
<Operation
  id="DevelopConcepts"
  type="ProjectCompletion"
  desc="Develop character, setting and plot concepts from the story requirements.">

  <FileRef id="story_requirements"/>

  <Output>
    <File id="plot_outline" extension="txt"/>
    <File id="setting_overview" extension="txt"/>
  </Output>

</Operation>
``
```

A ProjectCompletion Operation may be compiled into a generated submodule containing multiple smaller Operations.

The generated submodule should be retained as part of the execution trace.

Where possible, the Cognitive Engine should validate and optionally present the generated submodule for review before executing it.

DynamicCompletion

A `DynamicCompletion` Operation allows the Cognitive Engine to decide how best to complete the Operation.

It may resolve into:

- a direct LLM completion;
- a deterministic function call;
- a call to a pre-existing Module;
- a generated submodule;
- a combination of these strategies.

Because this introduces ambiguity, the chosen execution strategy should be recorded in the execution trace.

DynamicCompletion is useful for high-level authoring, but it should be used carefully in production workflows where auditability, cost control and predictability are important.

Operation Inputs

An Operation may receive input Data Units using `<FileRef>` elements.

Example:

```
``xml
<FileRef id="plot_outline"/>
<FileRef id="setting_overview"/>
``
```

An Operation may also receive grouped input using `<CollectionRef>`.

Example:

```
``xml
<CollectionRef id="all_character_profiles"/>
``
```

Before executing an Operation, the Cognitive Engine should resolve every input reference.

A reference may resolve to:

- a root-level input;
- an output from an earlier Operation;
- a generated output from an iterator expansion;
- a Collection;
- a submodule output;
- a workspace asset;
- an implementation-specific external reference.

If a required input cannot be resolved, the Operation should not execute unless the Cognitive Engine has an explicit recovery strategy.

Operation Outputs

Each Operation declares the outputs it is expected to produce.

Example:

```
``xml
<Output>
  <File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
</Output>
``
```

Outputs are important because they define the expected result of the Operation and create dependencies for later Operations.

A Cognitive Engine should verify that each Operation produces its declared outputs.

If an output already exists, the engine should not silently overwrite it.

Instead, it should use one of the following strategies:

- reject the operation as an output collision;
- create a versioned output;
- overwrite only if explicitly allowed;
- preserve both and require resolution;
- invoke an automated or human review step.

Silent overwriting should be invalid by default.

Final Output

The root-level `<Output>` element declares the final outputs of the Module.

Example:

```
``xml
<Output>
  <FileRef id="final_edited_novel"/>
</Output>
``
```

A Module is complete when:

- all required Operations have completed;
- all final output references can be resolved;
- outputs are available in the workspace;
- required validation has passed;
- no unrecovered execution errors remain;
- the execution trace has been recorded.

The final output section should only reference Data Units or Collections that the Module can actually produce or resolve.

Minimal TTML Example

The following is a minimal TTML Module that summarises an article and then reviews the summary.

```
``xml
<?xml version="1.0" encoding="UTF-8"?>

<TTML version="0.12.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="TTMLSchema.xsd">
```

```

<Project
  id="ArticleSummaryReview"
  genre="Analytical"
  author="Example Author"
  date="2026-06-23"
  desc="Summarise an article and review the summary for accuracy." />

<Inputs>
  <File id="source_article" folder="/inputs" extension="txt"/>
</Inputs>

<Vars/>

<Iterators/>

<Operations>

  <Operation
    id="SummariseArticle"
    type="TextCompletion"
    desc="Create a concise summary of the source article, preserving the main argument, key
evidence and important caveats.">

    <FileRef id="source_article"/>

    <Output>
      <File id="article_summary" extension="txt"/>
    </Output>

  </Operation>

  <Operation
    id="ReviewSummary"
    type="TextCompletion"
    desc="Review the article summary against the source article. Identify omissions, inaccuracies,
unsupported claims and suggested corrections.">

    <FileRef id="source_article"/>
    <FileRef id="article_summary"/>

    <Output>
      <File id="summary_review" extension="txt"/>
    </Output>

  </Operation>

  <Operation
    id="ReviseSummary"
    type="TextCompletion"
    desc="Revise the summary using the review. Produce a final accurate summary only.">

    <FileRef id="article_summary"/>

```

```

    <FileRef id="summary_review"/>

    <Output>
      <File id="final_article_summary" extension="txt"/>
    </Output>

  </Operation>

</Operations>

<Output>
  <FileRef id="final_article_summary"/>
</Output>

</TTML>
'''

```

This example illustrates the basic TTML pattern:

```

'''text
input article
  ↓
summarise
  ↓
review
  ↓
revise
  ↓
final summary
'''

```

Even in this small example, the process is more inspectable than a single prompt because it preserves intermediate artefacts and separates generation from review.

TTML Schema

TTML is defined by an XML Schema Definition, or XSD.

The schema defines the permitted structure of a TTML document, including:

- required root element;
- required metadata;
- supported Operation types;
- valid File and FileRef elements;
- variable and iterator structure;
- Collection and CollectionRef structure;
- Operation structure;
- final Output structure.

Schema validation confirms that a TTML document is structurally valid XML and follows the declared TTML rules.

However, schema validation alone does not prove that a Module is executable or semantically correct.

A TTML document may pass schema validation while still containing:

- unresolved FileRefs;
- missing input files;
- invalid function references;
- unavailable target submodules;
- output collisions;
- unclear Operation descriptions;
- weak decomposition;
- unusable outputs;
- missing semantic validation.

For this reason, Cognitive Engines should perform both:

1. **schema validation**, and
2. **execution validation**.

Advanced engines should also perform:

3. **semantic validation**.

Validation Levels

TTML supports three levels of correctness.

1. Schema Correctness

The TTML document conforms to the XML schema.

This answers:

```
```text
Is the document structurally valid TTML?
```
```

2. Execution Correctness

The Cognitive Engine can resolve dependencies, expand iterators, call required functions, execute submodules and produce declared final outputs.

This answers:

```
```text
Can this Module run?
```
```

3. Semantic Correctness

The produced Data Units satisfy the content, quality, structure and purpose required by the workflow.

This answers:

```
```text
Did the Module produce the right kind of result?
```
```

All three levels are important.

A Thought Tree workflow should not be considered successful merely because it produced files. It should be considered successful when it produced the expected artefacts and those artefacts satisfy the requirements of downstream use.

TTML and Semantic Contracts

The current TTML schema primarily defines structural execution.

Future versions of TTML should support explicit semantic contracts.

A semantic contract defines what a Data Unit is expected to contain or satisfy.

For example, a `PlotOutline` contract may require:

- premise;
- major characters;
- act structure;
- inciting incident;
- midpoint;
- climax;
- resolution;
- character arc integration;
- continuity notes.

A `TechnicalRequirementsRegister` contract may require:

- functional requirements;
- non-functional requirements;
- assumptions;
- dependencies;
- risks;
- open questions;
- source traceability.

Contracts may be validated using:

- deterministic validation functions;

- schema checks;
- LLM review rubrics;
- human approval;
- hybrid review workflows.

A future TTML version may allow contracts to be attached directly to Data Units or Operations.

Example of a possible future extension:

```
```xml
<File
 id="plot_outline"
 extension="txt"
 semanticType="PlotOutline"
 contract="ThreeActPlotOutlineContract"/>
```
```

This is not required in the current base schema, but it represents an important direction for the standard.

Compilation into an Execution Graph

A Cognitive Engine should not treat TTML as merely a list of prompts.

Instead, it should compile the TTML document into an executable cognitive transformation graph.

This compilation process may include:

1. parsing XML;
2. validating the schema;
3. resolving variables;
4. resolving input declarations;
5. resolving FileRefs and CollectionRefs;
6. expanding iterators;
7. identifying active iterator dimensions;
8. detecting output collisions;
9. resolving function and submodule references;
10. building dependency relationships;
11. constructing operation instances;
12. planning execution order;
13. identifying safe parallelisation opportunities;
14. preparing trace metadata.

The compiled graph is the executable representation of the Module.

The graph may then be executed by invoking:

- LLMs;
- deterministic functions;
- external tools;

- pre-existing Modules;
- generated Modules;
- human review steps.

This compilation model is what allows TTML to function as a cognitive programming language rather than as a simple prompt template format.

Authoring Guidance

A good TTML Module should be clear, modular and inspectable.

When authoring TTML, the author should:

1. ****Start with the final output****
 - Define what the Module is supposed to produce.
2. ****Identify required inputs****
 - Declare the Data Units needed before execution begins.
3. ****Decompose the task****
 - Break large cognitive work into smaller Operations.
4. ****Preserve useful intermediate artefacts****
 - Avoid hiding important reasoning or production stages inside a single Operation.
5. ****Use deterministic functions for mechanical work****
 - Do not ask an LLM to concatenate files, validate XML or copy assets when code can do it reliably.
6. ****Use LLM operations for semantic work****
 - Use LLMs for analysis, synthesis, drafting, planning, review, transformation and interpretation.
7. ****Make dependencies explicit****
 - Later Operations should reference the outputs they actually need.
8. ****Use Collections for grouped inputs****
 - Avoid accidental iterator expansion when the intent is aggregation.
9. ****Avoid output collisions****
 - Include active iterators in output identifiers unless aggregation is intended.
10. ****Add review and validation stages where quality matters****
 - Important outputs should be reviewed before being passed downstream.
11. ****Use clear identifiers****
 - Identifiers should describe the role of the Data Unit, not merely its position.
12. ****Keep Modules reusable****
 - Avoid hard-coding project-specific assumptions unless required.
13. ****Design for traceability****

- Each important transformation should produce an inspectable output.

14. ****Design for improvement****

- A Module should be understandable enough that another Module or human reviewer can improve it.

TTML Limitations

TTML is intentionally simple, but this simplicity creates limitations.

XML Verbosity

XML is explicit and easy to validate, but it can be verbose.

For complex workflows, visual editors or higher-level syntaxes may be preferable for authoring.

Schema Validation Is Limited

The XML schema can confirm structure, but it cannot guarantee that:

- dependencies exist;
- outputs are useful;
- Operation descriptions are good;
- LLM outputs satisfy intended quality;
- the workflow is well-designed.

Execution validation and semantic validation are therefore required.

File-Based Data Unit Representation

The current schema represents Data Units primarily as files.

This is practical and easy to understand, but future versions may need richer artefact models supporting:

- database records;
- vector queries;
- images;
- audio;
- video;
- structured JSON objects;
- tool outputs;
- streamed data;
- transient memory;
- semantic type metadata.

Limited Control Flow

The current base TTML model is primarily ordered and iterator-based.

Advanced behaviours such as conditionals, loops, scheduled execution and event-driven execution may require future schema extensions or engine-specific operation types.

Dynamic Execution Requires Governance

Operations such as `DynamicCompletion` and `ProjectCompletion` are powerful, but they introduce ambiguity.

Engines should make generated plans and submodules inspectable, traceable and, where appropriate, subject to human approval.

Versioning

Each TTML document declares a version:

```
``xml
<TTML version="0.12.0">
``
```

The version identifies the TTML schema and execution semantics expected by the Module.

Cognitive Engines should record:

- TTML version;
- schema version;
- engine version;
- function registry version;
- model configuration;
- execution timestamp.

Future changes to the TTML standard should preserve backwards compatibility where practical.

Breaking changes should result in a new version number and migration guidance.

Conformance

A basic TTML-conformant Cognitive Engine should support:

- loading TTML documents;
- validating against the TTML schema;
- resolving root-level inputs;
- resolving FileRefs;
- executing Operations in document order;
- supporting `TextCompletion`;
- supporting `ExecuteFunction` where functions are registered;
- resolving final outputs;
- recording an execution trace.

A more complete engine should also support:

- variables;
- iterators;
- Collections;
- CollectionRefs;
- output collision detection;
- submodule execution;
- generated submodules;
- semantic contracts;
- validation gates;
- human review;
- caching;
- parallel execution;
- module registries;
- function registries;
- model selection;
- automated improvement workflows.

Conformance levels may be defined separately to distinguish basic executors from advanced Cognitive Engines.

Summary

TTML is the current standard serialisation format for Thought Tree Modules.

It allows cognitive programs to be written, exchanged, validated, compiled and executed.

A TTML document describes:

- inputs;
- variables;
- iterators;
- Collections;
- Operations;
- intermediate artefacts;
- final outputs.

The Cognitive Engine compiles the TTML into an executable transformation graph and carries out the workflow using LLMs, deterministic functions, tools, submodules and review processes.

TTML's purpose is not merely to organise prompts. Its purpose is to make LLM-assisted work programmable, inspectable, reusable, traceable and improvable.

In the wider Thought Tree Framework, TTML provides the shared source format for a new kind of cognitive programming: one based on concepts, relationships, transformations, semantic contracts and execution traces.

10. Authoring Thought Tree Modules

Authoring a Thought Tree Module means designing a reusable cognitive program: a structured process that transforms input Data Units into useful output Data Units through explicit, inspectable operations.

A good Module does not merely ask an LLM to produce an answer. It describes the cognitive production process by which the answer should be produced, reviewed, validated and returned. The aim is to make complex LLM-assisted work modular, auditable, reusable and improvable.

A Thought Tree Module should therefore be designed like a small program: it should have clear inputs, named intermediate artefacts, ordered transformations, declared outputs, validation expectations and traceable dependencies.

1. Authoring Goal

The goal of Module authoring is to convert an intended task into an executable structure.

A task such as:

> “Create a technical design document from these game design notes.”

should be transformed into a sequence of explicit steps, such as:

1. normalise the source notes;
2. create a structured digest;
3. extract technical requirements;
4. identify game scope and product intent;
5. design system decomposition;
6. draft individual document sections;
7. assemble the draft document;
8. review the draft;
9. revise into a final document;
10. return the final Technical Design Document.

This makes the workflow inspectable. Instead of relying on one large completion, the Module produces intermediate artefacts that can be reviewed, corrected, reused or replaced.

2. Start With the Intended Transformation

Every Module should begin with a clear statement of the transformation it performs.

A useful authoring pattern is:

```text

Given [input Data Units],  
produce [output Data Units],

by performing [major transformations],  
subject to [constraints, contracts or review criteria].  
...

Example:

```
```text
Given a collection of video game design notes,
produce a complete Technical Design Document,
by extracting requirements, designing a system decomposition,
drafting document sections, reviewing the assembled draft,
and revising the final document,
while preserving source-supported details and clearly marking assumptions.
```
```

This statement should guide the Module's Project description, operation decomposition and final output definition.

---

### ## 3. Identify Inputs and Final Outputs

Before designing operations, identify the required inputs and intended final outputs.

#### ### Inputs

Inputs are the Data Units the Module needs before execution begins.

Examples:

```
```xml
<Inputs>
  <File id="video_game_design_notes" folder="/inputs" extension="txt"/>
</Inputs>
```
```

Inputs should be named according to their meaning, not merely their storage format.

Prefer:

```
```text
story_requirements
technical_design_notes
policy_documents
customer_feedback_export
```
```

Avoid vague names such as:

```
```text
input1
document
```
```

```
file
data
'''
```

The identifier should tell a human, an LLM and a Cognitive Engine what role the Data Unit plays in the Module.

### ### Final Outputs

Final outputs are the Data Units the Module promises to return when execution completes.

Example:

```
```xml
<Output>
  <FileRef id="video_game_technical_design_document"/>
</Output>
```
```

A good Module should make its final outputs explicit. If a workflow produces many intermediate artefacts but only one final deliverable, only the final deliverable should be declared at the root-level ``<Output>`` unless the intermediate artefacts are intentionally part of the Module's public result.

---

## ## 4. Design Intermediate Artefacts

Intermediate artefacts are one of the main strengths of the Thought Tree approach.

Instead of asking an LLM to jump directly from input to final output, the Module should produce useful intermediate Data Units.

Examples:

```
```text
source_digest
technical_requirements_register
game_intent_and_scope
system_decomposition
draft_tdd_section_01_overview
draft_video_game_tdd
tdd_review_and_correction_plan
```
```

Intermediate artefacts should be:

- meaningful;
- inspectable;
- reusable;
- useful to downstream operations;
- named clearly;
- small enough to validate or revise.



A good intermediate artefact acts like a stable stepping stone in the workflow.

Poor design:

```
```text
notes → final_document
```
```

Better design:

```
```text
notes
  → source_digest
  → requirements_register
  → system_decomposition
  → drafted_sections
  → assembled_draft
  → review_plan
  → final_document
```
```

This approach reduces cognitive load on the LLM and improves traceability.

---

## ## 5. Decompose the Task Into Operations

An Operation is a transformation step.

When authoring a Module, break the task into operations that each have a clear purpose.

A good operation should usually do one main thing:

```
```text
Analyze requirements.
Draft outline.
Review outline.
Revise outline.
Compile sections.
Validate XML.
Generate test cases.
```
```

Avoid operations that combine too many cognitive tasks at once.

Poor operation:

```
```xml
<Operation id="MakeWholeProject" type="TextCompletion" desc="Read the notes, understand everything, create all documents, review them, fix them, and output the final result.">
```
```

Better operations:

```
```xml
<Operation id="CreateSourceDigest" type="TextCompletion" desc="Create a structured digest of the
source notes.">
```
```

```
```xml
<Operation id="ExtractRequirements" type="TextCompletion" desc="Extract a technical requirements
register from the source digest.">
```
```

```
```xml
<Operation id="DraftArchitectureSection" type="TextCompletion" desc="Draft the architecture section
using the requirements register and system decomposition.">
```
```

```
```xml
<Operation id="ReviewDraftDocument" type="TextCompletion" desc="Review the assembled draft for
completeness, consistency, unsupported claims and missing dependencies.">
```
```

```
```xml
<Operation id="FinalizeDocument" type="TextCompletion" desc="Revise the draft using the review
plan and produce the final document.">
```
```

A Module should be decomposed enough that its process is understandable, but not so much that it becomes unnecessarily fragmented.

---

## ## 6. Choose the Right Operation Type

Different kinds of work should use different operation types.

### ### TextCompletion

Use `TextCompletion` for semantic or cognitive work that benefits from an LLM.

Appropriate uses include:

- summarisation;
- analysis;
- drafting;
- synthesis;
- creative generation;
- review;
- critique;
- planning;
- classification;

- extracting meaning from unstructured text.

Example:

```
```xml
<Operation id="AnalyzeStoryRequirements" type="TextCompletion" desc="Analyze the story
requirements and extract the core premise, constraints, genre expectations, target audience and
success criteria.">
  <FileRef id="story_requirements"/>
  <Output>
    <File id="story_requirements_analysis" extension="txt"/>
  </Output>
</Operation>
```
```

### ### ExecuteFunction

Use `ExecuteFunction` for deterministic, mechanical or code-driven work.

Appropriate uses include:

- copying files;
- concatenating files;
- converting formats;
- validating XML;
- querying a database;
- resizing images;
- archiving outputs;
- generating checksums.

Example:

```
```xml
<Operation id="CompileNovel" type="ExecuteFunction" desc="Concatenate the revised chapter files
into one manuscript." function="ConcatenateFiles">
  <CollectionRef id="all_revised_chapters"/>
  <Output>
    <File id="compiled_novel_manuscript" extension="txt"/>
  </Output>
</Operation>
```
```

Do not use an LLM for deterministic work if code can do it more reliably.

Poor design:

```
```text
Ask the LLM to combine twelve chapter files into one manuscript.
```
```

Better design:

```
```text
Use ConcatenateFiles.
```
```

### ### PreExisting

Use `PreExisting` when the operation should execute an existing Module.

This is useful when a task is reusable, complex or already standardised.

Example:

```
```xml
<Operation id="ConceptDevelopment" type="PreExisting" desc="Run the concept development
subproject." target="/subprojects/ConceptDevelopment.html">
  <FileRef id="story_requirements"/>
  <Output>
    <CollectionRef id="all_character_profiles"/>
    <File id="setting_overview" extension="txt"/>
    <File id="plot_outline" extension="txt"/>
  </Output>
</Operation>
```
```

### ### ProjectCompletion

Use `ProjectCompletion` when an operation is too large or ambiguous and should be expanded into a generated submodule.

This should be used carefully. The generated submodule should be retained as part of the execution trace and should ideally be validated before execution.

### ### DynamicCompletion

Use `DynamicCompletion` when the best execution strategy cannot be known at authoring time.

This is powerful but introduces ambiguity. A Cognitive Engine should record the strategy it chooses, and high-impact workflows should require review before dynamically generated plans are executed.

---

## ## 7. Write Good Operation Descriptions

The `desc` attribute is not just a label. It is the semantic instruction that tells the Cognitive Engine and the LLM what the operation is intended to do.

A good operation description should specify:

- the action to perform;
- the intended output;
- relevant constraints;
- quality expectations;

- what not to invent;
- how the output will be used downstream.

Poor description:

```
```xml
desc="Make it better."
```
```

Better description:

```
```xml
desc="Review the draft Technical Design Document for completeness, internal consistency, unsupported claims, duplicated content, missing dependencies, weak assumptions and unclear requirements. Produce a correction plan with concrete edits."
```
```

Poor description:

```
```xml
desc="Create characters."
```
```

Better description:

```
```xml
desc="Generate a distinct and detailed profile for the indexed major character using the unified character system. Include role, motivation, conflict, personality, background, relationships, arc, voice, vulnerabilities, strengths and narrative function."
```
```

Operation descriptions should be specific enough to guide execution, but not so over-specified that they prevent adaptation to the input.

---

## ## 8. Use Clear Data Dependencies

Each operation should explicitly reference the Data Units it needs.

Example:

```
```xml
<Operation id="DraftPlotOutline" type="TextCompletion" desc="Draft a coherent plot outline using the story requirements analysis, character system brief and setting overview draft.">
  <FileRef id="story_requirements_analysis"/>
  <FileRef id="character_system_brief"/>
  <FileRef id="draft_setting_overview"/>
  <Output>
    <File id="draft_plot_outline" extension="txt"/>
  </Output>
</Operation>
```
```

...

Avoid relying on hidden context or assumptions. If an operation needs an artefact, declare it with a `FileRef` or `CollectionRef`.

This improves:

- execution validation;
- traceability;
- reproducibility;
- parallel execution planning;
- debugging;
- human review.

A Cognitive Engine should be able to determine why each operation runs and which artefacts it depends on.

---

## ## 9. Use Collections for Groups of Related Data Units

Collections should be used when a group of Data Units should be treated as one logical input or output.

Example:

```
```xml
<Collection id="all_revised_chapters" orderBy="ChapterIterator">
  <FileRef id="revised_chapter_{{ChapterIterator}}"/>
</Collection>
```
```

This allows a later operation to consume all revised chapters as one grouped input:

```
```xml
<Operation id="CompileNovel" type="ExecuteFunction" desc="Concatenate the revised chapter files
into one manuscript." function="ConcatenateFiles">
  <CollectionRef id="all_revised_chapters"/>
  <Output>
    <File id="compiled_novel_manuscript" extension="txt"/>
  </Output>
</Operation>
```
```

Collections are especially useful for:

- aggregating iterator outputs;
- passing many files into one review operation;
- compiling sections;
- grouping feedback;
- avoiding output collisions;
- making aggregation explicit.

If an operation needs all feedback for a chapter, do not accidentally run once per feedback item and overwrite the same chapter output. Instead, collect the feedback first.

Poor design:

```
```xml
<FileRef id="feedback_chapter_{{ChapterIterator}}_persona_{{PersonalIterator}}"/>
<Output>
  <File id="revised_chapter_{{ChapterIterator}}" extension="txt"/>
</Output>
```
```

This risks multiple operations writing to the same revised chapter.

Better design:

```
```xml
<Collection id="feedback_for_chapter_{{ChapterIterator}}" orderBy="PersonalIterator">
  <FileRef id="feedback_chapter_{{ChapterIterator}}_persona_{{PersonalIterator}}"/>
</Collection>
```
```

```
```xml
<Operation id="ReviseChapter" type="TextCompletion" desc="Revise each chapter using all feedback
for that chapter.">
  <FileRef id="draft_chapter_{{ChapterIterator}}"/>
  <CollectionRef id="feedback_for_chapter_{{ChapterIterator}}"/>
  <Output>
    <File id="revised_chapter_{{ChapterIterator}}" extension="txt"/>
  </Output>
</Operation>
```
```

---

### ## 10. Use Iterators Carefully

Iterators allow one operation definition to expand into many concrete operation instances.

Example:

```
```xml
<Iterator id="ChapterIterator" from="1" to="{{ChapterCount}}"/>
```
```

Used in an output:

```
```xml
<File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
```
```

This can produce:

```
```text
draft_chapter_1
draft_chapter_2
draft_chapter_3
...
```
```

Iterators are powerful, but they can create unintended execution patterns.

When using iterators, authors should check:

- Which iterators are active for the operation?
- How many operation instances will be created?
- Will every output identifier be unique?
- Should the operation run once per item, or once over a collection?
- Is the result a Cartesian product of multiple iterators?
- Are outputs at risk of collision?

If multiple iterators are active, the default behaviour is usually a Cartesian product.

Example:

```
```xml
<FileRef id="draft_chapter_{{ChapterIterator}}"/>
<FileRef id="beta_reader_persona_{{PersonalIterator}}"/>
<Output>
  <File id="feedback_chapter_{{ChapterIterator}}_persona_{{PersonalIterator}}" extension="txt"/>
</Output>
```
```

If there are 12 chapters and 3 personas, this produces 36 feedback files.

That may be correct. But if the output omits one of the active iterators, the Module may produce collisions.

---

## ## 11. Avoid Silent Output Collisions

A Module should not allow multiple operation instances to write silently to the same output identifier.

Poor design:

```
```xml
<Operation id="ReviseChapter" type="TextCompletion" desc="Revise each chapter using persona feedback.">
  <FileRef id="draft_chapter_{{ChapterIterator}}"/>
  <FileRef id="feedback_chapter_{{ChapterIterator}}_persona_{{PersonalIterator}}"/>
</Output>
  <File id="revised_chapter_{{ChapterIterator}}" extension="txt"/>
```
```



```
</Output>
</Operation>
...

```

Here, `Personaliterator` is active, but the output does not include it. Multiple persona-specific executions may attempt to write:

```
```text
revised_chapter_1
...

```

Valid resolutions include:

1. include all active iterators in the output identifier;
2. aggregate the additional iterator dimension into a Collection;
3. add an explicit aggregation operation;
4. use versioned outputs;
5. explicitly declare overwrite behaviour, if supported;
6. require human or automated review.

Preferred solution:

```
```xml
<Collection id="feedback_for_chapter_{{ChapterIterator}}" orderBy="Personaliterator">
 <FileRef id="feedback_chapter_{{ChapterIterator}}_persona_{{Personaliterator}}"/>
</Collection>
...

```

Then:

```
```xml
<Operation id="ReviseChapter" type="TextCompletion" desc="Revise each chapter using all beta
reader feedback for that chapter.">
  <FileRef id="draft_chapter_{{ChapterIterator}}"/>
  <CollectionRef id="feedback_for_chapter_{{ChapterIterator}}"/>
  <Output>
    <File id="revised_chapter_{{ChapterIterator}}" extension="txt"/>
  </Output>
</Operation>
...

```

This makes aggregation explicit and prevents unintended overwrites.

12. Design for Review and Revision

High-quality cognitive workflows should include review and revision stages.

A common pattern is:

```
```text

```

Draft → Review → Revise → Final  
...

Example:

```
```xml
<Operation id="DraftDocument" type="TextCompletion" desc="Draft the document from the source
digest and requirements register.">
  <FileRef id="source_digest"/>
  <FileRef id="requirements_register"/>
  <Output>
    <File id="draft_document" extension="md"/>
  </Output>
</Operation>
```
```

```
```xml
<Operation id="ReviewDraftDocument" type="TextCompletion" desc="Review the draft document for
completeness, consistency, unsupported claims, missing requirements and unclear structure. Produce
a correction plan.">
  <FileRef id="draft_document"/>
  <FileRef id="source_digest"/>
  <FileRef id="requirements_register"/>
  <Output>
    <File id="draft_review_correction_plan" extension="txt"/>
  </Output>
</Operation>
```
```

```
```xml
<Operation id="FinalizeDocument" type="TextCompletion" desc="Revise the draft document using the
correction plan. Produce the final polished document only.">
  <FileRef id="draft_document"/>
  <FileRef id="draft_review_correction_plan"/>
  <Output>
    <File id="final_document" extension="md"/>
  </Output>
</Operation>
```
```

Review stages are especially important when:

- outputs are large;
- the workflow is creative;
- the workflow is analytical;
- source fidelity matters;
- outputs will be used commercially;
- hallucination or unsupported claims would be harmful;
- human approval may be required.

---

## ## 13. Design for Semantic Contracts

A Module should not only produce files. It should produce artefacts that satisfy their intended purpose.

Where supported by the Cognitive Engine, important Data Units should have semantic types and contracts.

For example:

```
```text
plot_outline
semantic type: PlotOutline
contract: includes premise, act structure, major turning points, climax and resolution
```
```

```
```text
technical_requirements_register
semantic type: RequirementsRegister
contract: each requirement is marked explicit, inferred or unresolved
```
```

```
```text
policy_gap_analysis
semantic type: ComplianceGapAnalysis
contract: each gap references a policy obligation and supporting evidence
```
```

Even when the current TTML schema does not fully encode these contracts, authors should design operations as if these expectations exist.

Operation descriptions should state the contract-like requirements for outputs.

Example:

```
```xml
desc="Extract an implementation-focused technical requirements register. Include functional
requirements, nonfunctional requirements, platform requirements, assumptions, dependencies, risks
and open questions. Mark every item as explicit, inferred or unresolved."
```
```

This makes the expected semantic content clear.

---

## ## 14. Design for Reuse

A good Module should be reusable across similar projects.

To improve reusability:

- avoid hard-coding unnecessary project-specific details;
- use generic input names where appropriate;

- separate domain-specific knowledge into input Data Units;
- use variables for counts, limits and configuration;
- use submodules for reusable processes;
- keep final outputs well-defined;
- make assumptions explicit.

For example, a `GenerateTechnicalDesignDocument` Module should not assume one specific game genre unless that is part of the Module's intended purpose.

Better:

```
```text
Input: video_game_design_notes
Output: video_game_technical_design_document
```
```

Less reusable:

```
```text
Input: sci-fi_racing_game_notes_for_project_nebula
Output: nebula_tdd
```
```

Project-specific details can still exist in the input content, but the Module structure should remain reusable where possible.

---

### ## 15. Design for Model Independence

Thought Tree Modules should describe what needs to be done, not which specific LLM provider must do it.

Avoid embedding provider-specific instructions unless necessary.

Poor design:

```
```text
Use GPT-4 to write this section.
```
```

Better design:

```
```text
Draft this section using a high-quality long-context text completion model.
```
```

The Cognitive Engine should decide which model to use based on availability, cost, policy, capability and configuration.

A Module may specify capability requirements, if the framework supports them, such as:

```
```text
requires long-context reasoning
requires structured output
requires high creative quality
requires low-temperature factual extraction
```
```

But the Module should not normally depend on a specific model vendor.

---

## ## 16. Design for Traceability

A Module should make it possible to understand where each output came from.

To improve traceability:

- name operations clearly;
- name intermediate artefacts clearly;
- avoid hidden dependencies;
- preserve review outputs;
- preserve generated submodules;
- preserve validation reports;
- avoid silent overwrites;
- declare final outputs explicitly.

A reader should be able to inspect the execution trace and answer:

- What produced this Data Unit?
- Which inputs were used?
- Which operation ran?
- Which model, function or tool was used?
- Was the output reviewed?
- Was the output revised?
- Was the output validated?
- Which final output depended on it?

Traceability is one of the main differences between a Thought Tree process and a single large prompt.

---

## ## 17. Design for Failure

Authors should assume that some operations may fail or produce weak outputs.

Common failure points include:

- missing inputs;
- invalid references;
- malformed LLM output;
- incomplete output;

- unsupported claims;
- output collisions;
- missing functions;
- invalid generated TTML;
- failed validation;
- excessive context length;
- ambiguous instructions.

A robust Module should include:

- validation operations;
- review operations;
- correction plans;
- retry-compatible outputs;
- human review points where appropriate;
- clear intermediate artefacts that allow partial recovery.

For high-impact workflows, authors should avoid a structure where a single failed operation invalidates the entire process without diagnostic output.

---

## ## 18. Recommended Authoring Process

A practical authoring process is:

### ### Step 1: Define the Module purpose

Write one paragraph describing the intended transformation.

### ### Step 2: Declare inputs and outputs

Identify the required initial Data Units and final deliverables.

### ### Step 3: Identify intermediate artefacts

Decide which intermediate outputs will make the process inspectable and reliable.

### ### Step 4: Decompose into operations

Break the workflow into ordered transformations.

### ### Step 5: Choose operation types

Use LLM operations for semantic work, deterministic functions for mechanical work, submodules for reusable work and dynamic generation only when necessary.

### ### Step 6: Define dependencies

Add `FileRef` and `CollectionRef` references so each operation has the context it needs.

### ### Step 7: Add iterators and collections

Use iterators for repeated structures and Collections for aggregation.

#### ### Step 8: Add review and validation

Insert review, correction and validation stages where quality matters.

#### ### Step 9: Check execution semantics

Validate that:

- all FileRefs can be resolved;
- all functions exist;
- all submodules are available;
- iterator expansion is correct;
- outputs do not collide;
- final outputs are produced.

#### ### Step 10: Test and improve

Run the Module on representative inputs, inspect outputs and revise the Module.

---

### ## 19. Good Module Checklist

A Thought Tree Module is well-authored when:

- the Module purpose is clear;
- inputs are explicit;
- final outputs are explicit;
- intermediate artefacts are meaningful;
- each operation has a clear purpose;
- operation descriptions are specific enough to guide execution;
- LLMs are used for semantic work;
- deterministic functions are used for mechanical work;
- dependencies are declared with `FileRef` or `CollectionRef`;
- Collections are used for grouped inputs;
- iterators do not create unintended collisions;
- review stages exist where quality matters;
- validation expectations are clear;
- generated submodules are retained where applicable;
- output names are consistent;
- the workflow is reusable;
- the workflow is inspectable;
- the workflow can be executed by a conformant Cognitive Engine;
- the execution trace will be understandable after the run.

---

### ## 20. Minimal Example

The following simplified Module takes an article, summarises it, reviews the summary and produces a final revised summary.

```
``xml
<?xml version="1.0" encoding="UTF-8"?>
<TTML version="0.12.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="TTMLSchema.xsd">

 <Project
 id="ArticleSummaryModule"
 genre="Analysis"
 author="Example"
 date="2026-06-23"
 desc="Summarise an article through a draft, review and revision process."/>

 <Inputs>
 <File id="source_article" folder="/inputs" extension="txt"/>
 </Inputs>

 <Vars/>

 <Iterators/>

 <Operations>

 <Operation
 id="DraftSummary"
 type="TextCompletion"
 desc="Create a concise structured summary of the source article. Include main argument, key
evidence, important caveats and unresolved questions. Do not introduce unsupported claims.">
 <FileRef id="source_article"/>
 <Output>
 <File id="draft_summary" extension="txt"/>
 </Output>
 </Operation>

 <Operation
 id="ReviewSummary"
 type="TextCompletion"
 desc="Review the draft summary against the source article. Identify omissions, inaccuracies,
unsupported claims, unclear phrasing and opportunities to improve structure. Produce a correction
plan.">
 <FileRef id="source_article"/>
 <FileRef id="draft_summary"/>
 <Output>
 <File id="summary_review_correction_plan" extension="txt"/>
 </Output>
 </Operation>

 <Operation
 id="ReviseSummary"
```



```

 type="TextCompletion"
 desc="Revise the draft summary using the correction plan. Produce the final summary only.
Ensure it is accurate, concise, well-structured and faithful to the source article.">
 <FileRef id="source_article"/>
 <FileRef id="draft_summary"/>
 <FileRef id="summary_review_correction_plan"/>
 <Output>
 <File id="final_summary" extension="txt"/>
 </Output>
 </Operation>

</Operations>

<Output>
 <FileRef id="final_summary"/>
</Output>

</TTML>
'''

```

This small example illustrates the basic authoring pattern:

```

'''text
Input → Draft → Review → Revise → Final Output
'''

```

Even for a simple task, the Module preserves intermediate artefacts and makes the process inspectable.

---

## ## 21. Summary

Authoring Thought Tree Modules is the practice of designing executable cognitive processes.

A well-authored Module should:

- express a clear transformation;
- decompose complex work into inspectable operations;
- preserve useful intermediate artefacts;
- use LLMs, functions, tools and submodules appropriately;
- declare dependencies explicitly;
- support validation and review;
- avoid hidden state and silent overwrites;
- produce traceable final outputs;
- be reusable across similar tasks.

The purpose of Module authoring is not merely to automate prompting. It is to create cognitive programs: structured, inspectable and improvable processes for transforming concepts and information into validated outputs.

# 11. Examples

This section illustrates how Thought Tree cognitive programs can be used to define, execute and improve LLM-assisted work. The examples are intentionally described at the program-model level rather than only as TTML, because the central idea of the framework is independent of any single serialisation format.

Each example shows how a task can be represented as:

```
```text
Concepts / Data Units
  ↓
Transformations / Operations
  ↓
New Concepts / Data Units
  ↓
Validation / Review / Trace
```
```

The examples also demonstrate the hybrid nature of the framework. Some transformations are performed by LLMs, some by deterministic functions, some by reusable Modules, and some may involve human review.

---

## # 1. Minimal Example: Article Summary and Review

### ## Purpose

This example demonstrates the smallest useful Thought Tree pattern: take an input document, transform it into an output, review the output, and produce a final version.

The task is:

> Given an article, produce a concise summary, review it for accuracy and completeness, then produce a final revised summary.

### ## Concepts

| Concept        | Description                                               |
|----------------|-----------------------------------------------------------|
| Article        | The source document to summarise.                         |
| Draft Summary  | The first generated summary.                              |
| Summary Review | A review of the draft summary against the source article. |
| Final Summary  | The revised summary produced after review.                |

### ## Data Units

| Data Unit        | Semantic Type | Physical Format      |
|------------------|---------------|----------------------|
| `source_article` | Article       | txt, md, html or pdf |
| `draft_summary`  | Summary       | txt or md            |

| `summary\_review` | ReviewReport | txt or md |  
| `final\_summary` | Summary | txt or md |

## ## Transformation Flow

```
```text
source_article
↓
SummariseArticle
↓
draft_summary
↓
ReviewSummaryAgainstArticle
↓
summary_review
↓
ReviseSummary
↓
final_summary
```
```

## ## Operations

### ### 1. Summarise Article

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_article
```
```

Output:

```
```text
draft_summary
```
```

Description:

> Produce a concise summary of the article. Preserve the main claims, supporting points and important qualifications. Do not introduce unsupported information.

---

### ### 2. Review Summary Against Article

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_article
draft_summary
```
```

Output:

```
```text
summary_review
```
```

Description:

> Review the draft summary against the source article. Identify omissions, inaccuracies, unsupported claims, unclear wording and areas where the summary could be more faithful to the source.

---

### ### 3. Revise Summary

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_article
draft_summary
summary_review
```
```

Output:

```
```text
final_summary
```
```

Description:

> Revise the draft summary using the review. Produce a final summary that is concise, accurate, complete and faithful to the source article.

## ## Contracts

The `final\_summary` Data Unit may have a semantic contract such as:

```text

SummaryContract:

- must identify the article's main subject;
- must include the primary claims or findings;
- must not introduce unsupported information;
- must be shorter than the source;
- must be understandable without reading the original article;
- must preserve important caveats or qualifications.

```

## ## Why This Is Useful

This example shows the basic Thought Tree advantage over a single prompt. Instead of asking the LLM to “summarise the article” once, the process preserves:

- the draft summary;
- the review;
- the final summary;
- the relationship between source, draft, review and revision;
- the execution trace showing how the final output was produced.

This makes even a simple task inspectable and improvable.

---

## # 2. Legacy Project Recovery Example

### ## Purpose

This example demonstrates how Thought Tree can turn scattered or incomplete project material into structured documentation.

The task is:

> Given old notes, meeting transcripts, incomplete specifications and miscellaneous files, recover the state of a legacy project and produce a clean project recovery pack.

### ## Concepts

| Concept | Description |

|---|---|

| Source Notes | Unstructured legacy material. |

| Source Digest | Structured summary of all available source material. |

| Project Timeline | Reconstructed history of the project. |

| Requirements Register | Extracted functional and non-functional requirements. |

| Decision Log | Important decisions and their evidence. |

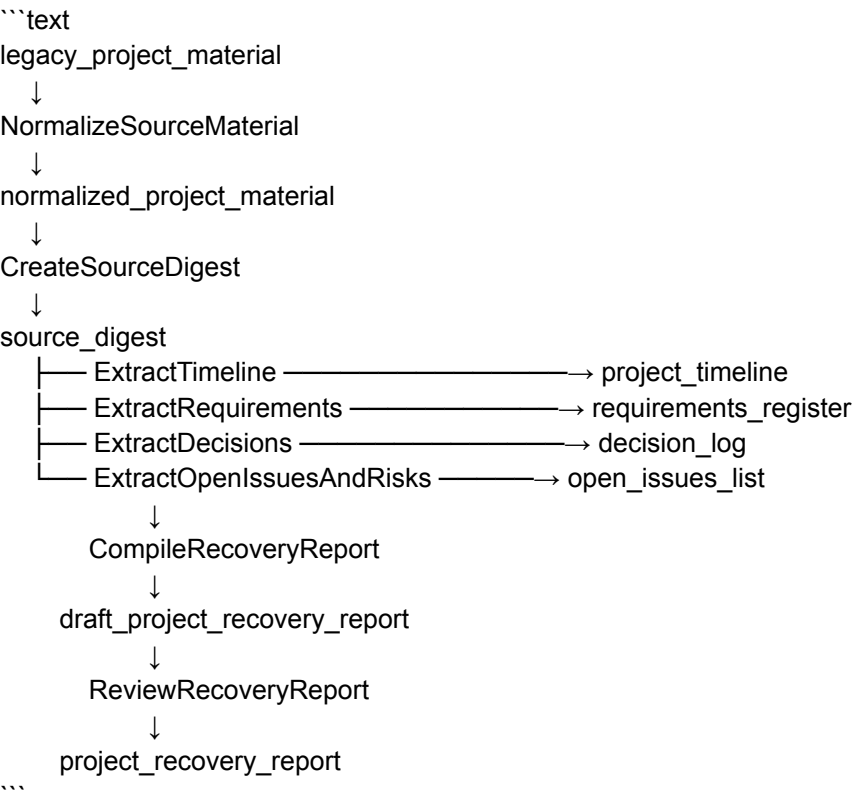
| Open Issues List | Unresolved questions, risks and gaps. |

| Recovery Report | Final coherent project recovery document. |

## Data Units

Data Unit	Semantic Type
`legacy\_project\_material`	SourceCollection
`normalized\_project\_material`	NormalizedTextCollection
`source\_digest`	SourceDigest
`project\_timeline`	Timeline
`requirements\_register`	RequirementsRegister
`decision\_log`	DecisionLog
`open\_issues\_list`	IssueRegister
`project\_recovery\_report`	RecoveryReport

## Transformation Flow



## Operations

### 1. Normalize Source Material

Type:

```

```text
ExecuteFunction
...

```

Function:

```
```text
ConvertDocumentsToText
```
```

Inputs:

```
```text
legacy_project_material
```
```

Output:

```
```text
normalized_project_material
```
```

Purpose:

Convert PDFs, documents, markdown files, meeting notes and other supported source files into readable text.

2. Create Source Digest

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
normalized_project_material
```
```

Output:

```
```text
source_digest
```
```

Purpose:

Create a structured digest of the source material, preserving facts, dates, names, decisions, contradictions and unresolved questions.

3. Extract Requirements

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_digest
```
```

Output:

```
```text
requirements_register
```
```

Purpose:

Extract explicit, inferred and unresolved requirements.

4. Extract Project Timeline

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_digest
```
```

Output:

```
```text
project_timeline
```
```

Purpose:

Reconstruct a chronological timeline of the project.

5. Extract Decisions

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_digest
```
```

Output:

```
```text
decision_log
```
```

Purpose:

Identify important decisions, rationale, dates and supporting evidence.

6. Extract Open Issues and Risks

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_digest
```
```

Output:

```
```text
open_issues_list
```
```

Purpose:

Identify unresolved issues, contradictions, missing information and project risks.

7. Compile Recovery Report

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
source_digest
project_timeline
requirements_register
decision_log
open_issues_list
```
```

Output:

```
```text
draft_project_recovery_report
```
```

Purpose:

Produce a coherent project recovery report.

8. Review Recovery Report

Type:

```
```text
TextCompletion
```
```

Inputs:

```
```text
draft_project_recovery_report
source_digest
requirements_register
decision_log
open_issues_list
```
```

Output:

```
```text
project_recovery_report
```
```

Purpose:

Review and revise the draft report for completeness, source fidelity, clarity and usefulness.

Contracts

The final report may require:

```text

RecoveryReportContract:

- must include project overview;
- must identify known goals and requirements;
- must distinguish confirmed facts from inferred assumptions;
- must list unresolved questions;
- must include risk register;
- must include recommended next actions;
- must avoid unsupported claims;
- must cite or reference source-derived evidence where possible.

```

Why This Is Useful

This example shows the value of preserving intermediate artefacts. A human reviewer can inspect the requirements register, timeline, decision log or open issues independently, instead of only seeing a final generated document.

3. Creative Production Example: Novel Development

Purpose

This example demonstrates how Thought Tree can support a large creative production process.

The task is:

> Given a high-level creative brief, develop a novel by generating concept documents, character profiles, setting material, a plot outline, chapter drafts, simulated reader feedback, revised chapters and a final manuscript.

Concepts

| Concept | Description |

|---|---|

| Story Requirements | Initial creative direction. |

| Character Profile | Description of a major character. |

| Setting Overview | Description of the world, locations and atmosphere. |

| Plot Outline | Structural outline of the story. |

| Chapter Draft | Draft prose for one chapter. |

| Beta Reader Persona | Simulated reader perspective. |

| Feedback Report | Simulated beta reader feedback on a chapter. |

Revised Chapter	Chapter revised using feedback.
Manuscript	Complete assembled novel.
Final Edited Novel	Polished final output.

Data Units

Data Unit	Semantic Type
`story\_requirements`	CreativeBrief
`character\_{{CharacterIterator}}\_profile`	CharacterProfile
`setting\_overview`	SettingOverview
`plot\_outline`	PlotOutline
`draft\_chapter\_{{ChapterIterator}}`	ChapterDraft
`beta\_reader\_persona\_{{PersonalIterator}}`	ReaderPersona
`feedback\_chapter\_{{ChapterIterator}}\_persona\_{{PersonalIterator}}`	FeedbackReport
`revised\_chapter\_{{ChapterIterator}}`	RevisedChapter
`compiled\_novel\_manuscript`	Manuscript
`final\_edited\_novel`	NovelManuscript

Iterators

```
```text
CharacterIterator = 1..CharacterCount
ChapterIterator = 1..ChapterCount
PersonalIterator = 1..PersonaCount
```
```

Collections

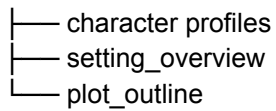
```
```text
all_character_profiles
 = character_1_profile
 character_2_profile
 ...

feedback_for_chapter_{{ChapterIterator}}
 = feedback_chapter_{{ChapterIterator}}_persona_1
 feedback_chapter_{{ChapterIterator}}_persona_2
 ...

all_revised_chapters
 = revised_chapter_1
 revised_chapter_2
 ...
```
```

Transformation Flow

```
```text
story_requirements
 ↓
ConceptDevelopment
```
```



character profiles + setting_overview + plot_outline



DraftChapter[for each chapter]



draft_chapter_n

setting_overview + plot_outline



CreateBetaReaders[for each persona]



beta_reader_persona_n

draft_chapter_n + beta_reader_persona_n



SimulateFeedback[for each chapter/persona pair]



feedback_chapter_n_persona_m

draft_chapter_n + feedback_for_chapter_n



ReviseChapter[for each chapter]



revised_chapter_n

all_revised_chapters



CompileNovel



compiled_novel_manuscript

compiled_novel_manuscript + plot_outline + setting_overview



FinalEditorialPass



final_edited_novel

...

Notable Features

This example demonstrates several core framework capabilities:

Recursive modularity

The `ConceptDevelopment` operation can be implemented as a pre-existing Module or generated submodule.

Iteration

Chapter drafting, persona creation and feedback simulation are expanded over iterators.

Collection aggregation

Feedback from multiple personas is grouped into a chapter-specific Collection before revision.

Hybrid execution

- LLMs generate concepts, drafts, feedback and revisions.
- A deterministic function concatenates revised chapters.
- Optional validation checks continuity, chapter completeness and formatting.

Traceability

The final edited novel can be traced back through:

```
```text
final_edited_novel
← compiled_novel_manuscript
← revised chapters
← feedback reports
← draft chapters
← plot outline / setting / character profiles
← story requirements
```
```

Contracts

Example contracts include:

```
```text
CharacterProfileContract:
- must include role, motivation, conflict, relationships and arc;
- must be distinct from other major characters;
- must support the plot outline.
```

```
ChapterDraftContract:
- must align with the plot outline;
- must use relevant character profiles;
- must preserve continuity;
- must contain a complete chapter-level scene or sequence.
```

```
NovelManuscriptContract:
- must contain all revised chapters in order;
- must be internally consistent;
- must preserve major plot and character arcs;
- must not include editorial notes unless requested.
```
```

Why This Is Useful

This example shows Thought Tree as a production pipeline rather than a single generation prompt. It allows a complex creative output to be decomposed, reviewed, revised and assembled through an inspectable process.

4. Technical Documentation Example: Video Game Technical Design Document

Purpose

This example demonstrates use of Thought Tree for structured technical synthesis.

The task is:

> Given design notes for a video game, produce a complete Technical Design Document suitable for engineers, designers, technical artists, producers and QA.

Concepts

| Concept | Description |
|-----------------------|---|
| Design Notes | Source material describing the game. |
| Source Digest | Structured summary of the source notes. |
| Requirements Register | Extracted technical and product requirements. |
| Game Intent and Scope | Product intent and scope constraints. |
| System Decomposition | Proposed technical system breakdown. |
| TDD Section | One section of the final document. |
| Draft TDD | Assembled technical design document. |
| Review Plan | Review and correction plan. |
| Final TDD | Polished Technical Design Document. |

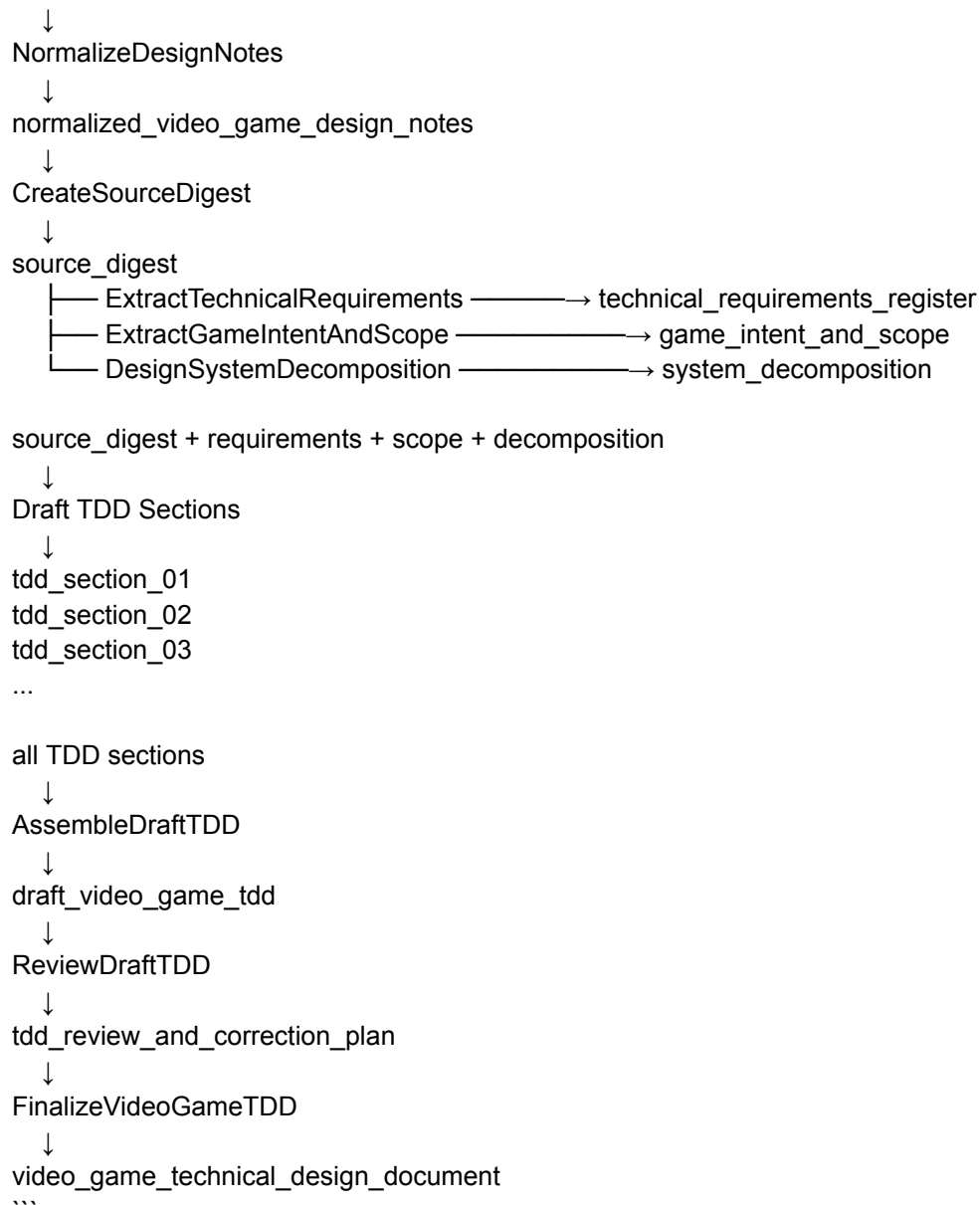
Data Units

| Data Unit | Semantic Type |
|---|------------------------------|
| `video_game_design_notes` | SourceNotes |
| `source_digest` | SourceDigest |
| `technical_requirements_register` | RequirementsRegister |
| `game_intent_and_scope` | ProductScope |
| `system_decomposition` | SystemArchitecturePlan |
| `tdd_section_01_overview` | DocumentSection |
| `tdd_section_02_requirements_constraints` | DocumentSection |
| `tdd_section_03_architecture` | DocumentSection |
| `tdd_section_04_gameplay_systems` | DocumentSection |
| `draft_video_game_tdd` | DraftTechnicalDesignDocument |
| `tdd_review_and_correction_plan` | ReviewPlan |
| `video_game_technical_design_document` | TechnicalDesignDocument |

Transformation Flow

```text

video\_game\_design\_notes



### ## Hybrid Execution

This example uses:

| Operation              | Execution Type         |
|------------------------|------------------------|
| ---                    | ---                    |
| Normalize design notes | Deterministic function |
| Create source digest   | LLM transformation     |
| Extract requirements   | LLM transformation     |
| Draft sections         | LLM transformation     |
| Assemble draft TDD     | Deterministic function |
| Review draft           | LLM transformation     |
| Finalise TDD           | LLM transformation     |

### ## Contracts



The final TDD may have a contract such as:

```
```text
```

TechnicalDesignDocumentContract:

- must include overview, requirements, architecture and implementation sections;
- must distinguish explicit source facts from inferred assumptions;
- must identify unresolved questions;
- must avoid unsupported engine-specific claims;
- must include risks and dependencies;
- must be useful to engineering, design, production and QA audiences;
- must preserve traceability to the source digest and requirements register.

```
```
```

## ## Why This Is Useful

This example shows that Thought Tree is not only for creative work. It can also perform structured organisational synthesis, where auditability, source fidelity and intermediate review are important.

```

```

## # 5. Compliance and Policy Review Example

### ## Purpose

This example demonstrates how Thought Tree can support audit, compliance and governance workflows.

The task is:

> Given policy documents, procedure notes and regulatory obligations, produce a compliance gap analysis and remediation plan.

### ## Concepts

| Concept                | Description                                         |
|------------------------|-----------------------------------------------------|
| Source Policy Material | Internal policy documents and notes.                |
| Regulatory Obligations | Requirements imposed by external rules.             |
| Obligation Register    | Structured list of obligations.                     |
| Evidence Map           | Mapping between obligations and internal evidence.  |
| Gap Analysis           | Identification of missing or weak compliance areas. |
| Risk Register          | Compliance risks and severity.                      |
| Remediation Plan       | Proposed actions to close gaps.                     |

### ## Transformation Flow

```
```text
```

policy_material + regulatory_material

↓

NormalizeSources

↓

normalized_policy_material + normalized_regulatory_material

```

↓
ExtractObligations
↓
obligation_register
↓
MapEvidenceToObligations
↓
evidence_map
↓
IdentifyComplianceGaps
↓
gap_analysis
↓
AssessComplianceRisks
↓
risk_register
↓
GenerateRemediationPlan
↓
remediation_plan
↓
ReviewForAuditReadiness
↓
final_compliance_report
...

```

Important Contracts

```text

ObligationRegisterContract:

- each obligation must be clearly stated;
- each obligation must identify source regulation or policy source;
- each item must be classified by type, priority and applicability.

GapAnalysisContract:

- must distinguish confirmed gaps from possible gaps;
- must identify missing evidence;
- must avoid legal conclusions unless explicitly authorised;
- must include uncertainty and recommended human review points.

RemediationPlanContract:

- must include actions, owners, priorities and dependencies;
- must connect each remediation action to one or more identified gaps;
- must flag high-risk items for human review.

```

Human Review

This example is a good candidate for required human review, because compliance outputs may affect legal, regulatory or operational decisions.

A Thought Tree Module may therefore include a review gate:

```text

HumanReview:

- approve obligation register;
- approve gap analysis;
- approve remediation plan;
- record reviewer identity and decision.

```

Why This Is Useful

The framework makes it possible to separate:

- extraction of obligations;
- evidence mapping;
- gap identification;
- risk assessment;
- remediation planning;
- human approval.

This is more auditable than a single prompt asking for a compliance report.

6. Cyclic Research and Monitoring Example

Purpose

This example demonstrates how a Thought Tree workflow can be executed repeatedly over time.

The task is:

> Monitor a topic, collect new information, summarise changes, assess significance and produce a periodic report.

Concepts

Concept	Description
--- ---	
Monitoring Topic	The subject being tracked.
Source Results	New information gathered during a cycle.
Change Summary	What has changed since the last report.
Significance Assessment	Evaluation of why the changes matter.
Periodic Report	Final report for this cycle.
Historical State	Previous reports and tracked context.

Transformation Flow

```text

monitoring\_topic + historical\_state

↓

RetrieveNewSources

```

↓
new_source_results
↓
SummariseNewInformation
↓
new_information_summary
↓
CompareWithPreviousState
↓
change_summary
↓
AssessSignificance
↓
significance_assessment
↓
GeneratePeriodicReport
↓
periodic_report
↓
UpdateHistoricalState
↓
updated_historical_state
...

```

### ## Execution Pattern

This example may use cyclic execution:

```

```text
Run workflow
Wait for scheduled interval
Run workflow again
Update state
Repeat
```

```

### ## Safeguards

Cyclic workflows should declare:

```

```text
- schedule or trigger;
- maximum runtime;
- maximum cost per cycle;
- source retrieval limits;
- stopping conditions;
- human review requirements;
- state update rules.
```

```

### ## Why This Is Useful

This example shows Thought Tree as more than a one-shot production system. It can support ongoing cognitive processes such as:

- market monitoring;
- competitor analysis;
- regulatory tracking;
- customer feedback reporting;
- project health reporting;
- research briefings.

---

## # 7. Module Authoring Example

### ## Purpose

This example demonstrates the self-referential capability of the framework: using a Thought Tree program to generate another Thought Tree program.

The task is:

> Given a high-level operation description, the TTML schema and available function definitions, generate a valid executable TTML Module that can perform the operation.

### ## Concepts

| Concept                | Description                                  |
|------------------------|----------------------------------------------|
| Original Operation     | The task to convert into a Module.           |
| Operation Requirements | Extracted requirements for the task.         |
| Function Usage Plan    | Deterministic functions that should be used. |
| Module Design          | Planned structure of the generated Module.   |
| Draft TTML             | First generated module definition.           |
| Validation Report      | Schema and execution validation results.     |
| Final TTML             | Corrected executable module.                 |

### ## Transformation Flow

```
```text
original_operation + TTML_schema + function_definitions
    ↓
ExtractOperationRequirements
    ↓
operation_requirements
    ↓
SelectFunctionUsage
    ↓
function_usage_plan
    ↓
DesignTTMLStructure
    ↓
ttml_structure_design
```
```

```

↓
DraftTTMLDocument
↓
draft_generated_ttml
↓
ValidateDraftTTML
↓
validation_reports
↓
AnalyzeValidation
↓
draft_correction_plan
↓
ReviseFinalTTML
↓
NewTTML
...

```

### ## Hybrid Execution

| Step                    | Type            |
|-------------------------|-----------------|
| Extract requirements    | TextCompletion  |
| Select function usage   | TextCompletion  |
| Design TTML structure   | TextCompletion  |
| Draft TTML document     | TextCompletion  |
| Validate against schema | ExecuteFunction |
| Validate with executor  | ExecuteFunction |
| Analyse validation      | TextCompletion  |
| Revise final TTML       | TextCompletion  |

### ## Why This Is Important

This example shows that the framework can be used to author its own programs.

This does not mean the system is automatically correct. The generated TTML still requires validation. But it does mean that Thought Tree can support a powerful workflow:

```

```text
ambiguous task
↓
generated cognitive program
↓
validation
↓
execution
...

```

This is one of the clearest ways in which Thought Tree differs from ordinary prompt chaining.

8. Module Improvement Example

Purpose

This example demonstrates how Thought Tree workflows can improve existing Thought Tree Modules.

The task is:

> Given an existing TTML Module, the operation it was intended to satisfy, the schema, function definitions and evaluation criteria, produce an improved TTML Module and an explanation of the changes.

Concepts

Concept	Description
Original TTML	Existing module to improve.
Operation Requirements	What the module is intended to accomplish.
Schema Validation Report	Whether the XML is structurally valid.
Execution Validation Report	Whether the module can run correctly.
Design Review	Assessment of module structure and quality.
Test Case Specifications	Representative scenarios for evaluation.
Improvement Findings	Combined list of issues and opportunities.
Improved TTML Design	Planned revised module structure.
Draft Improved TTML	Candidate improved module.
Comparison Report	Comparison between original and improved versions.
Final Improved TTML	Final revised module.
Improvement Summary	Human-readable explanation of changes.

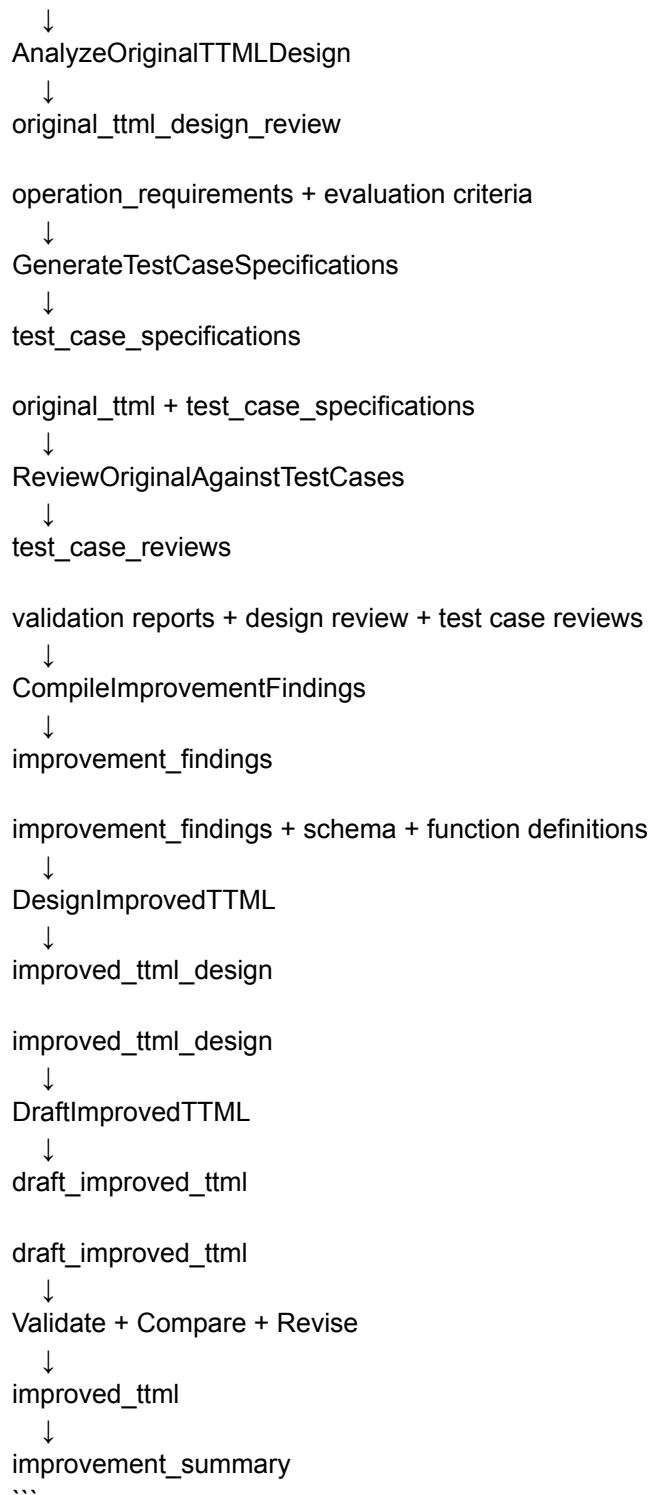
Transformation Flow

```
```text
original_ttml + project_completion_operation + evaluation_criteria
↓
ExtractOperationRequirements
↓
operation_requirements

original_ttml + schema
↓
ValidateOriginalTTMLXSD
↓
original_validation_report_xsd

original_ttml + executor
↓
ValidateOriginalTTMLExecutor
↓
original_validation_report_executor

operation_requirements + original_ttml + validation reports
```



### ## Why This Is Important

This example demonstrates a key strategic property of the framework:

> Thought Tree workflows can be reviewed, tested, revised and improved using the same framework they define.

This makes possible:



- module quality improvement;
- module library maintenance;
- regression testing;
- automated refactoring;
- organisational workflow evolution;
- comparison between versions;
- continuous improvement of cognitive processes.

---

## # 9. Pattern Summary

Across the examples, several reusable patterns appear.

### ## 9.1 Draft → Review → Revise

Used in:

- article summary;
- novel writing;
- technical documentation;
- compliance reporting.

Pattern:

```

```text
source
  ↓
draft
  ↓
review
  ↓
revision
  ↓
final output
```

```

This is one of the most important cognitive programming patterns because it makes quality control explicit.

---

### ## 9.2 Extract → Structure → Synthesize

Used in:

- legacy project recovery;
- technical design document generation;
- compliance analysis.

Pattern:

```
```text
unstructured source material
↓
extracted facts / requirements / obligations
↓
structured intermediate artefacts
↓
synthesised final document
```
```

This is useful when the source material is messy, incomplete or too large for direct final-output generation.

---

### ## 9.3 Iterate → Aggregate → Transform

Used in:

- chapter drafting;
- feedback simulation;
- test case generation;
- batch content generation.

Pattern:

```
```text
item_1
item_2
item_3
...
↓
per-item transformation
↓
collection
↓
aggregate transformation
```
```

This pattern is supported by Iterators and Collections.

---

### ## 9.4 Generate → Validate → Repair

Used in:

- TTML authoring;
- module improvement;
- structured document generation.

Pattern:

```
```text
generated artefact
↓
validation
↓
correction plan
↓
revised artefact
```
```

This is essential when LLMs are asked to produce structured outputs such as XML, JSON, code or formal documents.

---

### ## 9.5 Plan → Approve → Execute

Used in:

- dynamic module generation;
- high-impact compliance workflows;
- large production pipelines.

Pattern:

```
```text
ambiguous task
↓
execution plan or generated module
↓
validation / human approval
↓
execution
```
```

This reduces the risk of allowing dynamic LLM planning to directly execute without inspection.

---

## # 10. What the Examples Demonstrate

Together, these examples show that Thought Tree programs can support a wide range of cognitive work:

| Capability   Demonstrated By                                 |
|--------------------------------------------------------------|
| --- ---                                                      |
| Simple LLM workflow structuring   Article summary            |
| Intermediate artefact preservation   Legacy project recovery |
| Large-scale creative production   Novel development          |
| Structured technical synthesis   TDD generation              |

| Auditability and human review | Compliance analysis |  
| Scheduled/cyclic execution | Research monitoring |  
| Self-authoring | Module generation |  
| Self-improvement | Module improvement |

They also show the main distinction between Thought Tree and ordinary LLM prompting:

> A Thought Tree does not only describe the desired output. It describes the cognitive production process that should create, review, validate and preserve that output.

## 12. Improvement Process

A Thought Tree program can be improved using the same principles it applies to other cognitive work: decomposition, review, validation, testing, revision and traceable output generation.

The Improvement Process is the framework's mechanism for analysing an existing Thought Tree Module, identifying weaknesses, producing a revised version, and preserving a record of what changed and why. It allows Thought Tree programs to become improvable artefacts rather than static workflow definitions.

In this sense, the framework is capable of a limited form of self-improvement: a Thought Tree program may be used to review, test, revise and improve another Thought Tree program.

The purpose of the Improvement Process is not to guarantee that an LLM-generated workflow is automatically correct. Instead, it provides a structured method for making cognitive programs more valid, executable, robust, reusable and aligned with their intended purpose.

Purpose of the Improvement Process

The Improvement Process exists to answer four questions:

Is the module structurally valid?

Does it conform to the relevant schema or source-language rules?

Is the module executable?

Can a Cognitive Engine resolve its inputs, variables, iterators, dependencies, functions, submodules and final outputs?

Is the module semantically well-designed?

Does the workflow decompose the task sensibly, preserve useful intermediate artefacts, include appropriate reviews, and produce outputs that satisfy the intended purpose?

Is the revised module meaningfully better?

Does the improved version increase clarity, robustness, traceability, validation coverage, reusability or output quality?

The Improvement Process therefore treats a Thought Tree Module as both:

an executable cognitive program; and

an artefact that can itself be analysed, reviewed and transformed.

## Inputs to the Improvement Process

An improvement workflow may use some or all of the following inputs:

### 1. Original Module

The existing Thought Tree Module to be reviewed and improved.

This may be:

- a human-authored TTML file;
- an LLM-generated TTML file;
- a module from a library;
- a previously executed module;
- a generated submodule from a ProjectCompletion operation.

### 2. Original Task or Operation

The task the module was intended to complete.

This is important because a module cannot be judged in isolation. A structurally valid module may still fail to satisfy the original goal.

### 3. Framework Schema or Source-Language Definition

For TTML, this would be the TTML XML schema.

For future source formats, this may be a YAML schema, JSON schema, grammar, or abstract program model definition.

### 4. Function Definitions

A list of deterministic functions available to the Cognitive Engine.

This allows the improvement workflow to check whether operations correctly use functions such as:

- ConcatenateFiles;
- CopyFile;
- ConvertDocumentToText;
- ValidateXMLAgainstXSD;
- ValidateTTMLWithExecutor;
- CreateArchive.

### 5. Available Module Library

If the Cognitive Engine supports reusable modules, the improvement workflow may check whether parts of the original module should be replaced by existing modules.

### 6. Semantic Contracts

Contracts that define what important outputs should contain or satisfy.

For example:

- a PlotOutline contract;
- a TechnicalDesignDocument contract;
- a ComplianceGapAnalysis contract;
- a CharacterProfile contract;
- a RiskRegister contract.

### 7. Evaluation Criteria

Explicit criteria for judging whether the module is good.

These may include:

- correctness;
- completeness;
- traceability;
- modularity;
- cost efficiency;
- output quality;
- execution reliability;
- reusability;
- human readability;
- contract coverage.

#### 8. Execution Trace

If the module has already been run, its execution trace may be used as evidence.

The trace can reveal:

- failed operations;
- weak outputs;
- repeated retries;
- unresolved dependencies;
- expensive steps;
- unnecessary model calls;
- validation failures;
- human corrections.

#### Outputs of the Improvement Process

An improvement workflow should normally produce:

##### 1. Validation Reports

Reports showing whether the original and revised modules passed:

- schema validation;
- execution validation;
- contract validation;
- dependency validation;
- iterator expansion validation;
- output collision validation;
- function availability validation.

##### 2. Design Review

A review of the original module's structure and suitability.

This may identify:

- over-large operations;
- missing intermediate artefacts;
- unclear operation descriptions;
- missing review steps;
- missing validation gates;
- weak output naming;
- unresolved assumptions;
- unnecessary ambiguity;

poor decomposition;  
unsafe dynamic behaviour.

### 3. Test Case Specifications

Representative test cases for evaluating the module.

These may not always be executed automatically, but they help define what successful execution should look like.

### 4. Improvement Findings

A consolidated report describing the module's main weaknesses and recommended fixes.

### 5. Improved Module

A revised version of the original module.

The improved module should preserve the original intent while improving execution readiness, clarity, robustness and validation coverage.

### 6. Comparison Report

A comparison between the original and improved module.

This should explain whether the revised version is actually better and identify any remaining limitations.

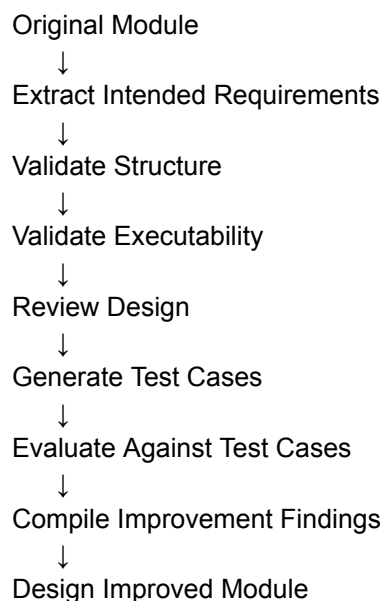
### 7. Improvement Summary

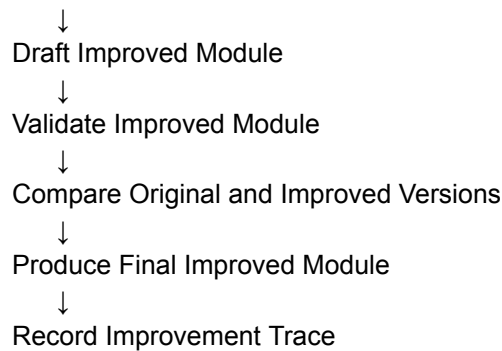
A concise summary of:

what was reviewed;  
what was wrong;  
what changed;  
what validations were performed;  
what risks remain;  
whether human review is recommended.

### Improvement Lifecycle

A typical Improvement Process follows this lifecycle:





Each stage should produce intermediate artefacts so that the improvement process can itself be inspected, challenged and revised.

### 1. Extract Intended Requirements

The first step is to determine what the original module was supposed to do.

The improvement workflow should analyse the original task, operation or project description and extract:

- intended purpose;
- required inputs;
- required outputs;
- expected intermediate artefacts;
- quality expectations;
- constraints;
- assumptions;
- dependencies;
- success criteria;
- failure conditions.

This step is necessary because the module may be invalid not because it is malformed, but because it misunderstands the original intent.

Example output:

operation\_requirements.txt

This artefact becomes the reference point for later validation and comparison.

### 2. Schema Validation

Schema validation checks whether the source document is structurally valid.

For TTML, this means validating the module against the TTML XML schema.

Schema validation may detect:

- invalid XML;
- missing required sections;
- invalid operation types;
- missing required attributes;
- invalid nesting;
- malformed File, FileRef, Collection or CollectionRef elements;
- missing root-level output definitions.

Schema validation confirms that the module is well-formed according to the source-language rules.



However, schema validation alone is not enough. A schema-valid module may still be impossible to execute or may produce poor outputs.

### 3. Execution Validation

Execution validation checks whether the module can actually run.

A Cognitive Engine should validate:

- root-level inputs are available or can be supplied;
- FileRef dependencies can be resolved;
- CollectionRef dependencies can be resolved;
- variables exist and have valid values;
- iterators expand correctly;
- iterator expansion does not create output collisions;
- deterministic functions exist;
- target submodules exist;
- generated outputs can satisfy final output references;
- required model or tool capabilities are available;
- dynamic operations have valid planning or approval behaviour;
- final outputs can be produced.

Execution validation is closer to compilation than schema checking. It verifies whether the source module can be transformed into an executable cognitive graph.

### 4. Semantic and Design Review

Semantic review evaluates whether the module is well-designed for its purpose.

This is where the Improvement Process becomes more than mechanical validation.

The review should consider:

#### Decomposition

Is the task broken into sensible operations?

Poor decomposition may cause:

- overloaded prompts;
- weak intermediate outputs;
- poor traceability;
- difficult debugging;
- excessive reliance on one LLM call.

#### Intermediate Artefacts

Does the module produce useful intermediate artefacts?

Good intermediate artefacts make later review, correction and reuse easier.

#### Operation Clarity

Are operation descriptions clear enough for a Cognitive Engine or LLM to execute reliably?

A weak operation description may be:

- too vague;

too broad;  
missing output expectations;  
missing constraints;  
unclear about source usage;  
unclear about whether invention is allowed.  
Dependency Structure  
Do operations depend on the correct previous outputs?

The review should detect:

missing inputs;  
unnecessary inputs;  
circular dependencies;  
hidden assumptions;  
weak ordering;  
excessive context passed into operations.  
Contracts and Validation  
Are important outputs validated?

A production-ready module should not merely produce files. It should confirm that important files satisfy their intended semantic role.

Review and Revision Loops

Does the module include appropriate review steps?

For example, a creative writing module may need continuity review. A technical document module may need unsupported-claim detection. A compliance module may need evidence traceability checks.

Reusability

Can the module be reused in other projects?

A reusable module should avoid hard-coded assumptions, unclear identifiers, and unnecessary project-specific details.

## 5. Test Case Generation

The Improvement Process should generate representative test cases for the module.

A test case specification should define:

input scenario;  
expected outputs;  
edge cases;  
expected intermediate artefacts;  
validation checks;  
quality criteria;  
failure conditions.

For example, a module that generates a Technical Design Document might have test cases such as:

minimal design notes;  
large unstructured design notes;  
contradictory notes;  
notes with missing platform information;

notes describing a multiplayer game;  
notes describing a single-player game;  
notes containing unsupported assumptions.

The purpose of test case generation is to make evaluation more concrete.

Even if the engine does not execute the tests automatically, the test specifications help reviewers judge whether the module is robust.

## 6. Test-Based Review

The original module should be reviewed against each test case.

The review should ask:

Is the module likely to handle this input scenario?  
Are required outputs produced?  
Are assumptions clearly labelled?  
Are unsupported claims avoided?  
Are edge cases addressed?  
Are validation gates sufficient?  
Are intermediate artefacts useful for diagnosing failure?  
Would the final output satisfy the relevant semantic contract?  
This produces one review artefact per test case.

Example:

test\_case\_1\_review.txt  
test\_case\_2\_review.txt  
test\_case\_3\_review.txt

These reviews can then be aggregated into a consolidated improvement findings report.

## 7. Compile Improvement Findings

The improvement workflow should combine:

schema validation report;  
execution validation report;  
semantic design review;  
test case reviews;  
execution trace evidence, if available;  
human review notes, if available.  
The result is a consolidated findings document.

This document should identify:

critical errors;  
execution blockers;  
schema violations;  
output collision risks;  
missing functions;  
missing submodules;  
weak operations;  
missing contracts;  
missing reviews;

poor naming;  
opportunities for deterministic functions;  
opportunities for module reuse;  
recommended restructuring.

The findings document acts as the specification for the improved module.

## 8. Design the Improved Module

Before drafting a revised module, the Improvement Process should produce an improved design.

This design should describe:

revised inputs;  
revised outputs;  
improved operation sequence;  
added intermediate artefacts;  
removed unnecessary steps;  
added validation gates;  
added review stages;  
added deterministic functions;  
added contracts;  
corrected dependencies;  
corrected iterator use;  
corrected collections;  
improved final output resolution.

This design stage reduces the risk that the improved module is merely a syntactic rewrite rather than a genuine improvement.

## 9. Draft the Improved Module

The revised module should then be generated from the improved design.

The improved module should:

preserve the original intent;  
conform to the relevant schema;  
use valid operation types;  
declare required inputs;  
resolve dependencies clearly;  
avoid output collisions;  
use deterministic functions where appropriate;  
include validation and review operations where needed;  
declare final outputs explicitly;  
remain human-readable;  
be suitable for execution by a conformant Cognitive Engine.

For TTML, the output may be:

improved\_ttml.ttml

For future formats, the output may be another source representation of the same cognitive program.

## 10. Validate the Improved Module

The improved module should undergo the same validation as the original.

At minimum, it should be checked for:

schema validity;  
execution validity;  
dependency correctness;  
iterator expansion correctness;  
output collision avoidance;  
function availability;  
module target availability;  
final output resolvability.

If semantic contracts are supported, the improved module should also be checked for contract coverage.

The improvement workflow should not assume that the revised module is valid simply because it was generated as a correction. LLM-generated improvements can introduce new errors.

## 11. Compare Original and Improved Versions

The Improvement Process should compare the original module and the revised module.

The comparison should evaluate whether the improved module is better according to the stated criteria.

Comparison dimensions may include:

| Dimension          | Question                                              |
|--------------------|-------------------------------------------------------|
| Schema validity    | Does the revised module better conform to the schema? |
| Executability      | Can the revised module be executed more reliably?     |
| Dependency clarity | Are inputs and outputs clearer?                       |
| Decomposition      | Are operations better sized and sequenced?            |
| Traceability       | Are intermediate artefacts more useful?               |
| Validation         | Are semantic contracts or review gates improved?      |
| Robustness         | Does the module handle edge cases better?             |
| Reusability        | Is the module less project-specific?                  |
| Cost efficiency    | Does it avoid unnecessary LLM calls?                  |
| Human readability  | Is it easier to understand and maintain?              |

The comparison report should also identify remaining issues.

An improved module does not need to be perfect. But the process should make clear why it is considered an improvement.

## 12. Produce Final Improved Module

After validation and comparison, the final improved module is produced.

This output should be treated as a new version of the original module.

The Cognitive Engine or module library should preserve:

original module;  
improved module;  
validation reports;  
comparison report;  
improvement summary;  
improvement trace;

version metadata.

The final improved module may then be:

- returned to the user;
- stored in a module library;
- used to replace the original;
- tested further;
- submitted for human review;
- executed as part of a production workflow.

### 13. Record Improvement Trace

Every improvement run should produce an improvement trace.

This trace should record:

- original module identifier;
- original module version;
- improved module version;
- input task or operation;
- evaluation criteria;
- validation results;
- generated test cases;
- design review outputs;
- improvement findings;
- revision decisions;
- model calls used;
- deterministic functions used;
- human review decisions;
- final comparison result;
- unresolved risks.

This makes the improvement process itself auditable.

The framework should avoid silently replacing modules without preserving evidence of why the replacement was made.

### Improvement as a Cognitive Build Process

The Improvement Process can be understood as a cognitive build pipeline.

In ordinary software development, code may be:

written → compiled → tested → reviewed → refactored → released

In the Thought Tree Framework, cognitive programs may be:

authored → validated → executed → reviewed → improved → versioned

This is one of the most important strategic features of the framework.

It means that Thought Tree Modules are not disposable prompts. They are maintainable cognitive assets.

### Improvement and Module Libraries

When Thought Tree Modules are stored in a module library, the Improvement Process becomes especially valuable.

A library module may improve over time through:

- repeated executions;
- user feedback;
- failed run analysis;
- output quality reviews;
- benchmark tests;
- model performance comparisons;
- contract improvements;
- human edits;
- automated refactoring.

An organisation could maintain a private library of high-value modules and improve them as reusable intellectual property.

For example:

- a compliance team may improve its audit-preparation modules;
- a game studio may improve its design-document generation modules;
- a creative studio may improve its world-building and narrative modules;
- a software team may improve its requirements-analysis modules.

The module library then becomes more than a collection of prompts. It becomes a repository of tested, versioned, inspectable cognitive programs.

#### Human Review in the Improvement Process

Automated improvement should not always be accepted without review.

Human review is recommended when:

- the module is high impact;
- the module will be reused widely;
- the module handles sensitive data;
- the module performs compliance, legal, financial or medical work;
- the revised module changes the intended process significantly;
- validation reports contain unresolved warnings;
- dynamic generation introduced new submodules;
- the comparison report is uncertain.

A human reviewer may approve, reject, edit or request another improvement pass.

Any human decision should be recorded in the improvement trace.

#### Risks and Safeguards

The Improvement Process introduces powerful capabilities, but also risks.

##### Risk: Rewriting Away the Original Intent

An LLM may produce a cleaner module that no longer satisfies the original task.

Safeguard:

- always extract and preserve operation requirements;
- compare the improved module against the original intent.

##### Risk: Introducing New Execution Errors

A revised module may fix one problem while creating another.

Safeguard:

validate the improved module independently;  
compare validation reports before accepting changes.

Risk: Overcomplication

An improvement pass may add unnecessary operations, reviews or abstractions.

Safeguard:

include simplicity and cost efficiency in the evaluation criteria.

Risk: Silent Behaviour Changes

A module may appear similar but produce different outputs or use different assumptions.

Safeguard:

produce a comparison report;  
version all changes;  
require human approval for major changes.

Risk: Recursive Uncontrolled Improvement

A system that repeatedly improves modules without limits could drift or degrade.

Safeguard:

require maximum iteration limits;  
preserve all versions;  
use benchmark tests;  
require approval before replacing stable modules.

Minimal Improvement Workflow

A minimal conformant improvement workflow should perform:

requirement extraction;  
schema validation;  
execution validation;  
design review;  
improved module generation;  
improved module validation;  
original-versus-improved comparison;  
final improved module output;  
improvement trace recording.

More advanced engines may additionally support:

generated test cases;  
automated test execution;  
semantic contract validation;  
benchmark scoring;  
module library updates;  
human approval gates;  
regression testing;  
multi-model comparison.

Summary

The Improvement Process makes Thought Tree programs maintainable.



It provides a structured way to take an existing module, understand its intended purpose, validate its structure, test its execution readiness, review its design, generate improvements, compare versions and preserve a trace of the change.

This supports one of the central aims of the Thought Tree Framework:

complex cognitive work should not only be executable; it should be inspectable, testable, reusable and improvable.

By applying the framework to its own modules, Thought Tree turns cognitive workflows into evolving assets that can be refined over time.

## 13. Commercial Applications

The Thought Tree Framework has commercial value wherever organisations need to transform ambiguous, incomplete or high-volume information into structured, reviewable and reusable outputs.

Its primary commercial advantage is that it turns LLM-assisted work from isolated prompting into an executable production process. A Thought Tree program can define not only the desired output, but also the intermediate artefacts, review stages, validation criteria, deterministic functions and execution trace required to produce that output reliably.

This makes the framework suitable for commercial domains where quality, repeatability, traceability and adaptation matter.

### 1. Bespoke Entertainment Media Production

The original commercial motivation for the Thought Tree Framework was the automated production of bespoke entertainment media. In an overcrowded content market, generic content has limited commercial value. Personalised, user-specific content offers a stronger opportunity, but only if production can be scaled without sacrificing coherence or quality.

Thought Tree programs can define production pipelines for generating personalised creative works from a user brief or preference profile.

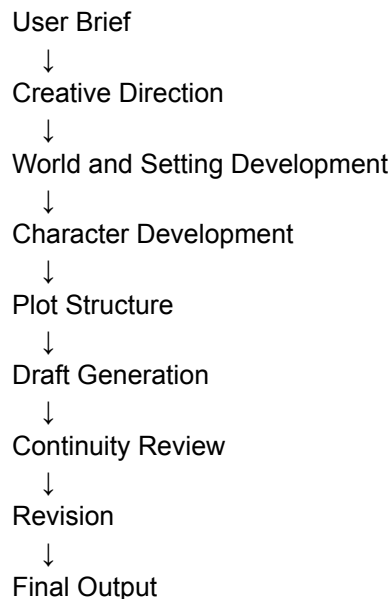
Example outputs include:

- world-building documents;
- character profiles;
- plot outlines;
- chapter drafts;
- dialogue scenes;
- screenplay drafts;
- storyboard descriptions;
- game lore;
- quest chains;
- faction histories;
- art direction briefs;

continuity reviews;

final edited manuscripts.

A simple prompt may generate a piece of content, but it does not reliably preserve the production process. A Thought Tree program can decompose the creative task into stages:



This allows creative production to become modular, repeatable and improvable.

Commercial opportunities include:

personalised fiction;

interactive story generation;

custom tabletop RPG campaigns;

AI-assisted screenwriting;

personalised comics or visual storyboards;

game narrative generation;

bespoke children's stories;

branded entertainment content;

fan-engagement experiences.

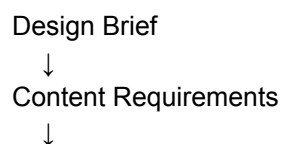
The strongest commercial value is not simply generating text. It is generating coherent media through a traceable production pipeline that can be reviewed, improved and reused.

## 2. Game Content Generation

Games often require large volumes of structured, internally consistent content. This includes items, quests, dialogue, characters, locations, abilities, enemies, factions, tutorials, achievements, flavour text and balancing notes.

The Thought Tree Framework is well suited to game content generation because it can combine creative generation with deterministic formatting and validation.

Example workflows include:



Generate Variants



Review for Gameplay Fit



Balance and Consistency Check



Export to JSON / XML / CSV



Validate Against Game Schema

Example outputs include:

item databases;  
weapon descriptions;  
enemy profiles;  
quest structures;  
dialogue trees;  
NPC biographies;  
ability descriptions;  
lore entries;  
tutorial text;  
level design briefs;  
encounter design notes;  
localisation-ready text;  
implementation-ready structured data.

The framework can also support hybrid production. LLMs can generate creative material, while deterministic functions validate schemas, convert formats, enforce naming conventions, package assets or compare generated content against design constraints.

Commercial benefits include:

faster content production;  
reduced design bottlenecks;  
consistent world-building;  
reusable content generation modules;  
easier localisation preparation;  
cheaper prototyping;  
scalable live-ops content generation;  
structured outputs suitable for direct ingestion by game tools.

This is especially valuable for studios that need to produce large quantities of content while maintaining design consistency.

### 3. Technical Documentation and Project Recovery

Many organisations hold valuable project knowledge in scattered, incomplete or inconsistent forms: notes, meeting transcripts, old design documents, tickets, emails, spreadsheets, code comments and legacy documentation.

Thought Tree programs can transform this material into structured, useful documentation.

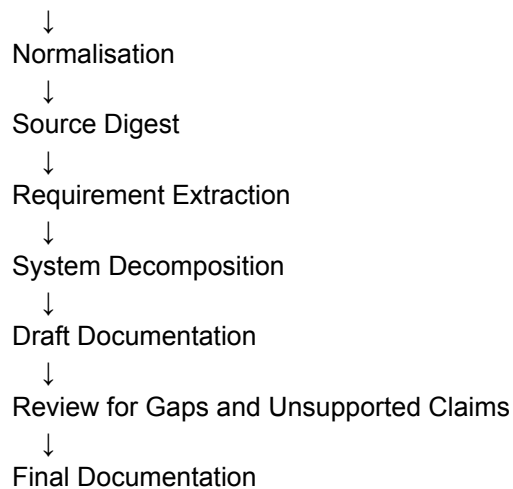
Example outputs include:

requirements documents;  
technical design documents;

onboarding guides;  
implementation notes;  
architecture summaries;  
dependency maps;  
decision logs;  
unresolved issue lists;  
migration plans;  
product briefs;  
test plans;  
support documentation.

A Thought Tree workflow can preserve traceability from source material to final document:

Raw Notes / Meeting Records / Legacy Files



This is commercially valuable because project knowledge is often expensive to recover manually. Thought Tree programs can make the recovery process repeatable, inspectable and auditable.

Potential customers include:

software companies;  
game studios;  
consultancies;  
engineering teams;  
product teams;  
public sector organisations;  
organisations with legacy systems;  
organisations undergoing handover, migration or audit.

The key value is not only document generation. It is turning unstructured organisational knowledge into reusable assets.

#### 4. Compliance, Policy and Audit Preparation

Compliance work often involves transforming large volumes of policy, regulation, evidence and operational material into structured outputs. This work benefits strongly from traceability and review.

Thought Tree programs can support compliance and audit workflows by decomposing the work into explicit stages:

Policies / Regulations / Evidence



Obligation Extraction



Evidence Mapping



Gap Analysis



Risk Assessment



Remediation Plan



Review Pack

Example outputs include:

policy drafts;  
compliance gap analyses;  
obligation registers;  
control matrices;  
risk registers;  
evidence maps;  
audit preparation packs;  
remediation plans;  
review checklists;  
executive summaries;  
board reports.

The framework's execution trace is especially valuable in this domain. Organisations need to know what information was used, how conclusions were reached, what assumptions were made and what still requires human judgement.

Thought Tree does not remove the need for professional review. Instead, it can reduce the cost of preparation and make review more efficient by producing structured intermediate artefacts.

Commercial benefits include:

reduced manual analysis time;  
consistent document structure;  
clearer audit trails;  
easier review by specialists;  
reusable compliance workflows;  
repeatable assessment processes;  
better management of regulatory change.

Potential customers include:

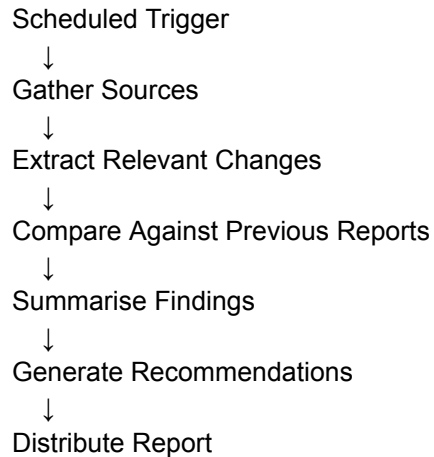
regulated businesses;  
legal and compliance teams;  
consultancies;  
insurers;  
financial services firms;  
healthcare organisations;  
public sector bodies.

5. Research, Monitoring and Reporting

Many commercial teams need to repeatedly gather, analyse and summarise changing information. This includes market intelligence, competitor monitoring, regulatory tracking, customer feedback analysis and internal reporting.

Thought Tree programs can define repeatable reporting pipelines.

Example workflows include:



Example outputs include:

- market reports;
- competitor summaries;
- regulatory briefings;
- customer feedback reports;
- product review summaries;
- risk updates;
- executive dashboards;
- sales intelligence briefs;
- trend analyses;
- weekly or monthly management reports.

The cyclic execution capability described in the framework is commercially important here. A Thought Tree program could run on a schedule, preserve previous outputs, compare new information against historical reports and produce updated summaries.

Commercial benefits include:

- automated recurring reporting;
- faster intelligence gathering;
- consistent report structure;
- preservation of historical context;
- reduced analyst workload;
- scalable monitoring across many topics;
- traceable source-to-summary workflow.

Potential users include:

- strategy teams;
- marketing teams;
- product teams;
- investment analysts;

policy teams;  
consultancies;  
communications teams;  
risk management teams.

## 6. Enterprise Knowledge Transformation

Large organisations often struggle to convert internal knowledge into useful outputs. Information exists in documents, chats, tickets, presentations, wikis, call transcripts and databases, but it is difficult to reuse effectively.

Thought Tree programs can act as reusable cognitive processes for transforming internal knowledge.

Example applications include:

onboarding pack generation;  
meeting-to-action-plan conversion;  
project status summarisation;  
internal knowledge base creation;  
standard operating procedure drafting;  
customer support knowledge extraction;  
sales enablement material generation;  
internal policy simplification;  
training material generation;  
decision history reconstruction.

The framework is especially useful when the same transformation must be performed repeatedly across different departments, projects or clients.

Example:

Project Folder



Extract Key Documents



Summarise Project Purpose



Identify Stakeholders



Extract Decisions and Risks



Generate Onboarding Guide



Review for Missing Information

Commercial benefits include:

improved knowledge reuse;  
reduced onboarding time;  
reduced dependency on individual experts;  
standardised internal documentation;  
better institutional memory;  
auditable knowledge transformation;  
reusable organisational module libraries.

For enterprises, the module library may become a valuable internal asset: a collection of approved cognitive programs that encode how the organisation performs recurring knowledge work.

## 7. Professional Services and Consultancy Automation

Consultancies, agencies and professional services firms often repeat similar analytical and production processes for different clients. The specific input changes, but the structure of the work is often reusable.

Thought Tree programs can encode these repeatable service processes.

Example applications include:

- client discovery analysis;
- proposal generation;
- requirements gathering;
- business case drafting;
- process mapping;
- risk assessment;
- implementation planning;
- workshop output synthesis;
- report generation;
- audit preparation;
- transformation roadmap creation.

A consultancy could maintain a private library of Thought Tree modules:

Client Notes → Discovery Summary

Client Documents → Gap Analysis

Workshop Transcript → Action Plan

Current State Material → Transformation Roadmap

Regulatory Text → Compliance Assessment

Commercial benefits include:

- faster delivery;
- more consistent outputs;
- reusable intellectual property;
- improved junior staff productivity;
- easier quality assurance;
- scalable service delivery;
- clearer audit trails for client work.

The commercial opportunity is particularly strong because Thought Tree modules can become packaged expertise. A firm's methodology can be encoded as reusable cognitive programs.

## 8. Productised Cognitive Workflows

The framework can support software products built around specific cognitive workflows.

Rather than selling general-purpose AI chat, a company could sell a product that executes a well-designed Thought Tree program for a specific outcome.

Examples:

"Generate a technical design document from game design notes."

"Create an audit preparation pack from compliance evidence."

"Turn meeting transcripts into a project recovery report."

"Generate a personalised novel outline."



“Convert customer feedback into a product roadmap.”

“Create a risk register from project documentation.”

“Produce onboarding documentation from a project repository.”

Each product could expose a simple user interface while hiding the underlying Thought Tree execution.

Commercial advantages include:

clearer value proposition than generic AI tools;

repeatable outputs;

easier pricing per workflow;

workflow-specific quality control;

configurable modules for different customers;

reusable backend execution infrastructure.

This makes the Thought Tree Framework suitable not only as an internal tool, but also as a foundation for vertical AI products.

## 9. Module Libraries as Commercial Assets

One of the most important long-term commercial opportunities is the creation of module libraries.

A Thought Tree Module is more than a prompt. It is a reusable cognitive process. Over time, organisations can build libraries of modules that encode domain knowledge, production methods, review processes and quality standards.

Module libraries may be:

private organisational assets;

client-specific libraries;

industry-specific libraries;

open-source community libraries;

commercial module marketplaces.

Examples of commercially valuable module categories:

creative production modules;

compliance analysis modules;

documentation generation modules;

research synthesis modules;

software planning modules;

policy analysis modules;

game content generation modules;

legal preparation modules;

customer insight modules;

educational content modules.

The commercial value of a module library increases over time because modules can be:

versioned;

reviewed;

tested;

improved;

adapted to new models;

combined into larger workflows;

reused across projects.

This creates a form of reusable cognitive intellectual property.

#### 10. Model-Independent AI Infrastructure

A major commercial risk in current LLM adoption is dependency on specific model providers. If workflows are tightly coupled to a particular model, prompt style or API, organisations face switching costs and operational fragility.

The Thought Tree Framework reduces this dependency by separating the workflow definition from the execution backend.

A Thought Tree program defines:

- what inputs are required;
- what transformations should occur;
- what outputs should be produced;
- what validations should pass;
- what trace should be recorded.

The Cognitive Engine decides how to execute those transformations using available models, functions and tools.

This creates commercial flexibility:

- switch model providers without rewriting workflows;
- use different models for different operation types;
- route sensitive work to local models;
- use cheaper models for low-risk steps;
- use stronger models for high-value review stages;
- benchmark model performance across the same workflow;
- preserve workflow IP independently of model vendors.

This is especially important for enterprises, regulated industries and companies building long-lived AI systems.

#### 11. Quality Control and Human-in-the-Loop Production

Fully automated generation is not always commercially appropriate. Many valuable outputs require review, approval or expert judgement.

The Thought Tree Framework supports human-in-the-loop workflows by making review stages explicit.

Examples:

```
graph TD; A[LLM Draft] --> B[Automated Validation]; B --> C[Human Review]; C --> D[Revision]; D --> E[Final Approval]
```

This is commercially useful because it allows organisations to adopt AI without pretending that LLM outputs are automatically correct.

Human review can be used for:

- approving generated plans;
- reviewing high-impact outputs;
- correcting factual or legal risk;
- resolving ambiguous decisions;
- approving generated submodules;
- validating creative direction;
- signing off client deliverables.

The result is a more realistic production model:

Automate the repeatable cognitive labour, preserve the intermediate artefacts, and involve humans where judgement matters.

This is likely to be more commercially credible than promises of complete autonomous replacement.

## 12. Commercial Differentiation

The Thought Tree Framework differs from many current AI automation approaches in several ways.

Compared with single prompts

A single prompt produces an output but does not define a reusable process.

Thought Tree defines the process.

Compared with prompt chains

Prompt chains are often brittle and implementation-specific.

Thought Tree formalises inputs, outputs, dependencies, validation and traceability.

Compared with autonomous agents

Agents can be flexible but may be unpredictable.

Thought Tree favours explicit structure, planned execution and inspectable intermediate artefacts.

Compared with traditional workflow tools

Traditional workflow tools handle deterministic processes well but struggle with ambiguous cognitive work.

Thought Tree combines deterministic workflow execution with LLM-based semantic transformation.

Compared with ordinary document automation

Document automation usually fills templates.

Thought Tree can analyse, generate, review, revise and improve the structure of the work itself.

This positioning gives the framework a distinct commercial role:

Thought Tree is a production framework for structured cognitive work, not merely an interface for generating text.

## 13. Revenue Opportunities

Potential commercial models include:

Vertical AI products

Products built around specific Thought Tree workflows, such as documentation generation, compliance preparation or bespoke content generation.

Enterprise licensing

Licensing the Cognitive Engine, module library tools or private deployment infrastructure.

Managed workflow services

Running Thought Tree workflows on behalf of clients.

Module marketplaces

Selling reusable modules, contracts, validators and workflow packs.

Consulting and implementation

Helping organisations convert existing processes into Thought Tree programs.

Creative media platforms

Offering personalised story, game, comic or interactive media generation.

Developer tools

Providing authoring, testing, debugging and visualisation tools for Thought Tree programs.

Governance and audit tooling

Providing traceability, validation, compliance and review infrastructure for LLM-assisted work.

Private module libraries

Helping organisations build and maintain proprietary cognitive process libraries.

Benchmarking and optimisation services

Comparing models, prompts, modules and execution strategies across standardised workflows.

#### 14. Strategic Commercial Value

The long-term commercial value of the Thought Tree Framework is that it allows cognitive work to be treated as reusable software.

Instead of each LLM interaction being a one-off prompt, organisations can create durable cognitive programs:

versioned;  
tested;  
reviewed;  
traceable;  
reusable;  
improvable;  
model-independent.

This has strategic value because many organisations do not merely need AI-generated outputs. They need reliable AI-assisted processes.

The framework therefore creates value at three levels:

Output value

Producing useful documents, media, reports, plans or structured data.

Process value

Making production workflows repeatable, inspectable and improvable.

Library value

Accumulating reusable cognitive modules as organisational intellectual property.

The most important commercial insight is that Thought Tree modules can become assets in their own right. A mature module library represents encoded expertise: not just what an organisation knows, but how it transforms knowledge into useful outputs.

This is the basis for a defensible commercial strategy.

## 14. Roadmap

The Thought Tree Framework is intended to evolve from a structured LLM workflow format into a broader cognitive programming framework: a way to define, compile, execute, validate and improve cognitive processes made from concepts, relationships and transformations.

The roadmap is divided into staged development phases. Each phase strengthens a different part of the framework: the conceptual model, the execution engine, the authoring format, validation, tooling, libraries and commercial production readiness.

Phase 1: Formalise the Cognitive Programming Model

The first priority is to clearly define the abstract model behind the framework, independent of any particular syntax or engine implementation.

The current framework already contains the essential components: Modules, Operations, Data Units, Collections, Iterators, TTML and the Cognitive Engine. The next step is to reframe these as parts of a cognitive programming model.

Goals

Define a Thought Tree Program as an executable cognitive process.

Define a Module as a reusable cognitive program or function.

Define an Operation as a transformation.

Define a Data Unit as a concrete artefact representing a concept.

Define semantic types for Data Units.

Define relationships between artefacts and operations.

Define contracts for expected output quality and structure.

Define the Cognitive Engine as compiler, runtime and execution manager.

Distinguish the abstract program model from TTML as one serialisation format.

Key Deliverables

Updated conceptual documentation.

Revised terminology and glossary.

Formal definition of:

Concept;

Data Unit;

Semantic Type;  
Operation;  
Transformation;  
Module;  
Collection;  
Contract;  
Execution Trace;  
Cognitive Transformation Graph.

Updated diagrams showing:

source definition → compiled graph → execution → validated outputs;  
relationship between TTML, Cognitive Engine and execution backends.

Success Criteria

This phase is complete when the framework can be explained without relying on XML-specific language, and TTML can be described clearly as one representation of a deeper cognitive program model.

Phase 2: Stabilise TTML as the Initial Source Format

TTML should remain the first formal source format for Thought Tree programs. The current schema already supports the core workflow model, but it should be refined to better support cognitive programming concepts.

Goals

Stabilise the TTML schema.

Improve schema clarity and documentation.

Add support for semantic types and contracts.

Clarify operation attributes.

Formalise Collections, Iterators and dependency resolution.

Improve conformance requirements.

Ensure TTML remains both human-readable and LLM-authorable.

Proposed Enhancements

Possible future additions to TTML include:

```
<File
 id="plot_outline"
 extension="md"
 semanticType="PlotOutline"
 contract="ThreeActPlotOutlineContract"/>
```

and:

```
<Operation
 id="ReviewPlotOutline"
 type="TextCompletion"
 desc="Review the plot outline for structure, pacing and continuity."
 contract="NarrativeReviewContract"/>
```

The aim is not to overcomplicate TTML, but to allow richer execution validation where needed.

Key Deliverables

TTML schema version 1.0.

Updated XML Schema Definition.

TTML authoring guide.

TTML validation guide.

Minimal example TTML.

Complex example TTML.

Conformance test files.

Clear versioning policy.

Success Criteria

This phase is complete when a Cognitive Engine can reliably parse and validate TTML modules, and when humans or LLMs can author valid TTML using clear guidance.

### Phase 3: Build a Reference Cognitive Engine

A reference Cognitive Engine is needed to prove the framework, validate the execution semantics and provide a baseline implementation for future engines.

The reference engine does not need to be commercially complete at first. Its purpose is to demonstrate that Thought Tree programs can be loaded, compiled, executed, traced and improved.

#### Goals

Load TTML.

Validate TTML against the schema.

Resolve inputs, variables, iterators and collections.

Compile TTML into an internal execution graph.

Execute operations in the correct order.

Support basic LLM completions.

Support deterministic function execution.

Support pre-existing submodules.

Preserve intermediate outputs.

Record execution traces.

Return final outputs.

#### Initial Required Features

The first reference engine should support:

TextCompletion;

ExecuteFunction;

PreExisting;

root-level inputs;

variables;

iterators;

collections;

final outputs;

workspace management;

output collision detection;

execution trace generation.

#### Later Engine Features

After the basic engine is stable, add:

DynamicCompletion;

ProjectCompletion;

generated submodules;

semantic contract validation;

retry logic;

output repair;

human review pauses;

module versioning;

caching;

parallel execution;  
cost tracking;  
model selection.

#### Key Deliverables

Reference Cognitive Engine prototype.

CLI runner.

Workspace format.

Execution trace format.

Function registry.

Example function implementations:

ConcatenateFiles;

CopyFile;

ConvertDocumentToText;

ValidateXMLAgainstXSD;

ValidateTTMLWithExecutor;

CreateArchive.

Example end-to-end executions.

#### Success Criteria

This phase is complete when the reference engine can execute the main example modules and produce final outputs, intermediate artefacts and execution traces.

#### Phase 4: Define the Cognitive Intermediate Representation

To support compilation, validation, optimisation and multiple source syntaxes, the Cognitive Engine should convert TTML into an internal representation before execution.

This internal representation should be a graph of artefacts, transformations, dependencies and contracts.

#### Goals

Define a standard internal model for execution.

Represent Modules as executable graphs.

Represent Operations as transformation nodes.

Represent Data Units as artefact nodes.

Represent dependencies as edges.

Represent Collections as grouped artefact sets.

Represent Contracts as validation requirements.

Represent execution state separately from source definition.

#### Compilation Pipeline

The intended compilation process is:

Source Module



Parse



Normalise



Resolve Symbols



Expand Iterators



Build Dependency Graph





Validate Dependencies and Contracts



Create Execution Plan



Execute



Validate Outputs



Record Trace

Key Deliverables

Internal representation specification.

Dependency graph model.

Operation instance model.

Artefact metadata model.

Contract attachment model.

Iterator expansion model.

Collision detection rules.

Graph visualisation prototype.

Success Criteria

This phase is complete when a TTML module can be compiled into a clear, inspectable execution graph before it is run.

#### Phase 5: Add Semantic Types and Contracts

The framework should distinguish between producing a file and producing a valid cognitive artefact.

For example, producing plot\_outline.txt is not enough. The output should satisfy the expectations of a PlotOutline.

#### Goals

Define semantic types for common artefacts.

Define contracts for required structure, quality and completeness.

Support deterministic validation functions.

Support LLM-assisted validation.

Support human approval gates.

Make contract failure visible in the execution trace.

Allow operations to retry, repair or escalate when contracts fail.

#### Example Semantic Types

Initial semantic types may include:

Summary;

Analysis;

RequirementsRegister;

PlotOutline;

CharacterProfile;

SettingOverview;

ReviewReport;

CorrectionPlan;

TechnicalDesignDocument;

RiskRegister;

PolicyDocument;

TTMLModule.

#### Example Contract Types

Contracts may define:

- required sections;
- expected format;
- completeness criteria;
- forbidden content;
- citation or traceability requirements;
- consistency requirements;
- quality rubric;
- validation function;
- LLM review prompt;
- human approval requirement.

Key Deliverables

Semantic type specification.

Contract specification.

Contract registry format.

Built-in contracts for common examples.

Validation gate execution semantics.

Contract failure handling model.

Success Criteria

This phase is complete when operation outputs can be validated for semantic correctness, not just existence.

Phase 6: Improve Authoring and Developer Experience

Raw TTML is useful as a standard interchange format, but authoring complex cognitive programs directly in XML may become difficult.

The framework should support improved authoring tools while preserving TTML as a stable serialisation format.

Goals

Make Thought Tree programs easier to write.

Make Thought Tree programs easier to inspect.

Support LLM-assisted module authoring.

Support visual editing.

Support schema-aware validation.

Support reusable module templates.

Support conversion between formats.

Possible Authoring Interfaces

TTML XML files.

YAML-based cognitive program syntax.

JSON representation.

Visual graph editor.

Form-based module builder.

LLM-assisted authoring interface.

IDE extension.

Module validation CLI.

Key Deliverables

Authoring guide.

Module design guide.

TTML formatter.

TTML linter.

TTML validator.

Example templates.

Optional YAML prototype.

Visual graph prototype.

LLM prompt pack for TTML authoring.

Success Criteria

This phase is complete when users can design, validate and revise Thought Tree modules without needing to manually manage every XML detail.

Phase 7: Testing, Evaluation and Improvement

A cognitive programming framework needs a way to test whether modules work.

Testing should include structural validation, execution validation and semantic evaluation.

Goals

Define a test harness for Thought Tree modules.

Generate representative input bundles.

Run modules against test cases.

Validate outputs against contracts.

Compare outputs across module versions.

Compare outputs across models.

Support automated module improvement.

Preserve test results as traceable artefacts.

Test Types

The framework should support:

Schema tests

Does the module conform to the source format?

Execution tests

Can the engine resolve dependencies and produce final outputs?

Semantic tests

Do the outputs satisfy their intended contracts?

Regression tests

Does a revised module preserve or improve behaviour?

Stress tests

Does the module handle large, incomplete or ambiguous inputs?

Adversarial tests

Does the module handle contradictory, malformed or hostile inputs?

Key Deliverables

Module test specification.

Test input bundle format.

Evaluation criteria format.

Test execution function.

Comparison report format.

Automated improvement module.

Benchmark examples.

## Success Criteria

This phase is complete when a module can be tested, scored, revised and compared against previous versions.

## Phase 8: Module Libraries and Registries

The long-term value of the framework depends on reusable cognitive programs.

A module library would allow individuals, teams and organisations to build collections of reusable Thought Tree programs for recurring cognitive work.

## Goals

Define a module package format.

Support module metadata.

Support versioning.

Support dependency declarations.

Support function requirements.

Support contract requirements.

Support engine capability requirements.

Support public and private module libraries.

## Registry Types

The ecosystem may include:

Module registry;

Function registry;

Contract registry;

Semantic type registry;

Model capability registry;

Tool registry;

Prompt strategy registry.

## Example Module Metadata

A reusable module may declare:

Module ID

Version

Author

Description

Required TTML version

Required Cognitive Engine conformance level

Required functions

Required contracts

Inputs

Outputs

Semantic types

Dependencies

Test cases

Known limitations

Key Deliverables

Module package specification.

Module metadata schema.

Module versioning policy.

Local module registry.

Private organisational library.

Public example library.

Module search and selection prototype.

Success Criteria

This phase is complete when modules can be shared, versioned, searched, validated and reused across projects.

#### Phase 9: Dynamic Planning and Self-Improving Workflows

Dynamic planning is one of the most powerful features of the Thought Tree Framework. It allows a Cognitive Engine to generate submodules or execution plans for ambiguous tasks.

However, this power must be controlled through validation, review and traceability.

Goals

Support DynamicCompletion.

Support ProjectCompletion.

Generate submodules from high-level operation descriptions.

Validate generated modules before execution.

Optionally require human approval before execution.

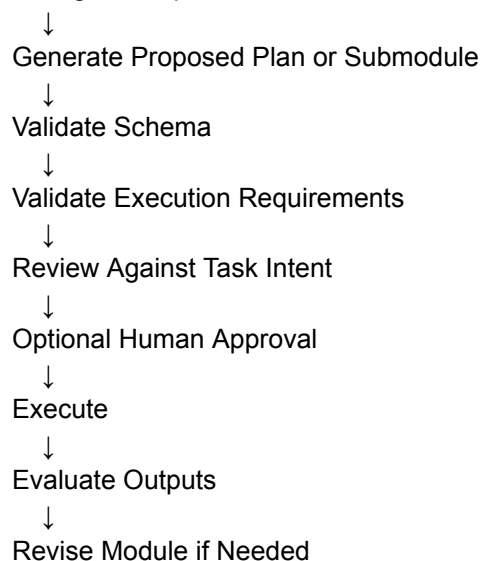
Retain generated modules as execution artefacts.

Use test cases to evaluate generated modules.

Improve modules over time.

Recommended Dynamic Planning Flow

Ambiguous Operation



Key Deliverables

Dynamic planning specification.

Generated submodule validation process.

Human approval policy.

Automated module improvement process.

Module comparison reports.

Self-improvement example workflows.

Success Criteria

This phase is complete when a Cognitive Engine can safely generate, validate, execute and improve submodules without losing auditability.

#### Phase 10: Safety, Security and Governance

As Thought Tree programs become more powerful, the framework must address safety, security, cost and governance concerns.

This is especially important for enterprise, compliance, legal, media production and automated publishing use cases.

#### Goals

- Define permission boundaries.
- Sandbox deterministic functions.
- Control tool access.
- Prevent unsafe file system access.
- Detect prompt injection risks.
- Control dynamic module generation.
- Track cost and resource usage.
- Support human approval policies.
- Support data privacy controls.
- Support audit and retention policies.

#### Key Areas

- Untrusted module execution.
- Prompt injection.
- External API access.
- File system access.
- Secrets management.
- Model provider privacy.
- Generated content governance.
- Human approval requirements.
- Cost limits.
- Loop termination safeguards.
- Trace retention.

#### Key Deliverables

- Security model.
- Permission model.
- Safe execution profile.
- Tool access policy.
- Human approval policy.
- Cost-control model.
- Governance checklist.
- Secure engine implementation guidance.

#### Success Criteria

This phase is complete when Thought Tree modules can be executed in controlled environments with clear permission, audit and safety boundaries.

#### Phase 11: Production-Grade Cognitive Engines

Once the framework and reference engine are stable, production-grade Cognitive Engines can be built for real-world use.

Different engines may optimise for different contexts:

- local execution;
- server execution;
- enterprise governance;
- creative production;

compliance workflows;  
research automation;  
distributed execution.

#### Goals

Support scalable execution.  
Support robust error recovery.  
Support multiple model providers.  
Support model routing.  
Support caching and incremental reruns.  
Support parallel execution.  
Support human review workflows.  
Support dashboards and visualisation.  
Support module libraries.  
Support detailed tracing and audit export.

#### Key Deliverables

Production engine architecture.  
API specification.  
CLI interface.  
Web interface.  
Visual execution dashboard.  
Model provider adapters.  
Tool adapters.  
Function registry integration.  
Module registry integration.  
Enterprise audit export.

#### Success Criteria

This phase is complete when the framework can support real production workflows with reliability, traceability and manageable cost.

#### Phase 12: Commercial and Ecosystem Development

The final phase is to develop the ecosystem around the framework.

This includes commercial products, public standards, community modules, private organisational libraries and specialised applications.

#### Potential Commercial Directions

Bespoke entertainment media generation.  
Creative production pipelines.  
Technical documentation automation.  
Compliance and audit preparation.  
Legacy project recovery.  
Requirements engineering.  
Game content generation.  
Organisational knowledge transformation.  
Research monitoring and reporting.  
Policy and regulatory analysis.  
Cognitive workflow consulting.  
Module library licensing.  
Hosted Cognitive Engine services.

#### Ecosystem Goals

Publish the TTML standard.  
Provide a reference engine.

Provide module examples.  
Encourage third-party engines.  
Encourage public module libraries.  
Support private enterprise module libraries.  
Support certification or conformance testing.  
Build tooling around authoring, testing and improvement.

#### Key Deliverables

Public documentation site.  
Open-source reference implementation, if desired.  
Example module repository.  
Commercial engine or platform.  
Module marketplace or library.  
Conformance test suite.  
Case studies.  
Developer guides.  
Enterprise adoption guide.  
Success Criteria

This phase is complete when Thought Tree becomes usable not only as a framework specification, but as a practical ecosystem for building, sharing and executing cognitive programs.

#### Summary Roadmap

The overall development path is:

1. Formalise the cognitive programming model
2. Stabilise TTML as the first source format
3. Build a reference Cognitive Engine
4. Define the internal compilation graph
5. Add semantic types and contracts
6. Improve authoring and developer experience
7. Add testing, evaluation and improvement
8. Build module libraries and registries
9. Support dynamic planning and self-improvement
10. Add safety, security and governance
11. Develop production-grade Cognitive Engines
12. Build the commercial and open ecosystem

The central objective is to move from structured LLM workflows toward a durable cognitive programming framework: one where complex work can be defined, compiled, executed, inspected, validated, reused and improved over time.

## 15. Glossary

### ## Abstract Program Model

The conceptual model underlying the Thought Tree Framework, independent of any particular file format such as TTML. It defines the core entities of the framework: Modules, Operations, Data Units, Concepts, Relationships, Contracts, Collections, execution traces and Cognitive Engines.

---



### ## Active Iterator

An iterator that causes an Operation to expand into multiple concrete operation instances.

An iterator becomes active when it is referenced directly by an Operation's input, output, CollectionRef identifier, operation description or other executable field supported by the Cognitive Engine.

Example:

```
```xml
<File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
```
```

If `ChapterIterator` ranges from 1 to 12, the Operation may execute once for each chapter.

---

### ## Artefact

A produced or supplied item used during execution. In the Thought Tree Framework, an artefact is usually represented as a Data Unit.

Examples include:

- source notes;
- generated summaries;
- character profiles;
- plot outlines;
- validation reports;
- generated TTML modules;
- execution traces;
- final documents.

---

### ## Cognitive Engine

The software system that loads, validates, compiles, executes and manages Thought Tree programs.

A Cognitive Engine is responsible for turning a human-readable Thought Tree definition into concrete execution steps. It may call LLMs, deterministic functions, external tools, submodules and human review processes.

The Cognitive Engine acts as both:

#### 1. **\*\*Compiler\*\***

It parses the source definition, resolves references, expands iterators, validates dependencies, detects collisions and builds an executable plan.

#### 2. **\*\*Runtime\*\***

It executes operations, manages the workspace, stores intermediate artefacts, handles errors, validates outputs and records the execution trace.

---

## ## Cognitive Program

An executable specification for cognitive work.

A cognitive program describes how inputs should be transformed into outputs through explicit concepts, relationships, operations, contracts and review steps.

A Thought Tree Module is a type of cognitive program.

Unlike a single prompt, a cognitive program describes not only the desired output, but also the structured process by which that output should be produced, validated and improved.

---

## ## Cognitive Programming

A programming approach in which the primary objects are concepts, documents, requirements, plans, reviews, decisions and transformations between them.

In cognitive programming, LLMs are used as semantic transformation engines, while deterministic software functions are used for mechanical or rule-based tasks. The Cognitive Engine coordinates both.

Cognitive programming aims to make LLM-assisted work:

- modular;
- inspectable;
- reusable;
- auditable;
- model-independent;
- improvable.

---

## ## Cognitive Source Language

A human-authorable language used to define Thought Tree programs.

TTML is the current source language and serialisation format for Thought Tree Modules. Future source languages may include YAML, JSON, visual graph editors or higher-level domain-specific languages.

---

## ## Cognitive Transformation Graph

The internal graph representation produced when a Cognitive Engine compiles a Thought Tree program.

In this graph:

- Data Units are artefact nodes;
- Operations are transformation nodes;
- dependencies are edges;
- Collections are grouped artefact references;
- Contracts are validation requirements;
- Modules are reusable graph fragments.

Although the framework is called “Thought Tree,” advanced execution may form a directed graph rather than a strict tree, especially when loops, shared dependencies, dynamic submodules or conditional execution are used.

---

### ## Collection

A named group of Data Units that can be treated as a single logical input or output.

Collections are useful when an Operation needs to consume or produce multiple related artefacts without expanding once per member.

Example:

```
``xml
<Collection id="all_character_profiles" orderBy="CharacterIterator">
 <FileRef id="character_{{CharacterIterator}}_profile"/>
</Collection>
``
```

A Collection may represent:

- all generated character profiles;
- all revised chapters;
- all test case reviews;
- all evidence files;
- all feedback items for a given output.

---

### ## Collection Iterator

An iterator used inside a Collection definition to enumerate the members of that Collection.

A Collection Iterator does not necessarily cause the consuming Operation to expand.

For example, if an Operation consumes:

```
``xml
```

```
<CollectionRef id="feedback_for_chapter_{{ChapterIterator}}"/>
...

```

then `ChapterIterator` may be active for the Operation, while an internal `PersonalIterator` used inside the Collection may only determine the Collection members.

---

## ## CollectionRef

A reference to a previously declared or generated Collection.

Example:

```
...xml
<CollectionRef id="all_revised_chapters"/>
...

```

A Cognitive Engine resolves a CollectionRef into the ordered set of Data Units contained in the Collection.

---

## ## Compiler

The part of the Cognitive Engine responsible for turning a Thought Tree source definition into an executable plan.

Compilation may include:

- parsing;
- schema validation;
- symbol resolution;
- variable resolution;
- iterator expansion;
- Collection resolution;
- dependency analysis;
- output collision detection;
- contract checking;
- execution planning.

The compiler does not necessarily execute the operations. It prepares the workflow for execution.

---

## ## Concept

A meaningful entity or idea represented within a Thought Tree program.

Examples:

- Character;

- Setting;
- Plot Outline;
- Requirement;
- Risk;
- Policy Obligation;
- Evidence Item;
- Game Mechanic;
- Technical Design Document;
- Review Report.

Concepts may be represented concretely as Data Units, but the Concept is the semantic meaning, not merely the storage format.

For example:

```
```text
plot_outline.txt
```
```

is a file, while:

```
```text
PlotOutline
```
```

is the concept or semantic type represented by that file.

---

## ## Conformance

The degree to which a Cognitive Engine correctly implements the Thought Tree Framework and TTML execution semantics.

A conformant engine should be able to:

- load Thought Tree definitions;
- validate their structure;
- resolve inputs, variables, iterators and references;
- execute supported Operation types;
- produce declared outputs;
- detect unresolved dependencies;
- detect output collisions;
- record execution traces.

Higher conformance levels may include semantic contracts, dynamic planning, generated submodules, human review, module testing and self-improvement.

---

## ## Contract

A formal or semi-formal specification of what a Data Unit or Operation output must satisfy.

Contracts allow the framework to distinguish between:

1. a file merely existing;
2. a file being structurally valid;
3. a file being semantically fit for purpose.

A Contract may define:

- expected semantic type;
- required sections;
- required structure;
- formatting rules;
- completeness criteria;
- quality criteria;
- validation functions;
- LLM review rubrics;
- human approval requirements.

Example:

A `PlotOutline` contract might require:

- premise;
- act structure;
- major turning points;
- central conflict;
- climax;
- resolution;
- character arc integration.

---

## ## Data Unit

A discrete unit of information supplied to, produced by or referenced by a Thought Tree Operation.

A Data Unit may be a text file, JSON object, XML document, image, audio file, database record, vector search result, generated prompt, validation report or any other processable artefact.

A Data Unit should ideally include:

- identifier;
- content;
- physical representation;
- semantic type;
- contract;
- version;
- provenance.

In TTML, Data Units are commonly represented using `` and `` elements.

---

## ## Dependency

A required relationship between an Operation and a Data Unit, Collection, function, tool or submodule.

If an Operation references a Data Unit as input, that Data Unit must be available before the Operation can execute.

Example:

```
``xml
<Operation id="ReviewOutline" type="TextCompletion" desc="Review the plot outline.">
 <FileRef id="plot_outline"/>
 <Output>
 <File id="plot_outline_review" extension="txt"/>
 </Output>
</Operation>
``
```

Here, `ReviewOutline` depends on `plot\_outline`.

---

## ## Deterministic Function

A non-LLM software function that performs a predictable operation.

Examples:

- concatenate files;
- copy files;
- validate XML;
- convert document formats;
- create an archive;
- query a database;
- resize an image;
- calculate a checksum.

Deterministic functions are used when ordinary software is more appropriate than LLM reasoning.

In TTML, these are usually invoked with an `ExecuteFunction` Operation.

---

## ## DynamicCompletion

An Operation type in which the Cognitive Engine decides how best to complete the task.

A DynamicCompletion may resolve into:

- a direct LLM completion;
- a generated submodule;
- a call to a pre-existing Module;
- one or more deterministic functions;
- a tool call;
- a human review process;
- or a combination of these.

Because DynamicCompletion introduces ambiguity, the selected strategy should be recorded in the execution trace.

---

## ## Edge

A connection in the Cognitive Transformation Graph.

Edges commonly represent relationships such as:

- depends on;
- produces;
- reviews;
- revises;
- validates;
- derives from;
- aggregates;
- decomposes into.

---

## ## ExecuteFunction

An Operation type that invokes a deterministic software function rather than an LLM.

Example:

```
```xml
<Operation id="CompileNovel"
  type="ExecuteFunction"
  desc="Concatenate the revised chapters."
  function="ConcatenateFiles">
  <CollectionRef id="all_revised_chapters"/>
  <Output>
    <File id="compiled_novel" extension="txt"/>
  </Output>
</Operation>
```
```

---

## ## Execution Plan



The concrete plan generated by the Cognitive Engine after parsing, resolving and validating a Thought Tree program.

An execution plan may include:

- concrete operation instances;
- resolved input references;
- resolved output identifiers;
- iterator expansions;
- dependencies;
- execution order;
- parallelisable branches;
- validation gates;
- required functions;
- required models;
- required tools;
- human review points.

---

### ## Execution Trace

A record of what happened during a Thought Tree execution.

The execution trace supports auditability, debugging, reproducibility and improvement.

It may include:

- source module version;
- input identifiers;
- input hashes;
- resolved variables;
- iterator expansions;
- operation start and end times;
- operation status;
- LLM provider and model;
- model settings;
- generated prompts;
- function calls;
- tool calls;
- submodule paths;
- output identifiers;
- output hashes;
- validation results;
- errors;
- retries;
- human review decisions;
- final outputs.

---

### ## Execution Validation

Validation that determines whether a structurally valid Thought Tree program can actually be executed by a Cognitive Engine.

Execution validation may check:

- whether FileRefs can be resolved;
- whether variables exist;
- whether iterators are valid;
- whether output collisions occur;
- whether referenced functions are available;
- whether target submodules exist;
- whether final outputs can be produced;
- whether required tools, models or permissions are available.

Execution validation is distinct from schema validation and semantic validation.

---

### ## File

In TTML, a ``<File>`` element declares a Data Unit that is expected to exist or be produced.

Example:

```
``xml
<File id="plot_outline" extension="txt"/>
``
```

The `id` is a logical identifier. The Cognitive Engine maps this logical identifier to a concrete asset in the workspace.

---

### ## FileRef

In TTML, a ``<FileRef>`` element refers to a Data Unit that has already been declared, supplied or produced.

Example:

```
``xml
<FileRef id="plot_outline"/>
``
```

The Cognitive Engine must resolve the reference before the Operation using it can execute.

---

### ## Final Output

A Data Unit or Collection declared as the final result of a Module.

In TTML, final outputs are declared in the root-level ``<Output>`` section.

Example:

```
``xml
<Output>
 <FileRef id="final_edited_novel"/>
</Output>
``
```

A Module is not complete until its declared final outputs can be resolved.

---

### ## Function Registry

A catalogue of deterministic functions available to a Cognitive Engine.

A Function Registry may define:

- function name;
- purpose;
- accepted inputs;
- produced outputs;
- supported file types;
- deterministic behaviour;
- error conditions;
- version;
- permissions required.

Example function names:

- ``ConcatenateFiles``;
- ``CopyFile``;
- ``ConvertDocumentToText``;
- ``ValidateXMLAgainstXSD``;
- ``CreateArchive``.

---

### ## Generated Module

A Module created dynamically by a Cognitive Engine, usually with LLM assistance, to complete a complex or ambiguous task.

Generated Modules are commonly produced by ``ProjectCompletion`` or ``DynamicCompletion`` Operations.

Generated Modules should be retained as part of the execution trace because they explain how the engine decomposed and executed the task.

---

## ## Human Review

A workflow step in which a human inspects, approves, rejects, edits or resolves part of a Thought Tree execution.

Human review may be used for:

- approving generated plans;
- checking high-impact outputs;
- resolving output collisions;
- validating ambiguous decisions;
- correcting failed outputs;
- approving generated submodules;
- governance or compliance.

Human review decisions should be recorded in the execution trace.

---

## ## Identifier

A logical name used to refer to a Data Unit, Collection, Operation, Variable, Iterator or Module.

Identifiers should be stable, descriptive and unique within their relevant scope.

Example identifiers:

- `story\_requirements`;
- `plot\_outline`;
- `draft\_chapter\_1`;
- `all\_character\_profiles`;
- `ReviewDraftTDD`.

---

## ## Intermediate Artefact

A Data Unit produced during execution that is not necessarily a final output.

Intermediate artefacts are important because they make the workflow inspectable, reviewable and improvable.

Examples:

- source digest;
- requirements analysis;
- draft outline;
- review report;
- correction plan;
- validation report.

---

## ## Iterator

A named range of values used to repeat Operations or define families of Data Units.

Example:

```
```xml
<Iterator id="ChapterIterator" from="1" to="{{ChapterCount}}"/>
```
```

An iterator can be inserted into identifiers:

```
```xml
<File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
```
```

This may produce:

- `draft\_chapter\_1`;
- `draft\_chapter\_2`;
- `draft\_chapter\_3`;
- and so on.

---

## ## Iterator Expansion

The process by which a Cognitive Engine converts an iterator-dependent Operation into multiple concrete operation instances.

For example, an Operation that drafts chapters using `ChapterIterator` from 1 to 12 may expand into:

- `DraftChapter[ChapterIterator=1]`;
- `DraftChapter[ChapterIterator=2]`;
- ...
- `DraftChapter[ChapterIterator=12]`.

---

## ## LLM Transformation

An Operation that uses a Large Language Model to perform semantic work.

Examples:

- summarise;
- analyse;
- draft;
- review;

- classify;
- extract;
- revise;
- synthesize;
- plan;
- generate.

In TTML, LLM transformations are commonly represented by `TextCompletion`, `DynamicCompletion` or `ProjectCompletion` Operations.

---

## ## Model Registry

A catalogue of LLMs or other AI models available to a Cognitive Engine.

A Model Registry may include:

- provider;
- model name;
- capabilities;
- context length;
- supported modalities;
- cost profile;
- latency profile;
- privacy characteristics;
- supported parameters;
- version information.

A Thought Tree Module should ideally be model-independent, but the Cognitive Engine may use the Model Registry to choose an appropriate model for each Operation.

---

## ## Module

A reusable cognitive program that transforms declared inputs into declared outputs through a sequence or graph of Operations.

A Module may:

- receive Data Units as inputs;
- execute Operations;
- call deterministic functions;
- call LLMs;
- call tools;
- execute submodules;
- generate submodules;
- validate outputs;
- produce final Data Units.

In TTML, a Module is represented by a single `<TTML>` document.

---

## ## Module Library

A collection of reusable Thought Tree Modules.

A Module Library may be:

- private;
- organisational;
- commercial;
- open-source;
- domain-specific;
- engine-specific;
- version-controlled.

Module Libraries allow cognitive processes to be reused, improved and shared.

---

## ## Module Registry

A searchable catalogue of available Modules.

A Module Registry may store:

- module ID;
- version;
- author;
- description;
- required inputs;
- produced outputs;
- required functions;
- required contracts;
- supported schema version;
- dependencies;
- test results;
- quality ratings.

---

## ## Operation

A transformation step inside a Module.

An Operation consumes zero or more inputs and produces one or more outputs.

Operations may be performed by:

- LLMs;
- deterministic functions;

- tools;
- humans;
- submodules;
- generated execution plans.

In TTML, an Operation has:

- ``id``;
- ``type``;
- ``desc``;
- input ``FileRef`` and/or ``CollectionRef`` elements;
- output ``File`` and/or ``Collection`` declarations;
- optional attributes such as ``target`` or ``function``.

---

### ## Operation Instance

A concrete executable instance of an Operation after variable substitution and iterator expansion.

For example, this Operation:

```
``xml
<Operation id="DraftChapter" type="TextCompletion" desc="Draft each chapter.">
 <Output>
 <File id="draft_chapter_{{ChapterIterator}}" extension="txt"/>
 </Output>
</Operation>
``
```

may compile into concrete instances such as:

- ``DraftChapter[ChapterIterator=1]``;
- ``DraftChapter[ChapterIterator=2]``;
- ``DraftChapter[ChapterIterator=3]``.

---

### ## Operation Type

The category of execution used by an Operation.

Current TTML Operation types include:

- ``TextCompletion``;
- ``DynamicCompletion``;
- ``ProjectCompletion``;
- ``PreExisting``;
- ``ExecuteFunction``.

Future Operation types may include:



- `HumanReview`;
- `ToolCall`;
- `Condition`;
- `Loop`;
- `ValidationGate`;
- `Schedule`;
- `WaitForInput`.

---

### ## Output Collision

A situation where two or more Operation instances attempt to write to the same logical output identifier.

Example:

If an Operation expands over both `ChapterIterator` and `PersonalIterator`, but the output only includes `ChapterIterator`, multiple persona-specific executions may try to produce the same chapter-level output.

Silent overwriting should be invalid by default. Valid resolutions include:

- include all active iterators in the output identifier;
- aggregate inputs into a Collection;
- create versioned outputs;
- explicitly allow overwrite;
- require human or automated resolution.

---

### ## Physical Representation

The concrete storage or encoding format of a Data Unit.

Examples:

- `.txt`;
- `.md`;
- `.json`;
- `.xml`;
- `.csv`;
- `.pdf`;
- `.png`;
- database record;
- vector embedding.

Physical representation is distinct from semantic type.

For example, both `plot\_outline.txt` and `risk\_register.txt` may be plain text files, but they represent different semantic types.

---

### ## PreExisting

An Operation type that executes an existing Thought Tree Module.

Example:

```
```xml
<Operation id="ConceptDevelopment"
  type="PreExisting"
  desc="Run the concept development subproject."
  target="/subprojects/ConceptDevelopment.ttml">
  <FileRef id="story_requirements"/>
  <Output>
    <File id="plot_outline" extension="txt"/>
  </Output>
</Operation>
```
```

The Cognitive Engine loads and executes the target Module, then maps the submodule outputs back to the parent workflow.

---

### ## Precondition

A requirement that must be satisfied before an Operation can execute.

Examples:

- required input Data Units exist;
- referenced Collection is non-empty;
- required function is available;
- required model capability exists;
- human approval has been granted;
- previous validation passed.

---

### ## ProjectCompletion

An Operation type that expands a complex or ambiguous task into a generated submodule.

A ProjectCompletion Operation allows the Cognitive Engine, often with LLM assistance, to create a new Thought Tree Module that decomposes the task into smaller Operations.

The generated submodule should be retained as part of the execution trace.

---

### ## Prompt

The instruction or context sent to an LLM to perform a completion.

In the Thought Tree Framework, prompts may be constructed by the Cognitive Engine using:

- Operation description;
- input Data Units;
- Collection contents;
- semantic contracts;
- output requirements;
- active iterator values;
- Module metadata;
- system instructions;
- model-specific formatting.

Generated prompts should be recorded in the execution trace where practical.

---

### ## Provenance

The recorded history of how a Data Unit was produced.

Provenance may include:

- producing Operation;
- input Data Units;
- source Module;
- model or function used;
- prompt;
- parameters;
- timestamp;
- validation result;
- human edits;
- previous version;
- output hash.

Provenance is essential for traceability, auditability and reproducibility.

---

### ## Relationship

A meaningful connection between Concepts, Data Units, Operations or Modules.

Examples:

- `derived from`;
- `depends on`;
- `produces`;
- `reviews`;
- `revises`;

- `validates`;
- `aggregates`;
- `decomposes`;
- `implements`;
- `references`.

Relationships are represented implicitly in TTML through FileRefs, Collections, Operation order and output declarations, but may become more explicit in future versions of the framework.

---

## ## Reproducibility

The degree to which a Thought Tree execution can be repeated with the same or similar results.

Because LLM outputs may be nondeterministic, exact reproduction is not always possible. However, the framework improves reproducibility by recording:

- source Module;
- inputs;
- hashes;
- prompts;
- model names;
- model settings;
- tool versions;
- function versions;
- execution trace;
- validation results.

The goal is to make the process repeatable and inspectable, even when generated content varies.

---

## ## Runtime

The part of the Cognitive Engine responsible for executing a compiled Thought Tree program.

Runtime responsibilities include:

- workspace management;
- LLM calls;
- function calls;
- tool calls;
- submodule execution;
- human review pauses;
- caching;
- retries;
- error handling;
- validation gates;
- output storage;
- trace recording.

---

## ## Schema Validation

Validation that checks whether a TTML document conforms to the TTML XML schema.

Schema validation confirms that the document is structurally valid XML according to the schema.

It does not guarantee that:

- all FileRefs can be resolved;
- functions exist;
- output collisions are avoided;
- outputs are semantically correct;
- the workflow is well-designed.

---

## ## Semantic Correctness

The degree to which a produced Data Unit satisfies its intended meaning, purpose and quality requirements.

For example, a file named `plot\_outline.txt` may exist and be valid text, but it is only semantically correct if it actually contains a usable plot outline satisfying the relevant contract.

Semantic correctness is usually checked through contracts, validators, LLM review, deterministic functions or human review.

---

## ## Semantic Type

The conceptual type of a Data Unit, distinct from its physical file format.

Examples:

- `PlotOutline`;
- `CharacterProfile`;
- `TechnicalRequirementRegister`;
- `RiskAssessment`;
- `PolicyDraft`;
- `EvidenceMap`;
- `ChapterDraft`;
- `ValidationReport`.

Semantic types allow a Cognitive Engine to reason about whether outputs are appropriate for downstream Operations.

---

## ## Semantic Validation

Validation that checks whether a Data Unit satisfies its intended semantic type and contract.

Semantic validation may involve:

- deterministic validators;
- schema checks;
- required section checks;
- rubric-based LLM review;
- comparison against source material;
- human approval.

---

## ## Source Definition

A human-readable file or document that defines a Thought Tree program before compilation.

TTML is the current source definition format.

Future source definitions may use:

- YAML;
- JSON;
- visual graph formats;
- natural language assisted authoring;
- domain-specific syntaxes.

---

## ## Submodule

A Module executed from within another Module.

Submodules allow complex workflows to be decomposed into reusable components.

A parent Module may call a submodule using a `PreExisting`, `ProjectCompletion` or `DynamicCompletion` Operation.

---

## ## Symbol Resolution

The process by which a Cognitive Engine resolves names and references in a Thought Tree program.

Symbol resolution may include:

- FileRefs;
- CollectionRefs;
- variables;
- iterators;
- function names;

- module targets;
- contract names;
- semantic types.

---

### ## TextCompletion

An Operation type that invokes an LLM to generate one or more output Data Units.

Example:

```
```xml
<Operation id="DraftOutline" type="TextCompletion" desc="Draft a plot outline.">
  <FileRef id="story_requirements"/>
  <Output>
    <File id="plot_outline" extension="txt"/>
  </Output>
</Operation>
```
```

The Cognitive Engine constructs an LLM request using the Operation description, inputs, output requirements and relevant execution context.

---

### ## Thought Graph

An alternative term for the compiled graph representation of a Thought Tree program.

The term emphasises that, although Modules may be visualised as trees, execution dependencies may form a graph.

---

### ## Thought Tree

A connected structure of Modules and Operations that represents a cognitive workflow.

A Thought Tree may be simple and linear, or complex and recursive. It may include submodules, generated modules, review loops, deterministic functions, LLM transformations and human review stages.

---

### ## Thought Tree Framework

The complete framework for defining, compiling, executing, reviewing and improving cognitive programs.

It includes:

- the abstract program model;
- Thought Tree Modules;
- TTML;
- Cognitive Engines;
- execution semantics;
- semantic contracts;
- module libraries;
- function registries;
- execution traces;
- authoring and improvement processes.

---

## ## Thought Tree Markup Language — TTML

The XML-based serialisation format for defining Thought Tree Modules.

TTML provides a structured way to declare:

- project metadata;
- inputs;
- variables;
- iterators;
- Collections;
- Operations;
- outputs.

TTML is intended to be readable by humans, authorable by LLMs and executable by Cognitive Engines.

---

## ## Thought Tree Module

A reusable unit of cognitive execution.

A Module defines:

- expected inputs;
- variables;
- iterators;
- Collections;
- Operations;
- final outputs.

A Module can be used as a standalone program or as a submodule within a larger Thought Tree.

---

## ## Tool

An external capability available to a Cognitive Engine.



Examples:

- web search;
- vector database retrieval;
- API call;
- file conversion service;
- code execution environment;
- image generation model;
- database query;
- rendering tool;
- project management system.

Tools are distinct from deterministic functions when they involve external systems, state, permissions or network access.

---

## ## Tool Registry

A catalogue of tools available to a Cognitive Engine.

A Tool Registry may define:

- tool name;
- purpose;
- inputs;
- outputs;
- permissions;
- authentication requirements;
- rate limits;
- side effects;
- version;
- safety constraints.

---

## ## Transformation

A change from one or more input Data Units or Concepts into one or more output Data Units or Concepts.

Transformations are the basic actions of cognitive programming.

Examples:

- source notes → summary;
- requirements → requirements register;
- plot outline → chapter draft;
- chapter draft + feedback → revised chapter;
- TTML draft → validated TTML;
- multiple files → archive.

In TTML, transformations are represented by Operations.

---

### ## Validation Gate

A point in the workflow where execution can only continue if a Data Unit or Operation output satisfies required validation criteria.

Validation gates may use:

- schema validation;
- deterministic functions;
- semantic contracts;
- LLM review;
- human approval.

---

### ## Variable

A named reusable value declared within a Module.

Variables may be used for:

- iterator ranges;
- file identifiers;
- output naming;
- operation descriptions;
- module configuration;
- engine-specific settings.

Example:

```
``xml
<Var name="ChapterCount" type="integer" value="12"/>
``
```

---

### ## Versioning

The process of preserving multiple versions of a Data Unit, Module, Contract or execution output.

Versioning supports:

- auditability;
- rollback;
- comparison;
- improvement;
- reproducibility;

- safe replacement.

Example versioned outputs:

```
```text  
draft_chapter_1.v1.txt  
draft_chapter_1.v2.txt  
draft_chapter_1.v3.txt  
```
```

---

## ## Workspace

The execution environment where a Cognitive Engine stores or references inputs, intermediate artefacts, generated files, logs, traces and final outputs.

A workspace may be implemented as:

- a local directory;
- cloud storage;
- a database;
- an in-memory context;
- a project repository;
- a hybrid asset store.

The workspace maps logical identifiers from the Thought Tree program to concrete stored assets.